

9. OSPF定义的4种类型链路的分类基础是什么?
10. 为什么OSPF报文比RIP报文传播速度更快?
11. BGP的目的是什么?
12. 试给出本章所讨论的两组多播路由选择协议的简要说明。

练习题

13. 试表示一个完全被隔离的主机的路由表。
14. 试表示一个连接局域网但未连接因特网的主机的路由表。
15. 如果路由器R1的路由表是如下表22.3, 试找出网络的拓扑结构。

表22.3 练习15的路由表

掩 码	网 络 地 址	下一跳地址	接 口
/27	202.14.17.224	—	m1
/18	145.23.192.0	—	m0
默认	默认	130.56.12.4	m2

16. 在图22.8中的路由器R1能否接收到目的地址为140.24.7.194的分组? 并说明其理由。
17. 在图22.8中的路由器R1能否接收到目的地址为140.24.7.42的分组? 并说明其理由。
18. 试表示图22.9中区域ISP的路由表。
19. 试表示图22.9中本地ISP1的路由表。
20. 试表示图22.9中本地ISP2的路由表。
21. 试表示图22.9中本地ISP3的路由表。
22. 试表示图22.9中本地ISP3的路由表。
23. 试比较距离向量路由选择和链路状态路由选择。
24. 一个路由器具有如下的路由表

```

Net1  4   B
Net2  2   C
Net3  1   F
Net4  5   G

```

如果该路由器从路由器C接收到下列的RIP报文, 那么路由表中的内容将会是怎样?

```

Net1  2
Net2  1
Net3  3
Net4  7

```

25. 通知N个网络的RIP报文中有多少个字节是空的?
26. 一个路由器具有如下的RIP路由表

```

Net1  4   B
Net2  2   C
Net3  1   F
Net4  5   G

```

试给出这个路由器发送的响应报文。

27. 试表示具有下列说明的自治系统:

a. 共有8个网络 (N1到N8);

b. 共有8个路由器 (R1到R8);

- c. N1、N2、N3、N4、N5和N6是以太网；
- d. N7和N8是点到点的广域网；
- e. R1连接到N1和N2；
- f. R2连接到N1和N7；
- g. R3连接到N2和N8；
- h. R4连接到N7和N8；
- i. R5连接到N6和N3；
- j. R6连接到N6和N4；
- k. R7连接到N6和N5；
- l. R8连接到N8和N5。

- 28. 试给出由OSPF看到的练习27中的自治系统的图形表示。
- 29. 练习27中的哪一个是过渡网络？哪一个是残桩网络？
- 30. 一个路由器使用DVMRP从接口2接收到源地址为10.14.17.2的一个分组。如果路由器转发该分组，在单播路由表中有关这个地址项目的内容是什么？
- 31. RPF实际上创建最短路径树吗？试说明之。
- 32. RPB实际上创建最短路径树吗？试说明之。树的叶子是什么？
- 33. RPM实际上创建最短路径树吗？试说明之。树的叶子是什么？

探究题

- 34. 如果你已访问过UNIX（或LINUX），试用*netsat*和*ifconfig*程序寻找你所连接的服务器路由表。
- 35. 查明你的ISP如何使用地址聚合和最长掩码匹配原则。
- 36. 查明你的IP地址是否地是域地址分配的部分。
- 37. 如果你使用路由器，求出路由表中列个数与列名。

第五部分 传输层

目标

传输层负责整个报文的进程到进程的传递。进程就是运行在主机上的应用程序。虽然网络层监管独立分组从源端到目的端的传递，但它不辨认这些分组之间的关系。它独立地对待每个分组，无论什么情况，好像每块都属于一个单独的报文。另一方面，传输层在从源端到目的端的层次中，监管差错控制和流量控制以确保全部报文完整地、按序地到达。

传输层负责报文从一个进程到另一个进程的传递。

计算机时常在同一个时间运行多个程序。因此，从源端到目的端的传递不仅是一个计算机到下一个计算机的传递，而且还是计算机上的一个特定进程到另一计算机上的一个特定进程的传递。所以，在OSI模型中，传输层头部必须包含一个称为服务点地址的地址类型，或在因特网和TCP/IP协议族中的端口号或端口地址。

传输层协议可以是无连接的，也可以是面向连接的。无连接传输层将每段作为一个独立的分组处理，将它传递到目的机器的传输层。面向连接传输层在传递分组前要与目的机器的传输层建立一个连接。当所有数据传输完成后，要终止这个连接。

在传输层，一个报文通常被划分成可传输的段。一个无连接协议（如UDP）对每一段独立处理。一个面向连接协议（如TCP和STCP）还要用序号生成这些段之间的关系。

像数据链路层一样，传输层也负责流量控制和差错控制。但是，在这一层的流量控制和差错控制是端到端的，而不是在单一一条链路上。我们将在本书的这部分讨论这些协议。UDP不包含流量控制和差错控制。另一方面，其他两个协议，TCP和SCTP使用滑动窗口进行流量控制，使用确认系统进行差错控制。

本书的第五部分专门讨论传输层以及该层提供的服务。

第23章 UDP、TCP和SCTP

本章开始给出传输层 (transport layer) 存在的基本理由, 即进程到进程传递的必要性。主要讨论这类传递产生的问题, 并讨论解决这些问题的方法。

在因特网模型中, 传输层有三种协议: UDP、TCP和SCTP。首先讨论UDP协议, 它是这三种中比较简单的一种协议。我们将看到如何使用缺乏另外两种协议的某些特性的这种简单的传输层协议。

然后讨论TCP协议, 它是一种复杂的传输层协议。我们将了解到如何将以前提到的概念应用于TCP中。并将有关TCP协议中拥塞控制和服务质量的讨论放在第24章, 因为这两个问题同样也适用于数据链路层和网络层。

最后, 我们讨论新的传输层协议SCTP, 它是为像多媒体这样的多接口与多流应用而设计的。

23.1 进程到进程的传递

数据链路层负责链路上的两个相邻节点之间的帧传递, 这称为节点到节点的传递。网络层负责两台主机之间的数据报传递, 这称为主机到主机的传递。因特网中的通信并不是定义为两个节点或两个主机之间的数据交换。实际的通信是发生在两个进程 (应用程序) 之间的。我们需要进程到进程的传递 (process-to-process delivery)。但是, 在任何时刻, 在源主机上可能运行着多个进程, 并且在目的主机上也运行着多个进程。为了完成传递过程, 需要一种机制将源主机上运行的某个进程的数据发送到目的主机上运行的对应进程上。

传输层负责进程到进程的传递, 即进程之间的分组传递以及部分消息传递。后面将会看到两个进程以客户/服务器的方式通信。图23.1表示了三种传送类型以及它们的应用范围。

传输层负责进程到进程的传递。

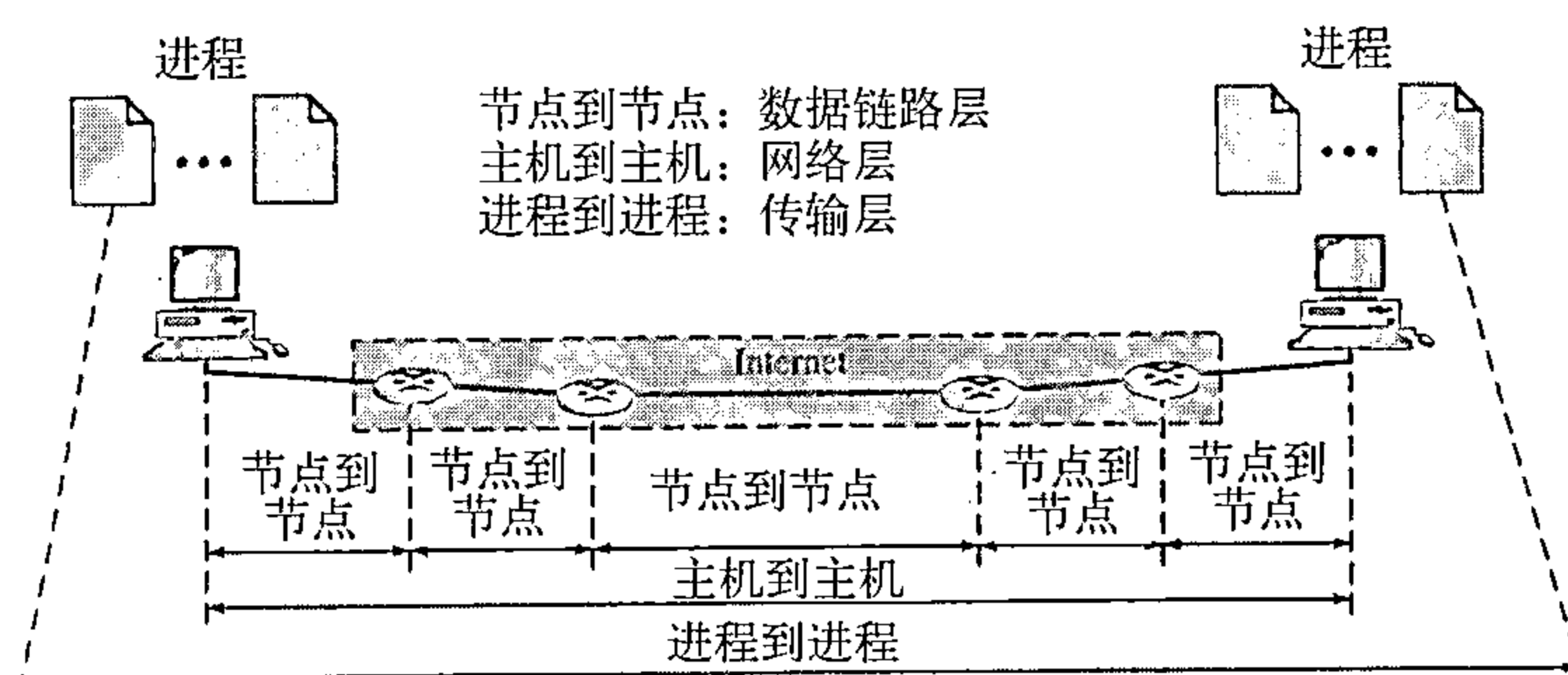


图23.1 数据传送类型

23.1.1 客户/服务器模式

虽然有多种方法可以实现进程到进程的通信, 但最常用的一个方法是通过客户/服务器模式 (client-server paradigm)。本地主机上的进程称为客户 (client), 它通常需要来自远程主机上的进程提供的服务, 这个远程主机称为服务器 (server)。

这两个进程（客户和服务端）有相同的名字。例如，如果要从远程机器上获得日期和时间，我们需要在本地主机上运行Daytime客户进程和在远程机器上运行Daytime服务器进程。

目前的操作系统支持多用户和多程序运行的环境。一个远程计算机在同一时间可以运行多个服务器程序，就像许多本地计算机可在同一时间运行一个或多个客户应用程序一样。对通信来说，我们必须确定：

1. 本地主机
2. 本地进程
3. 远程主机
4. 远程进程

寻址

每当需要传送信息到多个目的地之中的某一特定目的地时，就需要一个地址。在数据链路层，如果连接不是点到点的，则需要一个MAC地址从多个节点中选择一个节点。数据链路层的帧需要一个目的端MAC地址用于传送数据和一个源地址用于下一节点的回答。

在网络层，需要一个IP地址来选择数百万主机之中的一个主机。网络层中的数据报需要目的IP地址用于传送数据，需要源IP地址用于接收目的主机的回答。

在传输层，需要一个称为端口号（port number）的传输层地址，利用这一地址从目的主机上运行的多个进程中选择相应的进程。目的端口号用于传送，而源端口号用于接收回答。

在因特网模型中，端口号是在0~65 535之间的16位整数。客户程序用端口号定义它自己，这个端口号由运行在客户主机上的传输层软件随机选择的，这是临时端口号（ephemeral port number）。

服务器进程也必须用一个端口号定义它自己，但这个端口号不能随机选择。如果服务器站点的计算机运行一个服务器进程，并随机分配一个数字作为端口号，那么在客户站点的进程想访问该服务器并使用其服务时，将不知道此端口号。当然，一个解决方法是发送一个特殊分组，并请求一个特定服务器的端口号，但是这需要更多的开销。因特网决定给服务器使用全局端口号：它们称为熟知端口号（well-known port number）。但这条规则也有例外，例如，有一些客户端也被分配了熟知端口号。每一个客户进程都知道相应服务器进程的熟知端口号。例如，前面所讨论的Daytime客户进程可以使用临时（暂时）端口号52 000来表示自己，但服务器Daytime进程就必须使用熟知（永久的）端口号13。图23.2表示了这一概念。

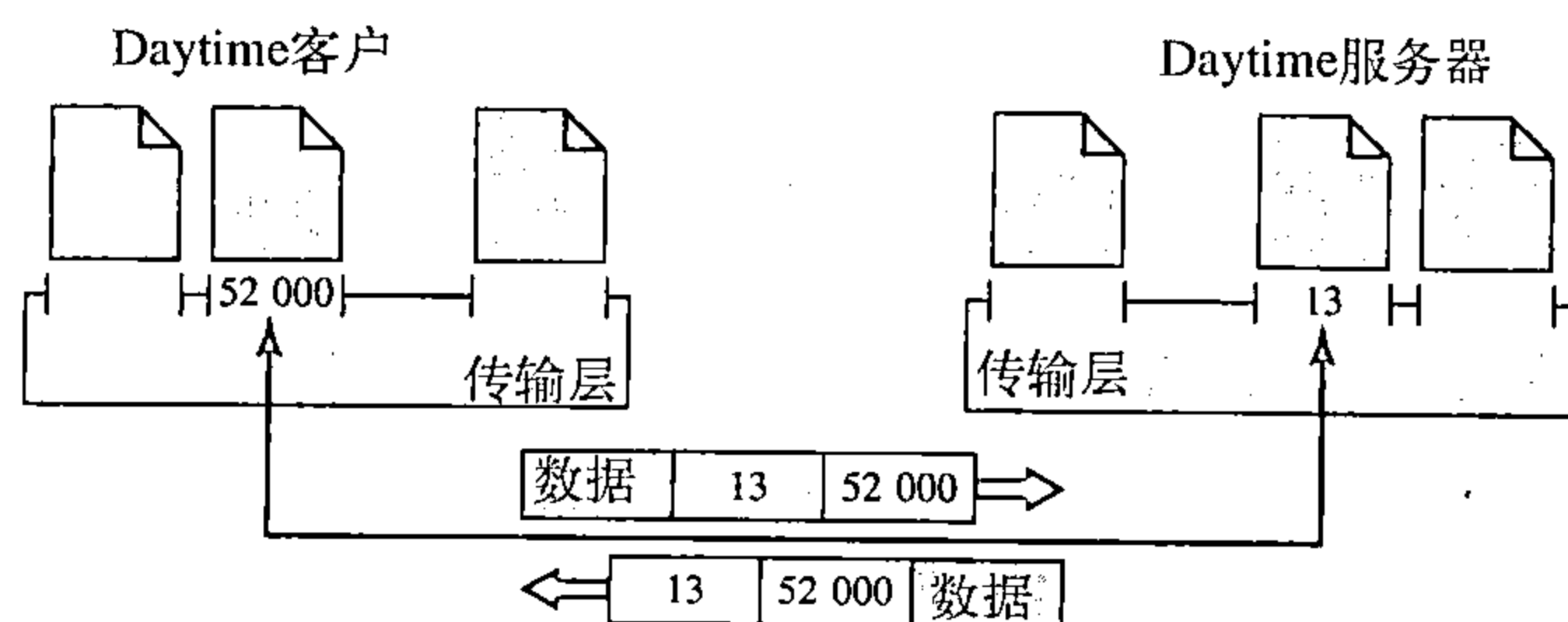


图23.2 端口号

现在必须搞清楚，在选择数据的最终目的端时，IP地址和端口号起着不同的作用。目的IP地址定义了在世界范围的不同主机中确定一个主机。但主机被选定后，端口号定义了在该特定主机上的多个进程中的一个进程（见图23.3）。

IANA范围

因特网号码分配管理局 (Internet Assigned Number Authority, IANA) 已经把端口编号划分为三种范围：熟知的、注册的 and 动态的 (或私有的)，如图23.4所示。

- 熟知端口。端口号的范围是0~1 023，由IANA分配和控制。这些是熟知端口。
- 注册端口。端口号的范围是1 024~49 151，IANA不分配或也不控制。它们可在IANA注册以防重复。
- 动态端口。端口号的范围是49 152~65 535。这一范围内的端口号既不受控制也不需要注册，可以由任何进程使用。它们是临时端口。

套接字地址

进程到进程的传递需要有两个标识符，IP地址和端口号，它们各用在一端以建立一条连接。一个IP地址和一个端口号结合起来称为套接字地址 (socket address)。客户套接字地址唯一地定义了客户机进程，而服务器套接字地址唯一地定义了服务器进程 (见图23.5所示)。

传输层协议需要一对套接字地址：客户套接字地址和服务器套接字地址。这四条信息是IP头部和传输层协议头部的组成部分。IP头部包含IP地址，而UDP或TCP头部包含端口号。

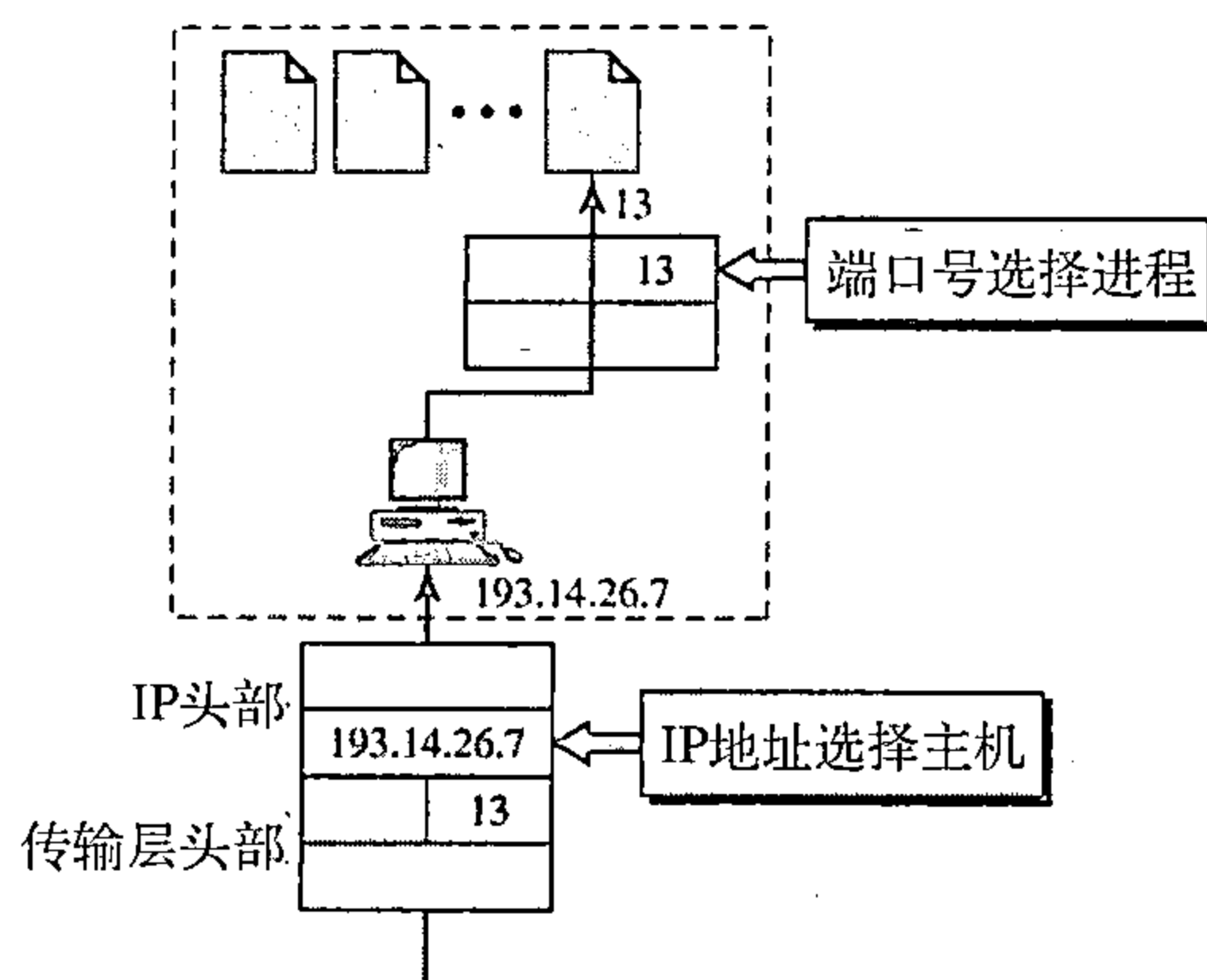


图23.3 IP地址与端口号

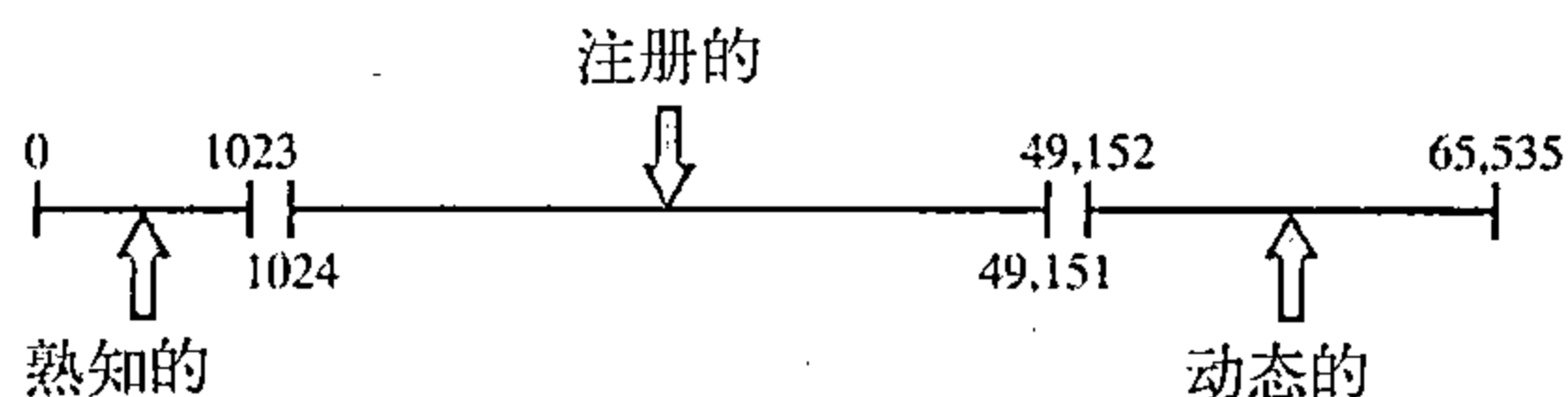


图23.4 IANA定义的范围

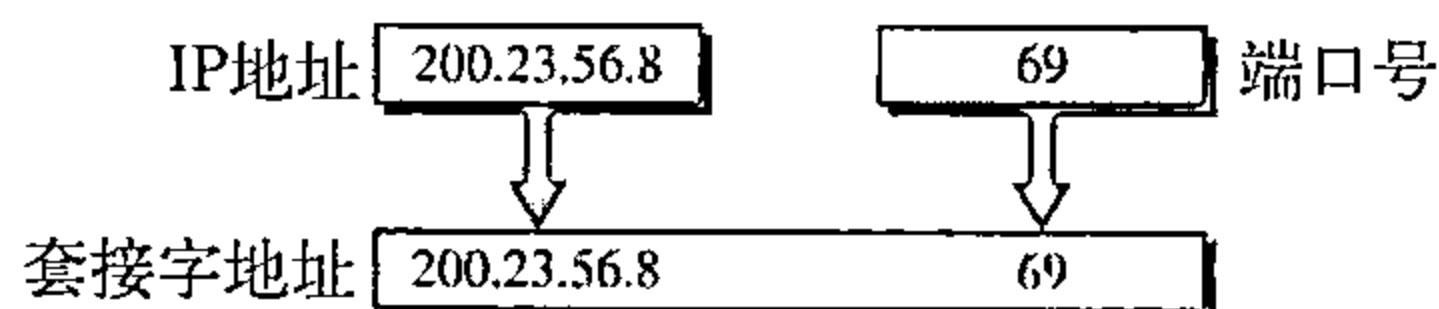


图23.5 套接字地址

23.1.2 复用与分离

寻址机制允许通过传输层进行复用和分离，如图23.6所示。

复用

在发送方站点，可能有多个进程要发送分组。但是，在任何时候只有一个传输层协议。这是一种多对一的关系，因而需要复用。传输层协议接收来自不同进程的报文，这些进程是由分配给它们的端口号进行区分。添加了头部以后，传输层把分组发送给网络层。

分离

在接收方站点，也只有一个传输层协议，但多个进程可能要接收分组。这是一对多的关系，因而需要分用。传输层接收来自网络层的数据报。经过纠错和去除头部以后，传输层根据端口号将每个报文传递到适当的进程。

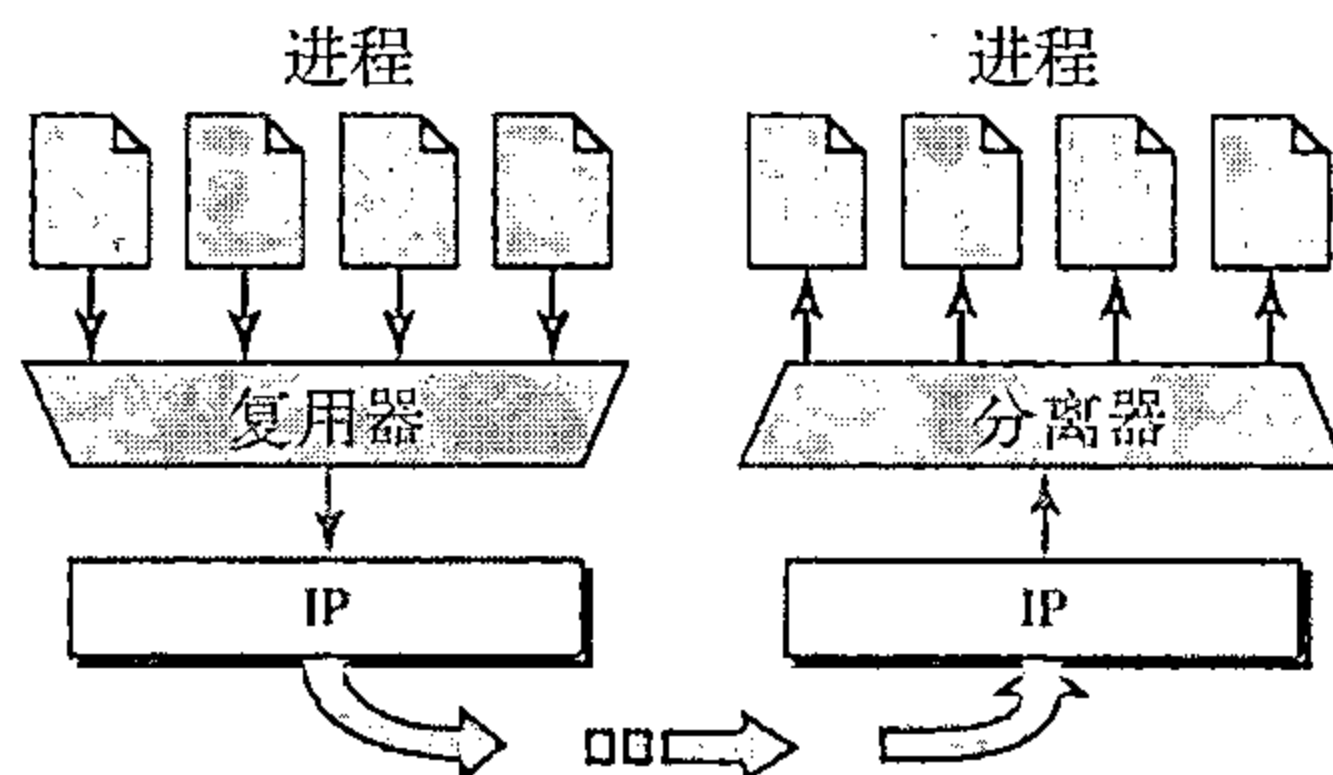


图23.6 复用与多路分离

23.1.3 无连接服务与面向连接的服务

传输层协议可以是无连接的或面向连接的服务。

无连接服务

在无连接服务 (connectionless service) 中, 分组从一方发送给另一方, 不需要建立连接和释放连接。分组没有编号; 它们可能被延迟、丢失或无序到达, 也没有确认过程。下面我们会看到, 在因特网模型中, 其中的一个传输层协议UDP就是无连接的。

面向连接的服务

在面向连接的服务 (connection-oriented service) 中, 首先在发送方和接收方之间建立一个连接, 然后传送数据, 最后释放连接。我们将看到TCP和SCTP都是一种面向连接的协议。

23.1.4 可靠服务与不可靠服务

传输层服务可以是可靠的或不可靠的。如果应用程序需要可靠性, 我们使用可靠的传输层协议, 通过在传输层实现流量控制和差错控制来获得这种可靠性。这意味着一种较慢和更复杂的服务。另一方面, 如果应用程序不需要可靠性, 因为它使用自己的流量和差错控制机制或者它需要快速服务或者服务的本质特性不要求流量和差错控制 (如实时应用), 那么就可以使用不可靠服务。

在因特网中, 有三种不同的传输层协议, 前面已提到过。UDP 是无连接的和不可靠的, 而TCP和SCTP则是面向连接的和可靠的。这三种协议都能与应用层程序的需要相对应。

常会想到一个疑问: 如果数据链路层是可靠的, 并且已经有了流量控制和差错控制, 那么还需要在传输层进行流量控制和差错控制吗? 回答是需要的。因为在数据链路层的可靠性存在于两个节点之间, 而我们需要端到端的可靠性。并且因为因特网的网络层是不可靠的 (尽力传递), 所以必须在传输层实现可靠性。要知道数据链路层的差错控制并不能保证传输层的差错控制, 让我们看一下图23.7。

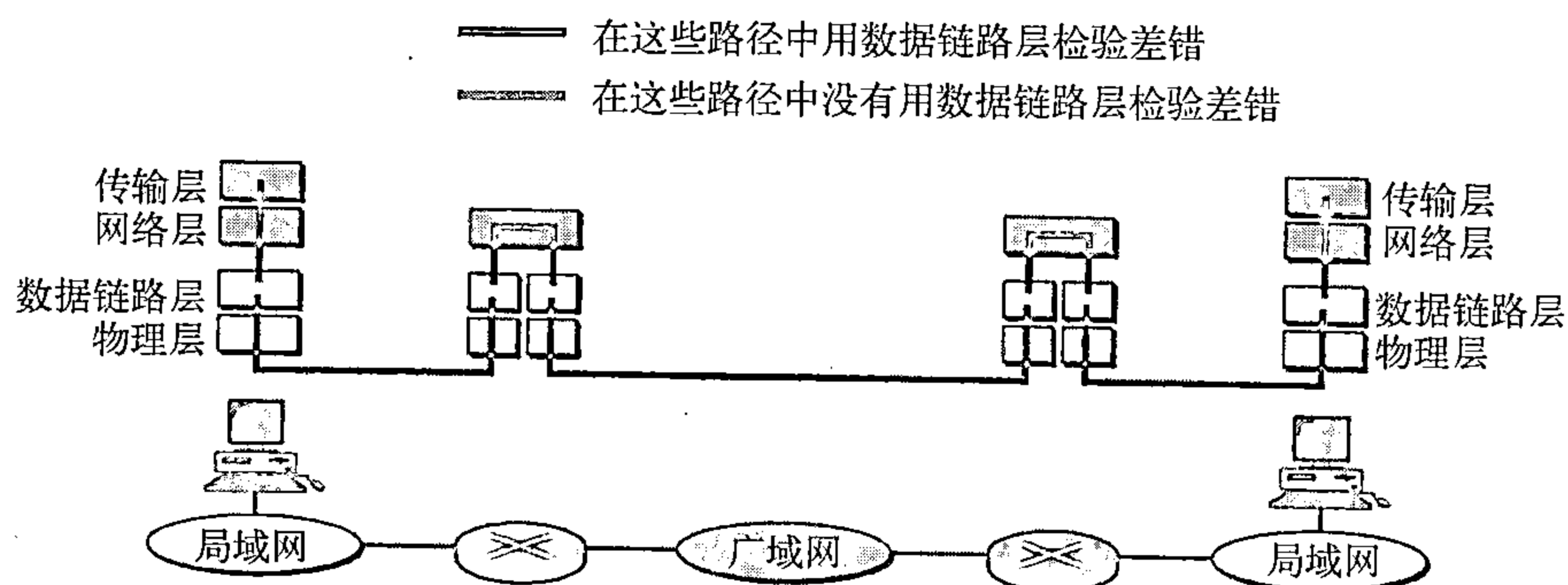


图23.7 差错控制

下面将看到TCP中的流量和差错控制是通过滑动窗口协议实现的, 就像我们在第11章中已经讨论过的那样。但是, 窗口是面向字符而不是面向帧的。

23.1.5 三种协议

原始的TCP/IP协议族在传输层只规定了两个协议: UDP和TCP。在讨论TCP以前, 我们首先关注这两个协议中较简单的UDP。然后在本章中也讨论新设计的传输层协议SCTP。图23.8表示该协议在TCP/IP协议族中的位置。

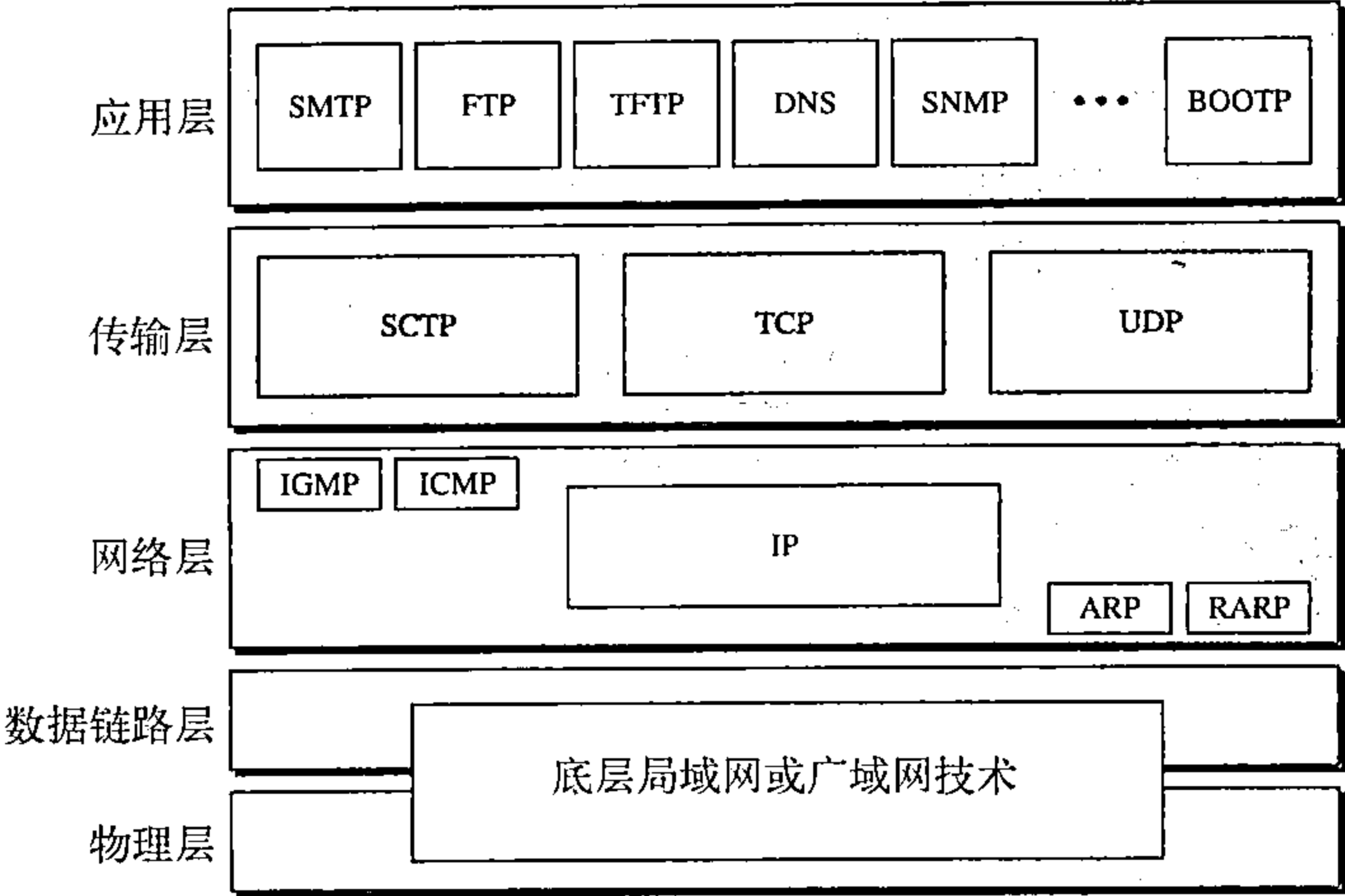


图23.8 在TCP/IP协议族中UDP、TCP和SCTP的位置

23.2 用户数据报协议 (UDP)

用户数据报协议 (User Datagram Protocol, UDP) 称为无连接不可靠传输层协议 (connectionless, unreliable transport protocol)。它除了提供进程到进程通信而不是主机到主机通信外，就没有给IP服务增加任何东西。此外，它还完成非常有限的差错检验。如果UDP功能是如此差，那么为什么进程还要使用它？它有缺点但也有优点。UDP是一个非常简单的协议，开销最小。如果一个进程想发送很短的报文，而且不在意可靠性，就可以使用UDP。使用UDP发送一个很短的报文，在发送方和接收方之间的交互要比使用TCP或SCTP时少得多。

23.2.1 熟知端口号

表23.1列出了UDP使用的一些熟知端口号。有些端口号是UDP和TCP都可使用的。在本章稍后我们讨论TCP时，将讨论它们。

表23.1 UDP使用的熟知端口

端 口	协 议	说 明
7	Echo	将接收到的数据报回送到发送方
9	Discard	丢弃接收到的任何数据报
11	Users	活跃的用户
13	Daytime	返回日期和时间
17	Quote	返回日期的引用
19	Chargen	返回一字符串
53	Nameserver	域名服务
67	BOOTPs	下载引导程序信息的服务器端口
68	BOOTPc	下载引导程序信息的客户机端口
69	TFTP	简单文件传送协议
111	RPC	远程过程调用
123	NTP	网络时间协议
161	SNMP	简单网络管理协议
162	SNMP	简单网络管理协议 (中断)

例23.1

在UNIX中，熟知端口号存储在/etc/services文件中，这个文件的每行给出服务器名和熟知端口号。我们可用grep命令提取该行所对应的应用。下面表示了FTP的端口。注意：FTP可使用UDP或TCP的端口21。

```
$ grep ftp /etc/services
ftp      21/tcp
ftp      21/udp
```

SNMP使用两个端口号（161和162），我们将在第28章中看到每一个用于不同目的。

```
$ grep snmp /etc/services
snmp      161/tcp      #Simple Net Mgmt Proto
snmp      161/udp      #Simple Net Mgmt Proto
snmptrap  162/udp      #Traps for SNMP
```

23.2.2 用户数据报

UDP分组称为用户数据报（user datagram），有8字节的固定头部。图23.9说明了用户数据报的格式。

用户数据报共有下列字段：

- 源端口号。这是在源主机上运行的进程使用的端口号。它有16位长，这就表示端口号可以从0到65 535。如果源主机是客户端（当客户进程发送请求时），那么大多数情况下这个端口号就是暂时端口号，它由该进程请求，由源主机上运行的UDP软件进行选择。如果源主机是服务器（当服务器进程发送响应时），那么在大多数情况下这个端口号是熟知端口号。
- 目的端口号。这是在目的主机上运行的进程使用的端口号。它有16位长。如果目的主机是服务器（当客户进程发送请求时），那么在大多数情况下这个端口号是熟知端口号。如果目的主机是客户端（当服务器进程发送响应时），那么大多数情况下这个端口号是暂时端口号。在这种情况下，服务器将它接收到的请求分组中的暂时端口号复制下来。
- 长度。这是一个16位字段，它定义了用户数据报的总长度，头部加上数据。16位可定义的总长度是从0到65 535字节。但是，总长度必须比这个小。因为UDP数据报存放在具有总长度为65 535字节的IP数据报中。

UDP用户数据报的长度字段实际上是没有必要的。用户数据报封装成IP数据报。在IP数据报中有一个字段定义总长度。IP数据报还有一个字段定义头部长度。所以如果我们将上述第一个字段的值减去第二个字段的值，我们就能求得封装在IP数据报中的UDP数据报的长度。

UDP长度=IP长度-IP头部长度。

但是，UDP协议设计者认为，要让IP软件提供这一信息更有效。我们应该记得，当IP软件把UDP用户数据报提交给UDP层时，它已经除去了IP头部。

- 校验和。这个字段用来校验整个用户数据报（头部加数据）出现的差错。校验和将在下一节讨论。

23.2.3 校验和

我们已在第10章讨论过校验和的概念与计算方法。我们还讨论过如何计算IP和ICMP分组

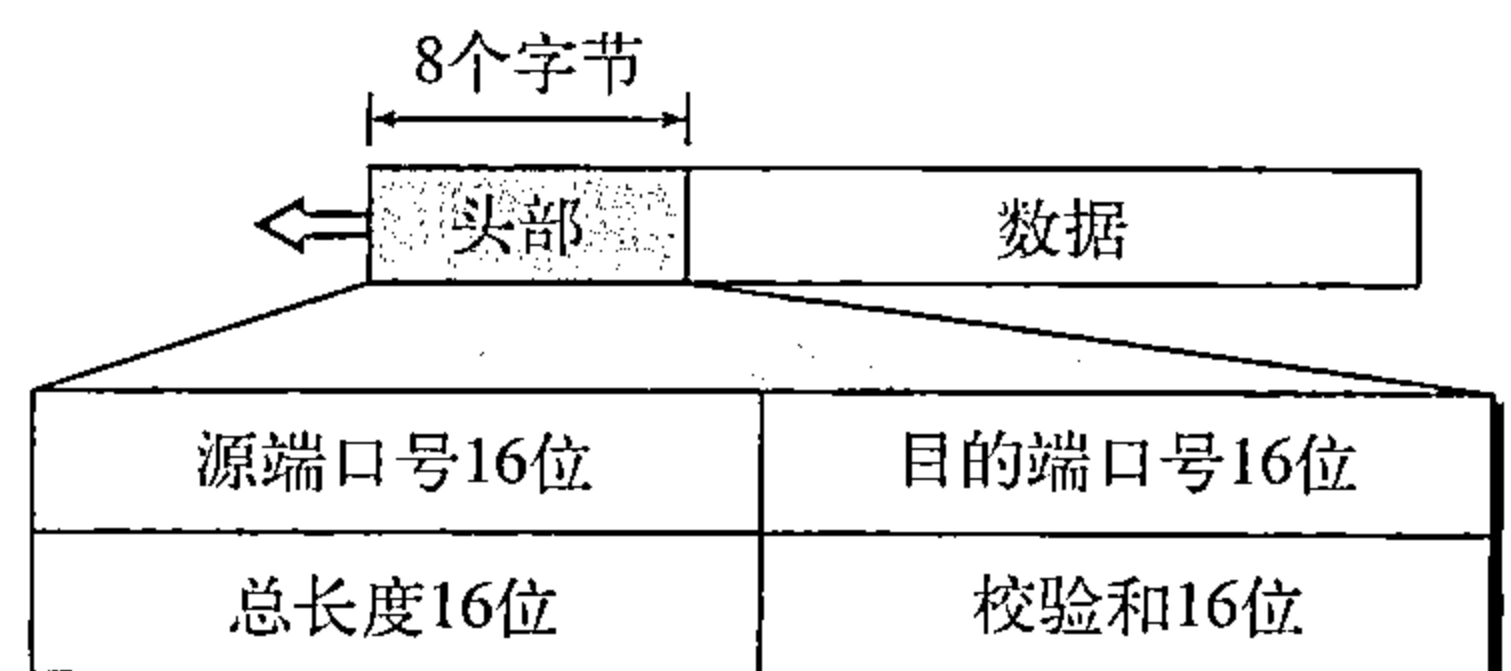


图23.9 用户数据报格式

的校验和。现在我们讨论如何计算UDP的校验和。

UDP校验和的计算与IP和ICMP校验和的计算不同。这里校验包括三个部分：伪头部、UDP头部和从应用层来的数据。

伪头部（pseudoheader）是IP分组的头部的一部分，其中有些字段要填入0，用户数据报封装在IP分组中（见图23.10）。

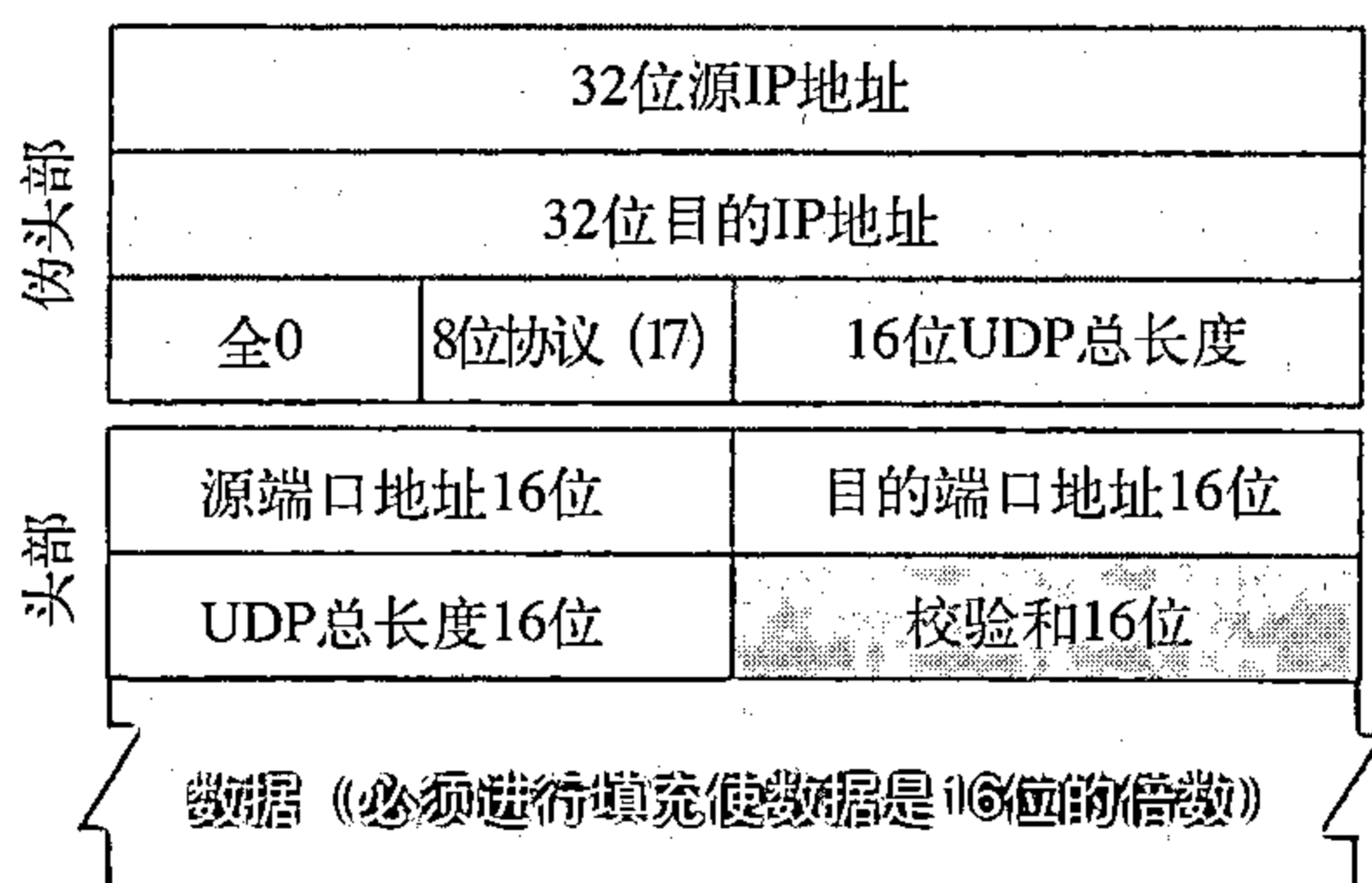


图23.10 用于校验和计算的伪头部

如果校验和不包括伪头部，用户数据报也可能是安全的和正确地到达。但是，如果IP头部受到损坏，那么它可能被提交到错误的主机。

增加协议字段可确保这个分组是属于UDP，而不是属于其他传输层协议。我们在后面将会看到，如果一个进程既可用UDP又可用TCP，则端口号可以是相同的。UDP的协议字段值是17。如果在传输过程中这个值改变了，在接收端计算校验和时就可检测出来，UDP就可丢弃这个分组。这样就不会传递给错误的协议。

注意伪头部字段和IP头部最后12字节的相似性。

例23.2

图23.11给出了只有7个字节数据的很小的用户数据报的校验和计算。因为数据的字节数是奇数，因此为了计算校验和就要加上填充。当用户数据报要传递给IP时，就将伪头部和填充丢弃。

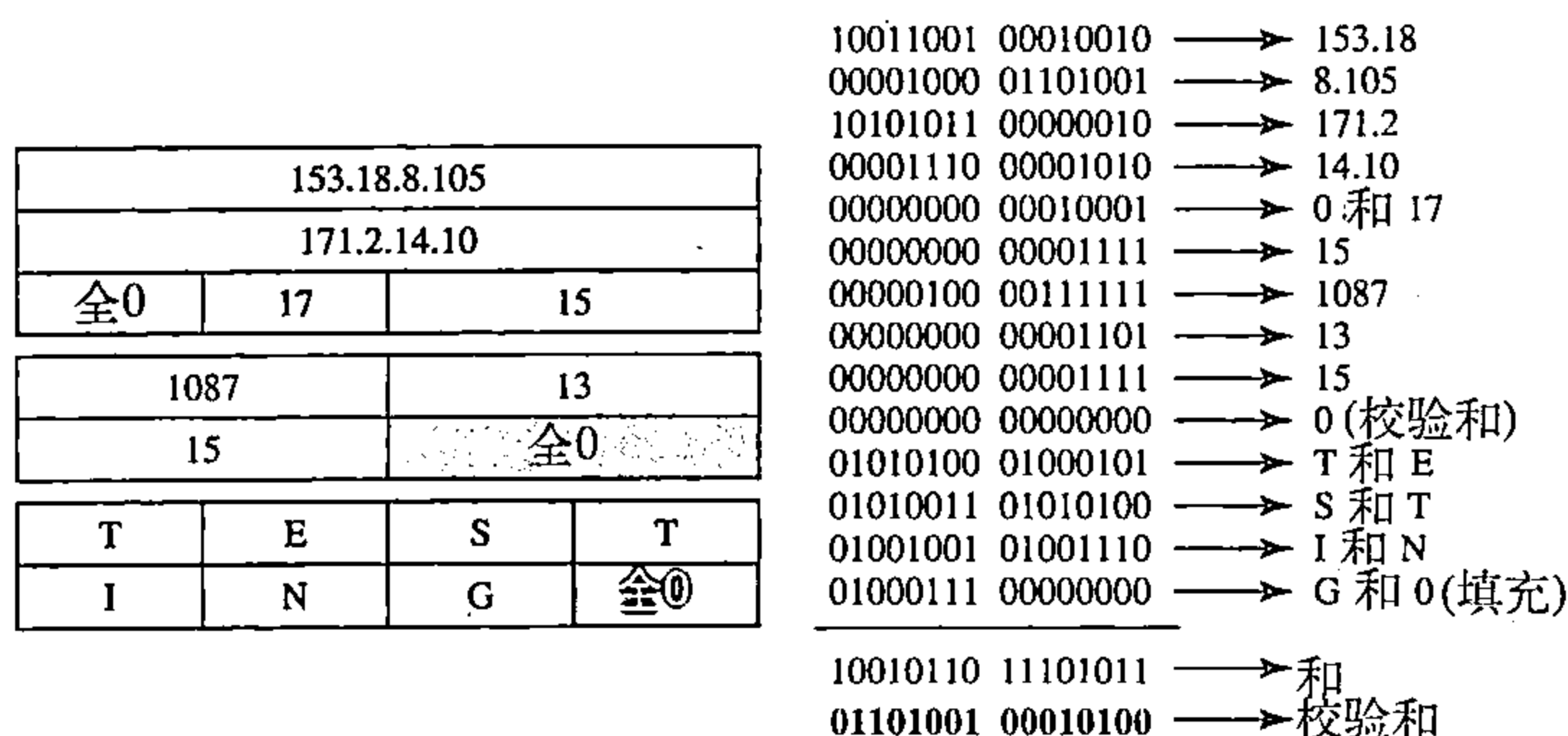


图23.11 简单UDP用户数据报的校验和计算

可选的校验和

校验和的计算以及在用户数据报中包括的校验和都是可选的。如果不进行校验和的计算，则这个字段就填入全1。注意：计算得到的校验和永远不会是全1，因为这就是说和是全0，这是不可能的。由于它要求字段的值为全0，这是不可能的。

23.2.4 UDP的操作

UDP使用了传输层共同的一些概念。将简要地讨论这些概念，然后在下一节介绍TCP协议时展开讨论。

无连接服务

如前所述，UDP提供无连接服务。这就是表示UDP发送出去的每一个用户数据报都是一个独立的数据报。不同的用户数据报之间没有关系，即使它们都来自相同的源进程并发送到相同的目的程序。用户数据报不进行编号。此外，也没有像TCP协议那样的连接建立和连接终止，这就表示每一个用户数据报可以沿着不同的路径传递。

无连接的一个结果就是使用UDP的进程不能向UDP发送数据流，也不能期望UDP将这个数据流分割成许多不同的相关联的用户数据报。相反，每一个请求必须足够小，使其能够装入到用户数据报中，只有那些发送短报文的进程才应当使用UDP。

流量控制和差错控制

UDP是一个简单的不可靠的传输协议，它没有流量控制，因而也没有窗口机制。如果到来的报文太多时，接收方可能会溢出。

除校验和外，UDP也没有差错控制机制，这就表示发送方不知道报文是否丢失还是重复传递。当接收方使用校验和检测出差错时，就悄悄地将此用户数据报丢弃。

缺少流量控制（flow control）和差错控制（error control）就表示使用UDP的进程必须要提供这些机制。

封装和拆封

要将报文从一个进程发送到另一个进程时，UDP协议就要将报文在IP数据报中进行封装和拆封。

排队

我们已经提到过端口，但没讨论端口的实际实现。在UDP中，队列是与端口联系在一起的（见图23.12所示）。

在客户端，当进程启动时，它从操作系统请求一个端口号。有些实现是创建一个入队列和一个出队列与每一个进程相关联。而有些实现只创建与每一个进程相关的入队列。

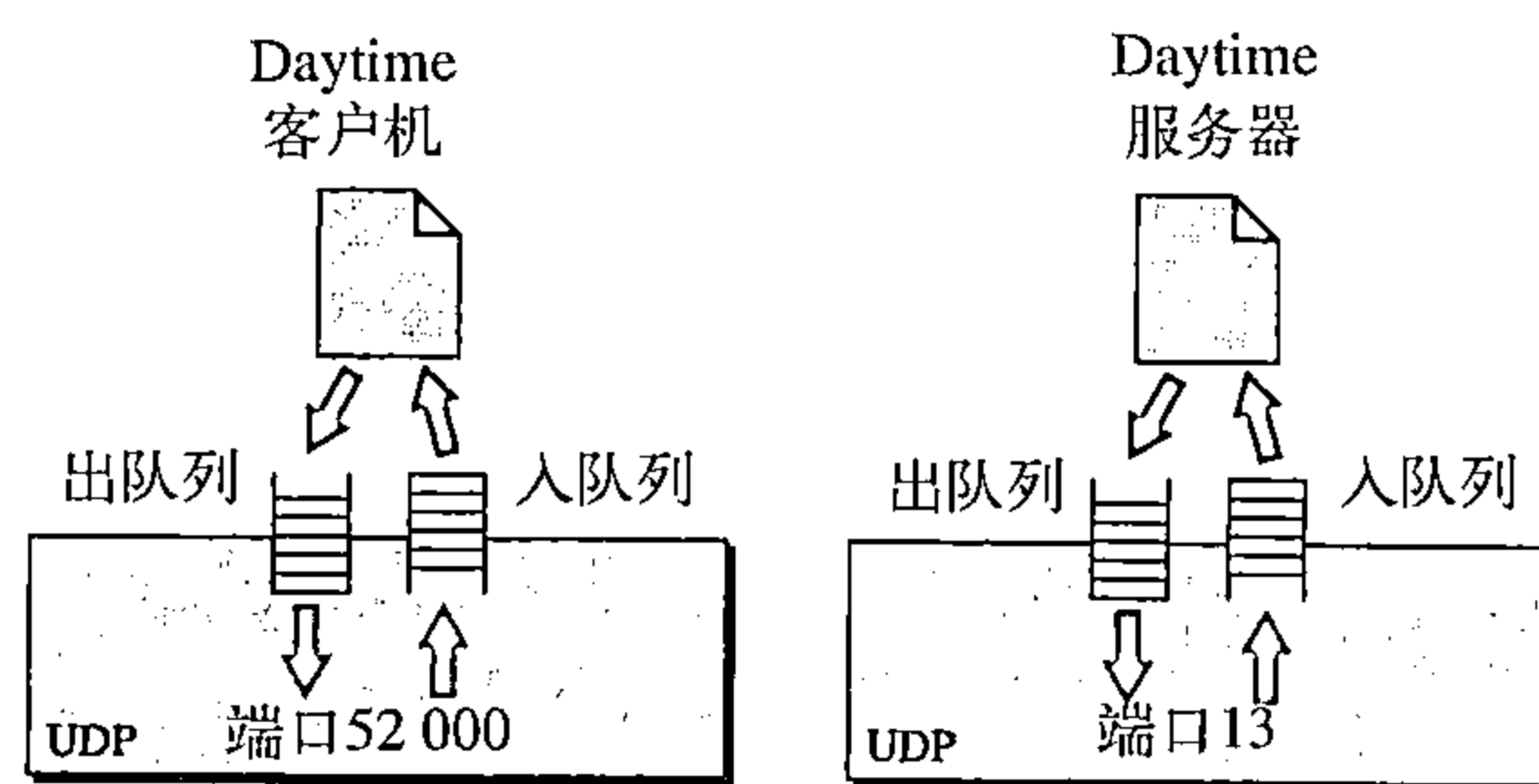


图23.12 UDP中的队列

注意：即使当一个进程想与多个进程通信时，它也只得到一个端口号，而最后也只有一个出队列和一个入队列（queue）。在多数情况下，由客户打开的队列由暂时端口号来标识，只要进程在运行，这些队列就起作用。当进程终止时，队列就撤销。

客户进程使用在请求中指明的源端口号将报文发送到出队列。UDP逐个将报文取出，加上UDP头部递交给IP。出队列可能发生溢出。如果发生溢出，操作系统就要求客户进程在继续发送报文之前要等待。

当报文到达客户端时，UDP要检查一下以确认对应于该用户数据报中目的端口号字段指明的端口号是否创建了入队列。如果有这样的入队列，UDP就将接收到的用户数据报放在该队列的末尾。如果没有这样的队列，UDP就丢弃该用户数据报，并请求ICMP协议向服务器发送端口不可达报文。所有发送给一个特定客户程序的入报文，不管是来自相同的服务器还是

不同的服务器，都被放在同一个队列中。入队列可能会溢出，如果发生溢出，UDP就丢弃此用户数据报，并请求向服务器发送端口不可达报文。

在服务器端，创建队列的机制是不同的。用最简单形式，服务器在它开始运行时就用它的熟知端口请求入队列和出队列。只要服务器运行，队列就一直是打开的。

当报文到达服务器进程时，UDP要检查一下以确认对应于该用户数据报中目的端口号字段指明的端口号是否创建了入队列。如果有这样的队列，UDP就将接收到的用户数据报放在该队列的末尾。如果没有这样的队列，UDP就丢弃该用户数据报，并请求ICMP协议向客户端发送一个端口不可达报文。一个特定服务器程序的所有入报文，不管是来自相同的客户还是不同的客户机，都放入同一队列。入队列可能溢出。如果发生溢出，UDP就丢弃这个用户数据报，并请求向客户发送端口不可达报文。

当服务器想要回答客户时，它就使用在请求报文中指明的源端口号将报文发送到出队列。UDP逐个将报文取出，加上UDP的头部，递交给IP。出队列可能溢出，如果发生溢出，操作系统就要求服务器进程在继续发送报文之前要等待。

23.2.5 UDP的使用

下列是UDP协议的某些应用：

- UDP适用于这样的进程，它需要简单要的请求-响应通信，而较少考虑流量控制和差错控制。对于需要传送成块数据的进程，如FTP（见第26章），则通常不使用它；
- UDP适用于具有有内部流量控制和差错控制机制的进程。例如，简单文件传输协议（TFTP）的进程就包括流量控制和差错控制，它能够很容易地使用UDP；
- 对多播来说，UDP是一个合适的传输协议。多播能力已被嵌入在UDP软件中，但没有嵌入在TCP软件中；
- UDP可用于管理进程，如SNMP（见第28章）；
- UDP可用于某些路由选择更新协议，如路由选择信息协议（RIP）（见第22章）。

23.3 TCP

本章我们讨论的第二个传输层协议，称为传输控制协议（Transmission Control Protocol, TCP）。与UDP一样，TCP是一个进程到进程（程序到程序）的协议，所以它使用端口号；但与UDP不同的是，TCP是面向连接协议。为发送数据它在两个TCP之间建立一个虚拟连接。另外，TCP在传输层使用流量控制和差错控制机制。

简单地说，TCP被称为面向连接的、可靠的传输协议。它为IP服务增加了面向连接和可靠性的特性。

23.3.1 TCP服务

在详细讨论TCP之前，让我们解释TCP向应用层进程提供的服务。

进程到进程的通信

像UDP一样，TCP用端口号提供进程到进程的通信。表23.2列出了TCP使用的一些熟知端口号。

流传递服务

与UDP不同，TCP是一个面向流的协议。在UDP中，进程（应用程序）发送一些具有预先规定界限的报文给UDP进行传递。UDP将它自己的头部添加到这些报文中每一个报文上。来自进程的每一个报文称为一个用户数据报，最后变成一个IP数据报。IP和UDP都不认识这

些数据报之间的关系。

表23.2 TCP使用的熟知端口

端 口	协 议	说 明
7	Echo	将接收到的数据报回送到发送方
9	Discard	丢弃任何接收到的数据报
11	Users	活跃的用户
13	Daytime	返回日期和时间
17	Quote	返回日期的引用
19	Chargen	返回一字符串
20	FTP, 数据	文件传输协议 (数据连接)
21	FTP, 控制	文件传输协议 (控制连接)
23	TELNET	终端网络
25	SMTP	简单邮件传输协议
53	DNS	域名服务器
67	BOOTP	引导程序协议
79	Finger	Finger
80	HTTP	超文本传输协议
111	RPC	远程过程调用

另一方面，TCP允许发送进程以字节流形式传递数据，并且接收进程也以字节流形式接收数据。TCP建立一种环境，在这种环境中，两个进程好像由一个假想的“管道”连接，这个管道通过因特网传送这些进程的数据。这种假想的环境，如图23.13所示。发送进程产生（写入）字节流，而接收进程消费（读出）这些字节流。

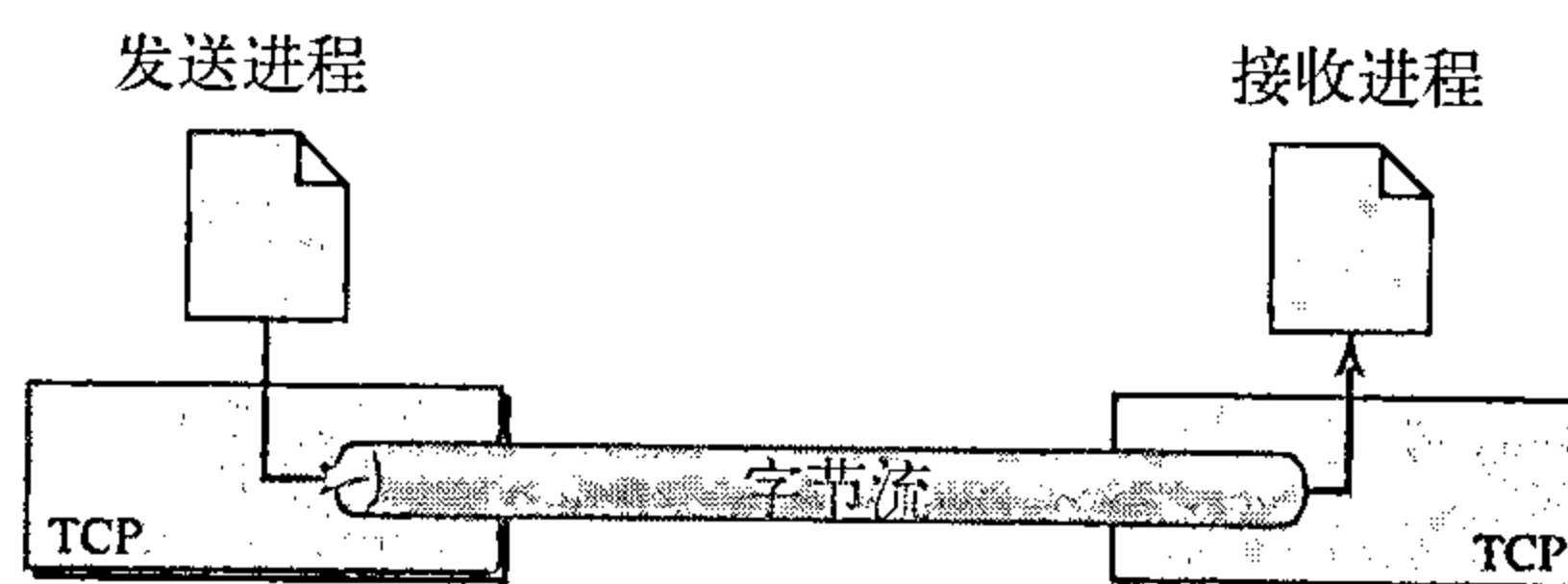


图23.13 字节流传递

发送和接收缓冲区 因为发送和接收进程可能以不同的速度写入和读出数据，所以TCP需要用于存储的缓冲区。每一个方向都存在一个缓冲区：发送缓冲区和接收缓冲区（稍后我们会看到这些缓冲区也用于TCP流量和差错控制机制）。实现缓冲的一种方法是使用一字节存储单元的循环数组，如图23.14所示。为了简化，我们只画出了两个缓冲区，每个缓冲区20个

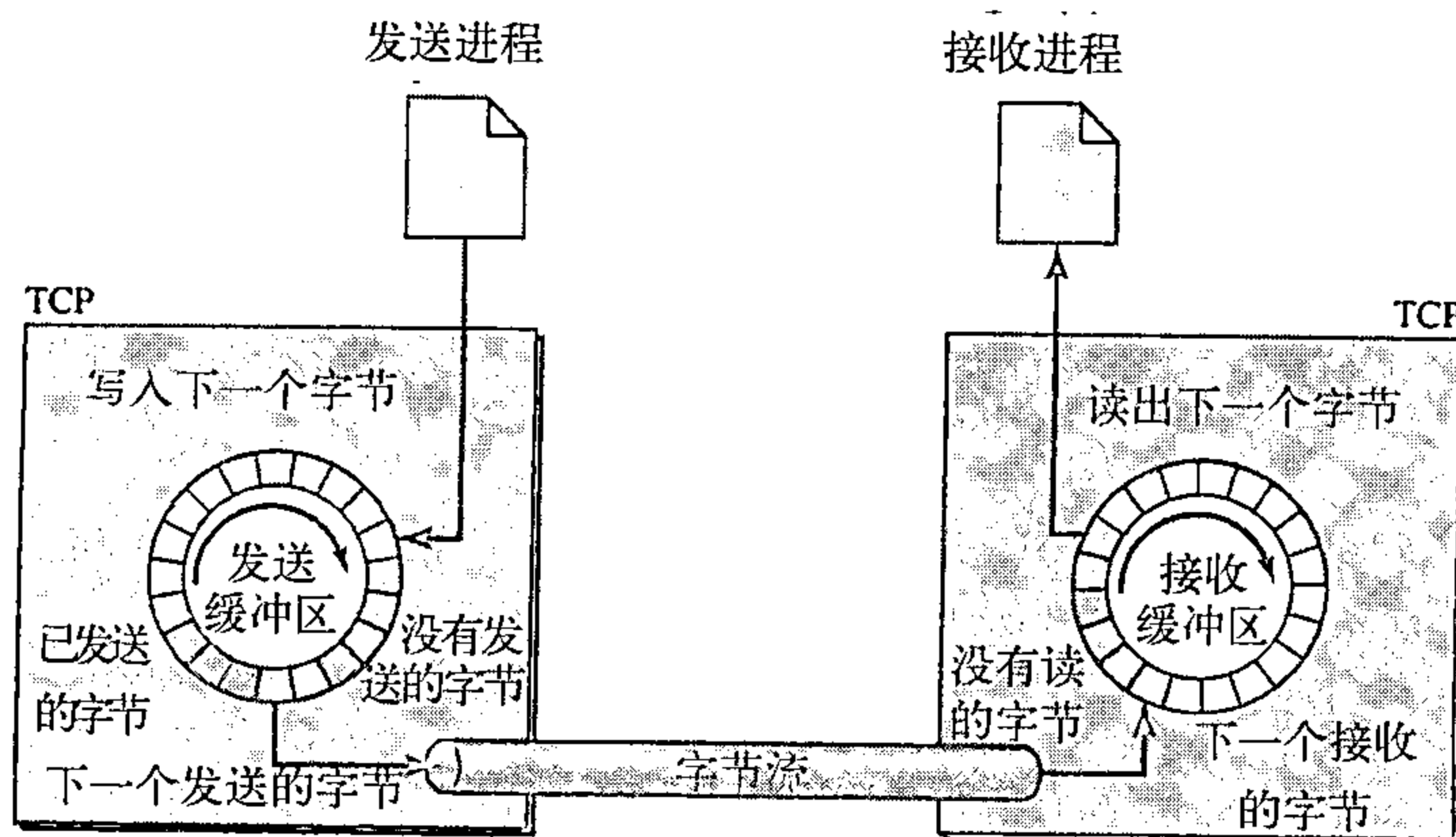


图23.14 发送与接收缓冲区

字节。通常情况下，缓冲区是成百或上千个字节，这取决于实现方法。这里表示的缓冲区大小是相同的，实际上并非总是如此。

图23.14中表示了在一个方向上数据的移动。在发送端，缓冲区有三种类型的存储单元。白色的部分是空存储单元，可以由发送进程（生产者）填充。灰色的部分用于保存已经发送但还没有得到确认的字节。TCP在缓冲区中保留这些字节，直到收到确认为止。灰色缓冲区是即将由TCP发送的字节。但是，在本章的后面将会看到，TCP可能只发送灰色部分。这可能是由于接收进程缓慢或者网络中可能发生的拥塞而造成的。还要注意，灰色存储单元的字节被确认后，这些存储单元可以回收并且对发送进程可用，这就是我们表示一个环形缓冲区的原因。

接收端的缓冲区操作比较简单。环形缓冲区分成两个区域（表示为白色和灰色）。白色区域包含空存储单元，可以由从网络上接收的字节进行填充。灰色区域表示接收到的字节，可以由接收进程读出。当某个字节被接收进程读出以后，这个存储单元可被回收，并加入到空存储单元池中。

段 尽管缓冲能够处理生产进程速度和消费进程速度之间的不相称问题，但在发送数据之前，还需要多个步骤。IP层作为TCP服务的提供者，需要以分组的方式而不是字节流的方式发送数据。在传输层，TCP将多个字节分组合在一起成为一个分组，这个分组称为段（segment）。TCP给每个段添加头部（为了控制的目的），并将该段传递给IP层。段被封装到IP数据报中，然后再进行传输。整个操作对接收进程是透明的。稍后，我们会看到这些段可能被无序接收、丢失、或者损坏和重发。所有这些均由TCP处理，接收进程不会觉察到任何操作。图23.15表示了缓冲区中如何从字节生成段。

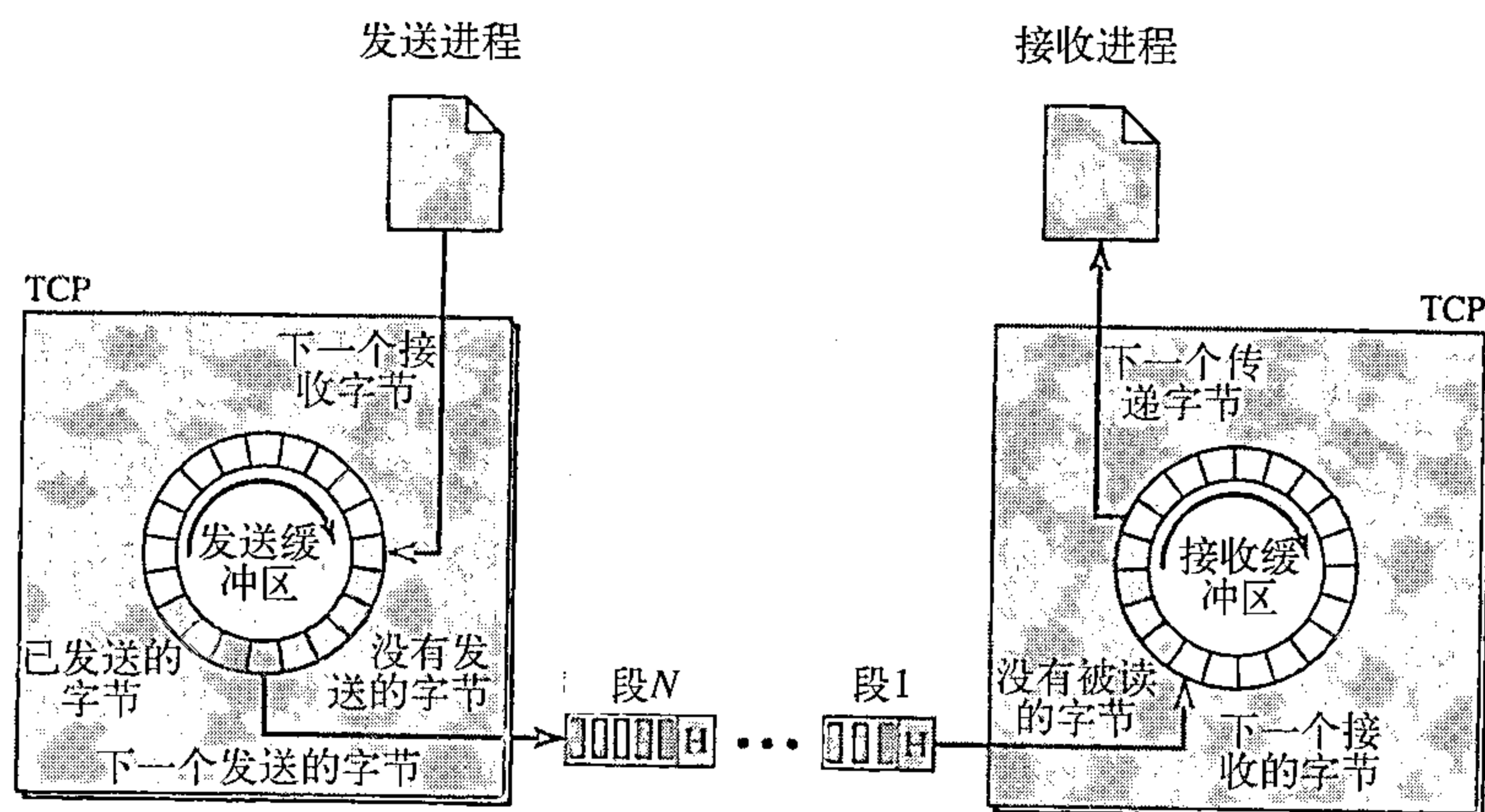


图23.15 TCP中的段

注意：段的大小不必相同。为了简单起见，我们在图23.15中只表示了一个包含3个字节的段和另一个包含5个字节的段。实际的段可能包含数百（或者数千）个字节。

全双工通信

TCP提供全双工服务（full-duplex service），即数据可以在同一时间双向流动。每一方TCP都有发送和接收缓冲区，它们能在双向发送和接收段。

面向连接的服务

与UDP不同，TCP是一种面向连接的协议。站点A的一个进程要发送和接收来自站点B的另外一个进程的数据时，步骤如下：

1. 在两个TCP之间建立一个连接;
2. 在两个方向交换数据;
3. 连接终止。

注意：这是一个虚连接，而不是一个物理连接。TCP段封装成IP数据报，并且可能被无序地发送、或丢失或被破坏，然后重发。每个段都可以通过不同的路径到达目的端。虽然不存在物理连接，但TCP会建立一种面向字节流的环境，在这种环境中，TCP能承担按顺序传递这些字节到其他站点的任务。这就好像建立了横跨多个岛屿的一座桥，以单一的连接从一个岛屿到另一个岛屿传送所有字节。

可靠的服务

TCP是一种可靠的传输协议。它使用确认机制来检查数据是否安全和完整地到达。我们将在本章的后面进一步讨论差错控制的特点。

23.3.2 TCP特点

前节已提到TCP提供服务本节将简要概述TCP的其他一些特点，稍后将进行详细的讨论。

序号系统

虽然TCP软件能够记录发送或接收的段，但是在段的头部没有段序号字段。TCP在段的头部采用称为序号（sequence number）和确认号（acknowledgment number）的两个字段。这两个字段指的是字节序号，而不是段序号。

字节序号 TCP为在一个连接中传输的所有数据字节编号。在每个方向上序号都是独立的。当TCP接收来自进程的一些数据字节时，将它们存储在发送缓冲区中时并给它们编号。不必从0开始编码，TCP在0到 $2^{32}-1$ 之间生成一个随机数作为第一个字节的序号，例如，如果随机数是1 057，并且发送的全部字节个数是6 000，那么这些字节序号是1 057~7 056。下面将会看到字节序号是用于流量和差错控制。

在每个连接中传送的字节都由TCP编号，序号开始于一个随机产生的数。

序号 字节被编号后，TCP对发送的每一个段分配一个序号。每个段的序号是这个段中的第一个字节的序号。

例23.3

假设一个TCP连接正在传送一个5 000字节的文件，第一个字节序号是10 001。如果数据被分成5个段，每一个数据段携带1 000字节，试问每个段的序列号是什么？

解：

每个段的序号如下所示：

- 段 1→序号：10 001 （范围：10 001~11 000）
- 段 2→序号：11 001 （范围：11 001~12 000）
- 段 3→序号：12 001 （范围：12 001~13 000）
- 段 4→序号：13 001 （范围：13 001~14 000）
- 段 5→序号：14 001 （范围：14 001~16 000）

一个段的序号字段的值定义为该段包含的第一个字节的序号。

当一个段携带数据和控制信息（捎带）时，它使用一个序号。如果一个段没有携带用户数据，那么它逻辑上不定义序号。虽然字段存在，但值是无效的。然而，有些段当仅携带控制信息时也需要有一个序号用于接收方的确认。这些段用做连接建立、连接终止或连接废弃。这些段中的每一个好像携带一个字节那样使用一个序号，但都没有实际的数据。如果随机产

生的序号是 x ，则第一个数据字节的编号是 $x+1$ 。字节 x 被认为是一个控制段打开一个连接的一个伪字节，我们不久将会见到。

确认号 正如前面所讨论过的那样，TCP中的通信是全双工的；当建立一个连接时，双方同时都能发送和接收数据。每一方为字节编号，经常使用不同的起始字节号。每一方向的序号表明了该段所携带的第一个字节的序号。每方也使用确认号来确认它已收到的字节。但是，确认号定义了该方预期接收的下一个字节的序号。另外，确认号是累积的，这意味接收方记下它已安全而且正确地接收到最后一个字节的序号，然后将它加1，并通告这个结果作为确认号。在这里，术语“累积”指的是，如果一方使用5 643作为确认号，则表示它已经接收了所有从开始到序号为5 642的字节。但要注意，这并不是指接收方已经接收了5 642个字节，因为第一个字节的编号通常并不是从0开始的。

段中确认字段的值定义了通信一方预期接收的下一个字节的编号。确认号是累加的。

流量控制

与UDP不同的是，TCP提供流量控制。数据的接收方控制发送方发送数据的数量，这样做是为了防止接收方数据溢出。序号系统允许使用面向字节的流量控制。

差错控制

为了提供可靠的服务，TCP有一个差错控制机制。虽然差错控制以段作为差错检测（丢失或损坏段）的数据单元。稍后我们将会看到面向字节的差错控制。

拥塞控制

与UDP不同的是，TCP考虑网络中的拥塞。发送方发送的数据量不仅由接收方控制（流量控制），而且还要由网络中的拥塞程度决定。

23.3.3 段

在我们更详细地讨论TCP之前，让我们讨论TCP分组本身。在TCP中的分组称为段(segment)。

格式

段的格式，如图23.16所示。

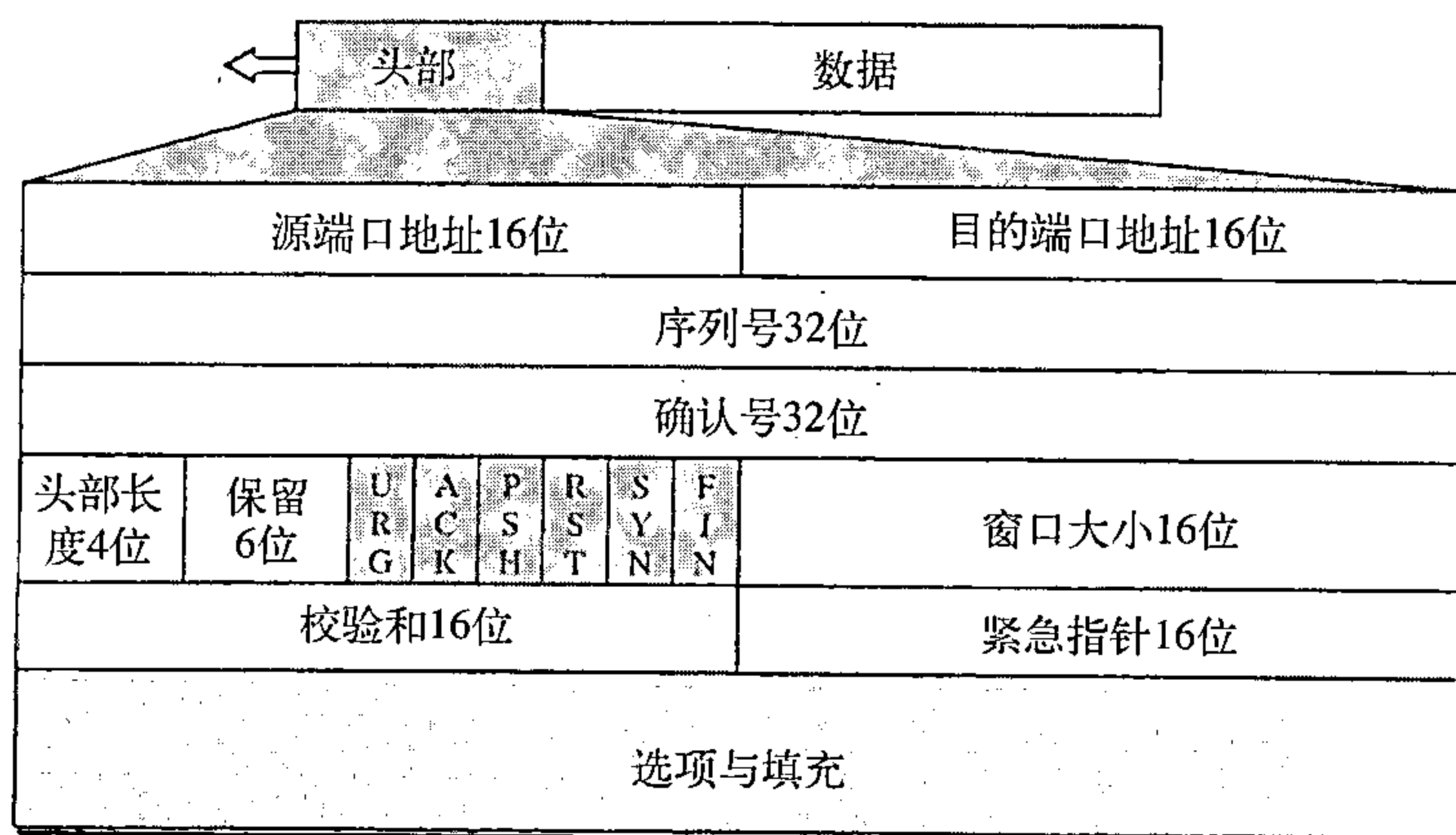


图23.16 TCP段格式

段包括20字节到60字节的头部，接着是来自应用程序的数据。如果没有选项，那么头部

是20字节；如果有选项，最多是60字节。在本节中，我们将讨论某些头部字段，通过本章的学习可以对这些字段的意思和目的更加清楚。

- 源端口地址。这是一个16位的字段，它定义了主机中发送该段的应用程序的端口号。这与UDP头部的源端口地址的作用一样。
- 目的端口地址。这是一个16位的字段，它定义了主机中接收该段的应用程序的端口号。这与UDP头部的目的端口地址的作用一样。
- 序列号。这个32位的字段定义了一个数，它分配给段中数据的第一个字节。如前所述，TCP是一种字节流传输协议。为了确保连通性，对要发送的每一个字节都进行编号。序列号告诉目的端，在这个序列中哪一个字节是该段的第一个字节。在连接建立时，每一方都使用随机数生成器产生一个初始序列号（initial sequence number, ISN），通常每一个方向的ISN都不同。
- 确认号。这个32位的字段定义了段的接收方期望从对方接收的字节号。如果段的接收方成功地接收了对方发来的字节号 x ，它就将确认号定义为 $x + 1$ 。确认和数据可捎带一起发送。
- 头部长度的。这个4位的字段指明了TCP头部中共有多少个4字节长的字。头部的长度可以在20字节到60字节之间。因此，这个字段的值在5（ $5 \times 4 = 20$ ）到15（ $15 \times 4 = 60$ ）之间。
- 保留。这个6位的字段保留为将来使用。
- 控制。这个字段定义了6种不同的控制位或标记，如图23.17所示。在同一时间可设置一位或多位。

URG: 紧急指针有效 ACK: 确认有效 PSH: 请求急迫			RST: 连接复位 SYN: 同步序列号 FIN: 终止连接		
URG	ACK	PSH	RST	SYN	FIN

图23.17 控制字段

这些位用在TCP的流量控制、连接建立和终止、连接失败和数据传送方式等方面。表23.3显示了每位的说明。本章中，当我们讨论TCP的详细操作时，我们将对它们做深入的讨论。

表23.3 控制字段的标志描述

标 记	说 明
URG	紧急指示符字段的值有效
ACK	确认字段的值有效
PSH	将数据推向前
RST	连接复位
SYN	在连接时对序列号进行同步
FIN	终止连接

- 窗口大小。这个字段定义对方必须维持的窗口的大小（以字节为单位）。注意：这个字段的长度是16位，这意味着窗口的最大长度是65 535字节。这个值通常称为接收窗口（rwnd），它由接收方确定。此时，发送方必须服从接收端的支配。

- 校验和。这16位的字段包含了校验和。

TCP校验和的计算过程与前面描述的UDP所采用的计算过程相同。但是，在UDP数据报中校验和是可选的。然而，对TCP来说，将校验和包括进去是强制的。起同样作用的伪头部被加到段上。对TCP伪头部，协议字段的值是6。

- 紧急指示符。这个16位的字段只有当紧急标志置位时才有效，这个段包含了紧急数据。它定义了一个数，将此数加到序列号上就得出此段数据部分中最后一个紧急字节，在本章稍后将会讨论它。

- 选项。在TCP头部中可以有多达40字节的可选信息。我们这里不讨论这些选项，更多信息请参见参考文献。

23.3.4 TCP连接

TCP是一种面向连接的协议。面向连接的传输协议在源端和目的端之间建立一条虚路径。然后，属于一个报文的所有段都沿着这条虚路径发送。整个报文使用单一的虚路径有利于确认处理以及对损坏或丢失帧的重发。读者可能想知道TCP如何使用IP服务，一个无连接协议如何能面向连接。这就是TCP的连接是虚连接，而不是物理连接。TCP在一个较高层次上操作，TCP使用IP服务向接收方传递独立的段，但它控制连接本身。如果一个段丢失或损坏了，则重新发送它。与TCP不同，IP不知道这个重新发送过程。如果一个段失序到达，则TCP保存它直到缺少的段到达。IP是不知道这个重新排序过程的。

在TCP中，面向连接的传输需要三个过程：连接建立、数据传输和连接终止。

连接建立

TCP以全双工方式传输数据。当两个机器中的两个TCP建立连接后，它们就能够同时向对方发送段。这就表示，在传输数据之前，每一个方都必须对通信进行初始化，并得到对方的认可。

三次握手 在TCP中连接建立称为三次握手（three-way handshaking）。在我们的例子中，称为客户的应用程序想要与另一个称为服务器的应用程序使用TCP作为传输层协议建立连接。

该过程从服务器开始。服务器程序告诉它的TCP，它已准备好接受一个连接。这就称为请求被动打开。虽然TCP已准备好接受从世界上任何一个机器来的连接，但它自己并不能完成这个连接。

客户程序发出请求主动打开。想要与服务器进行连接的客户告诉它的TCP，它需要连接到特定的服务器。TCP现在就开始如图23.18所示的三次握手过程。为了表示该过程，我们使用了两个时间序列，每端一个。每个段头部的所有字段都有值，或许它的某些选项字也有。但是，我们每次仅表示少数几个必须要知道的字段，如果序列号、确认号、控制标记（仅是其中被置位的）和窗口大小等有值，我们就表示它们。这个阶段的三个步骤如下：

1. 客户发送的第一个段是SYN段。这个段仅有SYN标志被置位，它用于序列号同步。它占用一个序列号。当数据传输开始时，序号加1。我们说SYN段不携带实际数据，但我们可以认为它是一个字节。

SYN段不携带数据，但它占用一个序列号。

2. 服务器发送第二个段，两个标志位SYN和ACK置位的段，即SYN和ACK段。这个段有两个目的。它是另一方向通信的SYN段，并用ACK标志作为对第一个SYN段的确认。它占用一个序列号。

SYN+ACK段不携带数据，但它占用一个序列号。

3. 客户发送第三个段。这个段仅仅是一个ACK段，它使用ACK标志和确认序号字段来确认收到了第二个段。注意：这个段的序列号与SYN段中的序列号相同，ACK段没有占用任何序列号。

ACK段，如果不携带数据，则它不占用序列号。

同时打开 两个进程都发出主动打开的极少发生的情况，我们称这种情况为同时打开

(simultaneous open)。在这种情况下，两个TCP彼此发送SYN+ACK段，在它们之间建立一条单独的连接。

SYN洪泛攻击 在TCP中，连接建立过程易遭受到称为SYN洪泛攻击(SYN flooding attack)的严重安全问题。在当一个恶意的攻击者将大量的SYN段发送到一个服务器，在数据报中通过伪装源IP地址假装这些段来自不同的客户端时就发生了这种情况。假定客户机发出主动打开，服务器分配必要的资源，如生成通信表和设置计时器等。然后服务器送SYN+ACK段给这些假装客户，但这些段都丢失了。然而，在这段期间，许多资源被占用但没有被使用。如果在短时间内，SYN段的数量很大，服务器最终耗尽它的资源而崩溃。这种SYN洪泛攻击属于一种称为拒绝服务攻击(denial-of-service attack)的安全攻击类型，此时一个攻击者独占系统如此多的服务请求使得系统崩溃，拒绝对每个请求提供服务。

TCP的某些实现有减轻SYN攻击影响的策略。有些实现在特定时间周期内对连接请求进行限制，另一些实现过滤掉来自不需要的源地址的数据报。一个新的策略使用cookie推迟资源分配直到一个完整的连接建立。新的传输层协议STCP使用这种策略，我们将在下一节中讨论。

数据传输

连接建立后，可进行双向数据传输(data transfer)，客户端与服务器双方都可发送数据和确认。在本章稍后我们将学习确认的规则。目前，知道在同一段内携带确认时，在同一方向上也可以传输数据就行了。这就是数据捎带确认。一个例子如图23.19所示，在连接建立后(图中没有表示出)，客户端用两个段发送2 000个字节的数据。然后，服务器用一个段发送2 000个字节的数据。客户端发送另一个段。前面三个段携带数据与确认，但是最后一个段仅携带确认，这是因为已没有数据发送了。注意序列号与确认号的数值，客户端发送的数据段有PSH(急迫)标志，所以服务器TCP知道当收到数据后立刻传递给服务器进程。稍后我们讨论这个标志的用法。另一方面，来自服务器的段没有设置急迫标志。大多数TCP的实现都有可选标志，可设置或不设置。

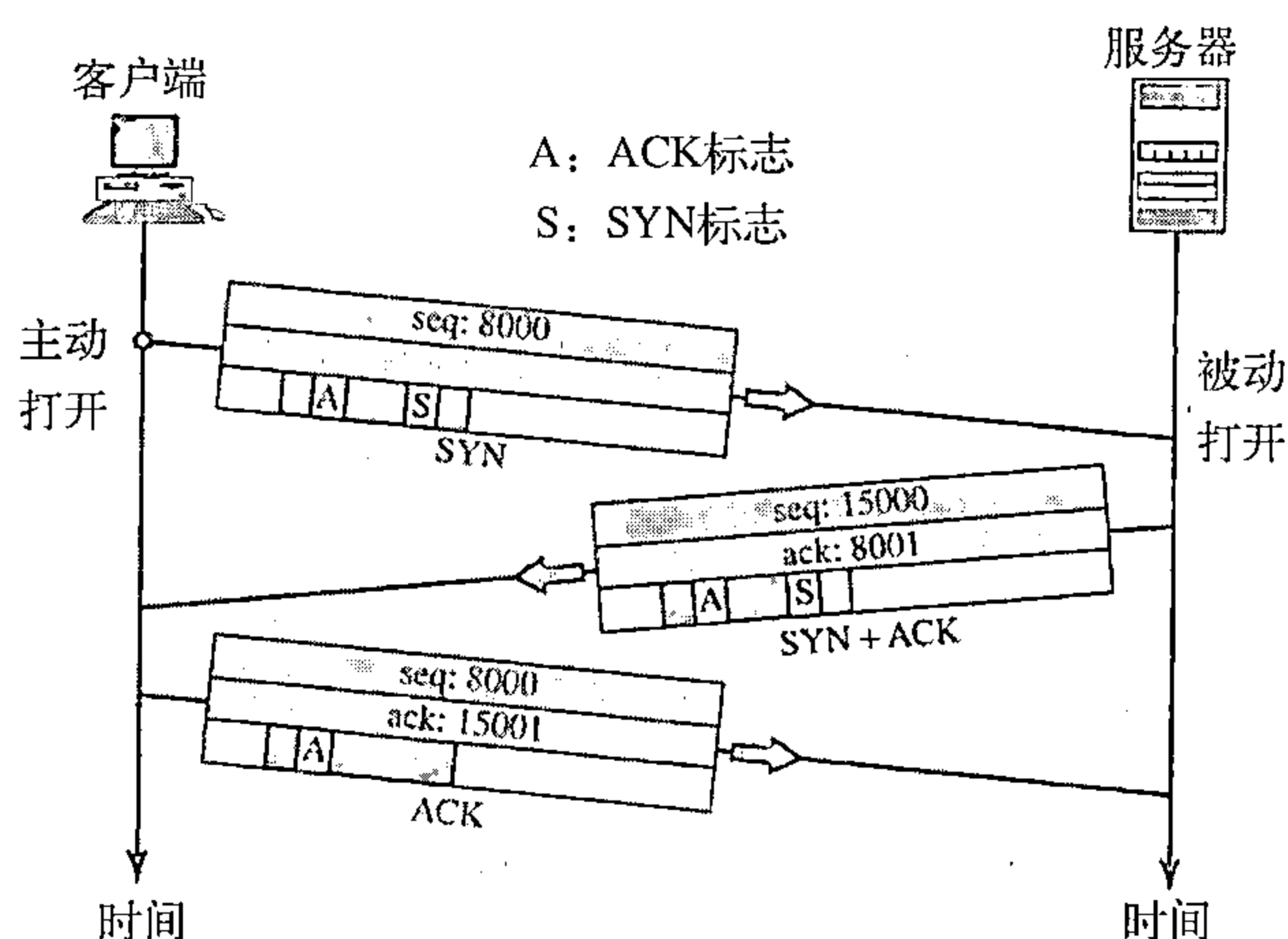


图23.18 使用三次握手建立连接

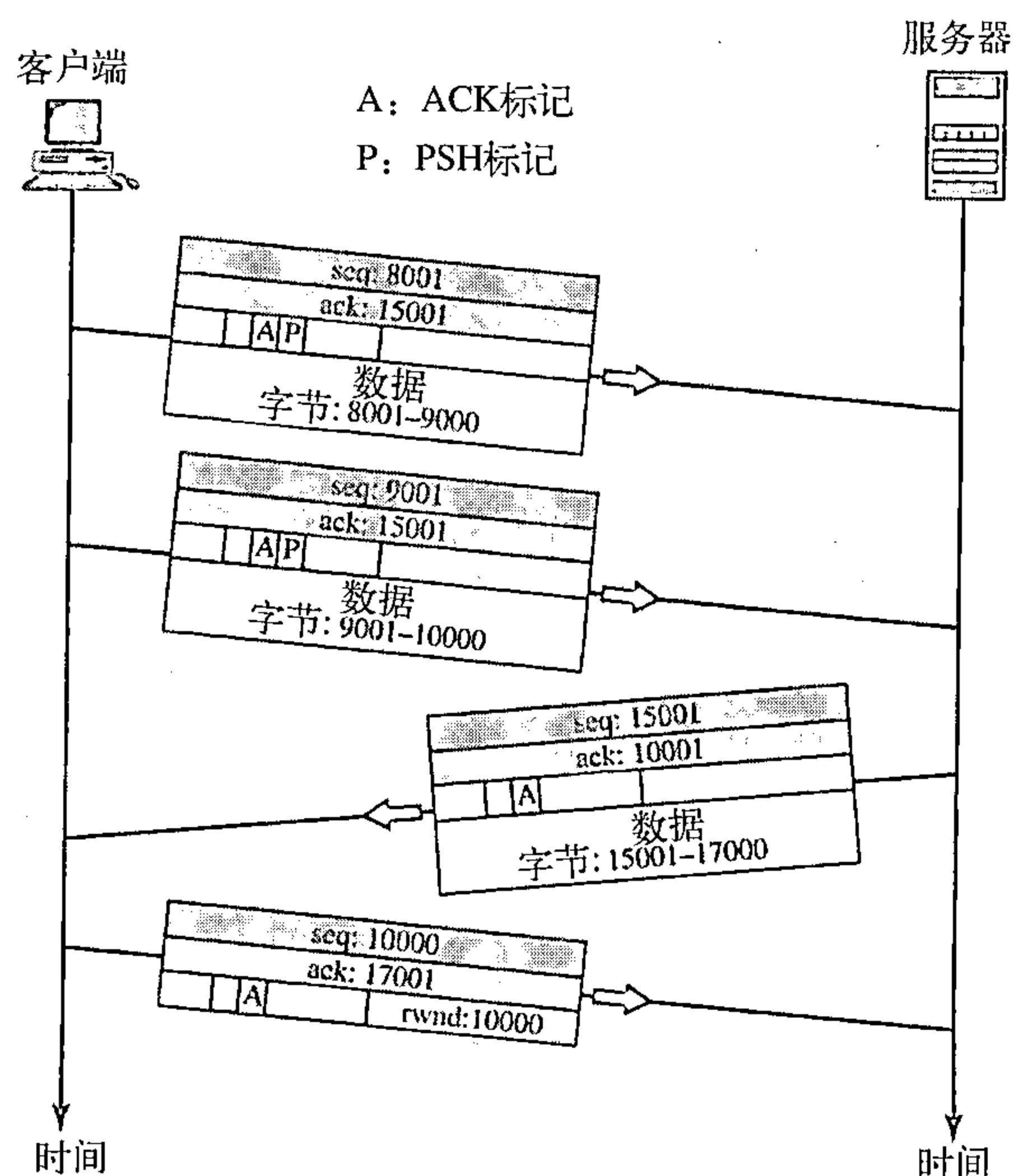


图23.19 数据传输

急迫数据 我们知道发送方的TCP使用缓冲区存储来自发送方应用程序的数据流。发送方的TCP可以选择段的大小。接收方的TCP在数据到达时也将数据进行缓存，并当应用程序准备就绪时或当接收端TCP认为方便时将这些数据传递给应用程序。这种灵活性增加了TCP的效率。

但是，在有些情况下，应用程序并不需要这种灵活性。例如，应用程序与另一方应用程序进行交互式通信。一方的应用程序打算将其击键发送给对方应用程序，并希望接收到立即响应。数据的延迟传输和延迟传递对这个应用程序来说是不可接受的。

TCP可以处理这种情况。在发送端的应用程序可请求急迫操作。这就表示发送端的TCP不必等待窗口被填满。它创建一个段就立即将其发送。发送端的TCP还必须设置急迫位（PSH）以告诉接收端的TCP，这个段所包括的数据必须尽快地传递给接收应用程序，而不要等待更多数据的到来。

虽然急迫操作可由应用程序提出，但是现在的大多数实现却忽视了这些请求。TCP可以选择使用或不使用这个操作。

紧迫数据 TCP是面向字节流的协议。这就是说，从应用程序到TCP的数据被表示成一串字节流。数据的每一个字节在流中占有一位置。但是，在某些情况下，应用程序需要发送紧急字节。这表示发送应用程序希望某一块数据由接收应用程序不按序读出。作为一个例子，假定发送应用程序正在发送被接收应用程序处理的数据。当处理结果返回时，发送应用程序出现了某些差错。它希望中止此过程，但它已发送了大量的数据。如果它发送出异常中止命令（control + C），这两个字符将被存放在接收端TCP缓冲区的末尾。当所有的数据被处理完毕时，这两个字符才被传递给接收应用程序。

解决这个问题的方法是发送将URG位置1的段。发送应用程序告诉发送端的TCP这块数据是紧急的。发送端的TCP创建段，并将紧急数据放在段的开始。段的其余部分可以包括来自缓冲区的普通数据。头部中的紧急指针字段定义了紧急数据的结束和普通数据的开始。

当接收端的TCP接收到URG位置1的段时，它就利用紧急指针的值从段中提取出紧急数据，并不按序地将其传递给接收应用程序。

连接终止

交换数据双方的任一方（客户或服务器）都可关闭连接，通常它是由客户端发起。当前大多数对连接终止的实现有两个方法：三次握手和带有半关闭选项的四次握手。

三次握手 当前对连接终止的多数实现是三次握手，如图23.20所示。

1. 在正常情况下，从客户进程接收到一个关闭命令后，客户的TCP发送第一个段：FIN段，即其中的FIN位置位。注意：FIN段可包含客户机要发送的最后数据块，或如图23.20所示的只是控制段。如果它只是控制段，它仅占用一个序列号。

如果FIN段不携带数据，则该段占用一个序列号。

2. 服务器TCP接收到FIN段后，通知它的进程，并发送第二个段：FIN + ACK段，证实它接收到来自客户端的FIN段，同时通告另一端连接关闭。这个段还可以包含来自服务器最后数据块。如果它不携带数据，则这个段仅占用一个序列号。

如果FIN + ACK段没有携带数据，则该段仅占用一个序列号。

3. 客户端的TCP发送最后一段：ACK段，证实它接收到来自服务器的FIN段。这个段包含确认号，它是接收到来自服务器FIN段的序号加1。这个段不携带数据也不占用序列号。

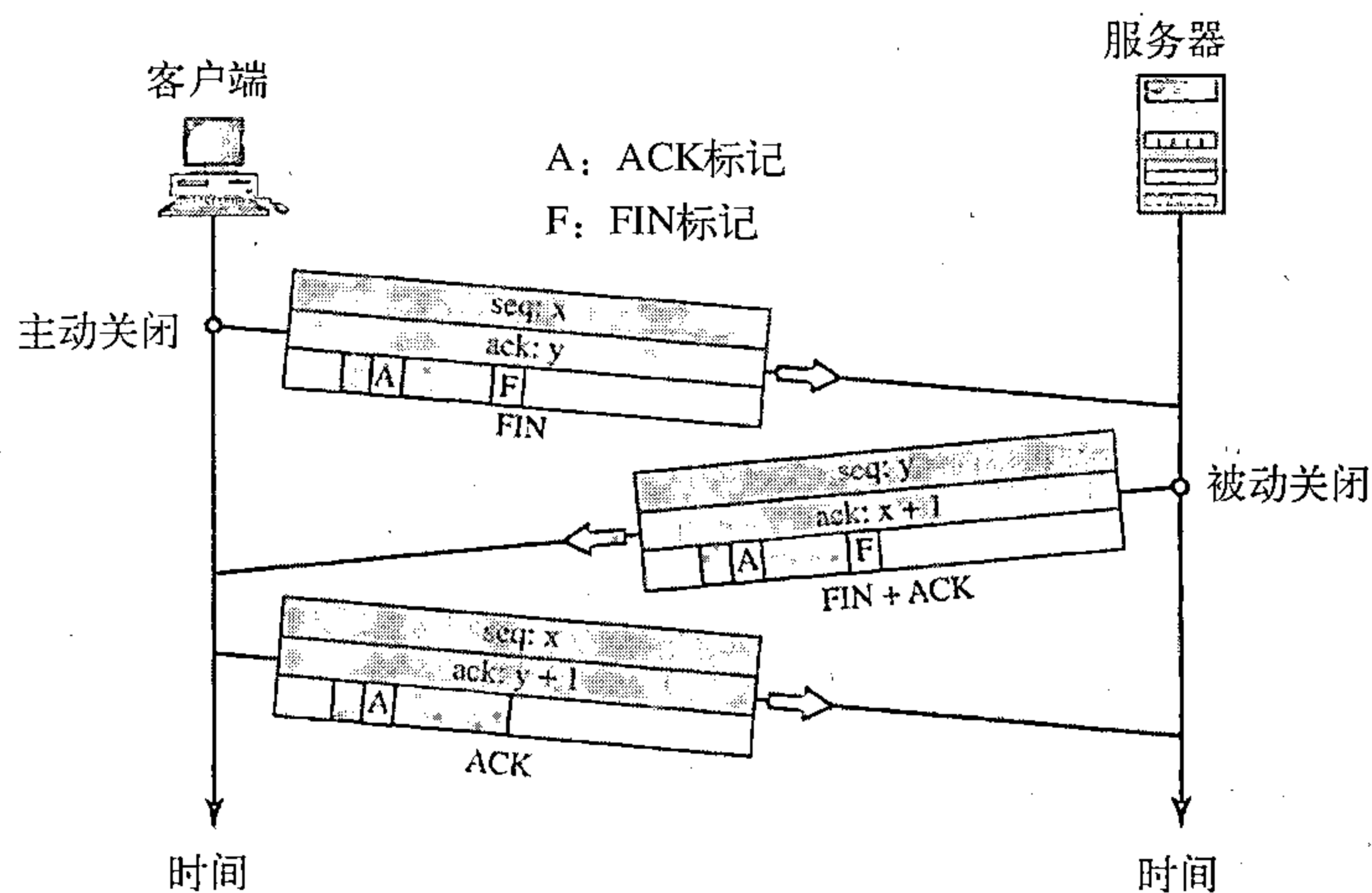


图23.20 三次握手的连接终止

半关闭 在TCP中，一端可以停止发送数据后，还可以继续接收数据。这就是所谓的**半关闭** (half-close)。虽然任一端都可发出半关闭，但通常都是由客户端发起的。当服务器在开始处理之前需要所有数据时，就会出现半关闭。例如，排序是一个很好的例子。当客户端发送数据给服务器进行排序时，在开始排序之前，服务器需要接收到全部数据。这就是说客户端发送完全部数据之后，它在向外方向可连接可关闭。但在向内方向，为了接收排序数据必须保持打开。服务器在接收到数据后还需要时间进行排序，它的向外方向必须保持打开。

图23.21表示了半关闭的例子。客户端发送FIN段半关闭连接，服务器通过发送ACK段确认半关闭。从客户端到服务器的数据传输结束，但是服务器还可以发送数据。当服务器已经发送完被处理的数据时，它发送一个FIN段。该FIN段由客户端的ACK来确认。

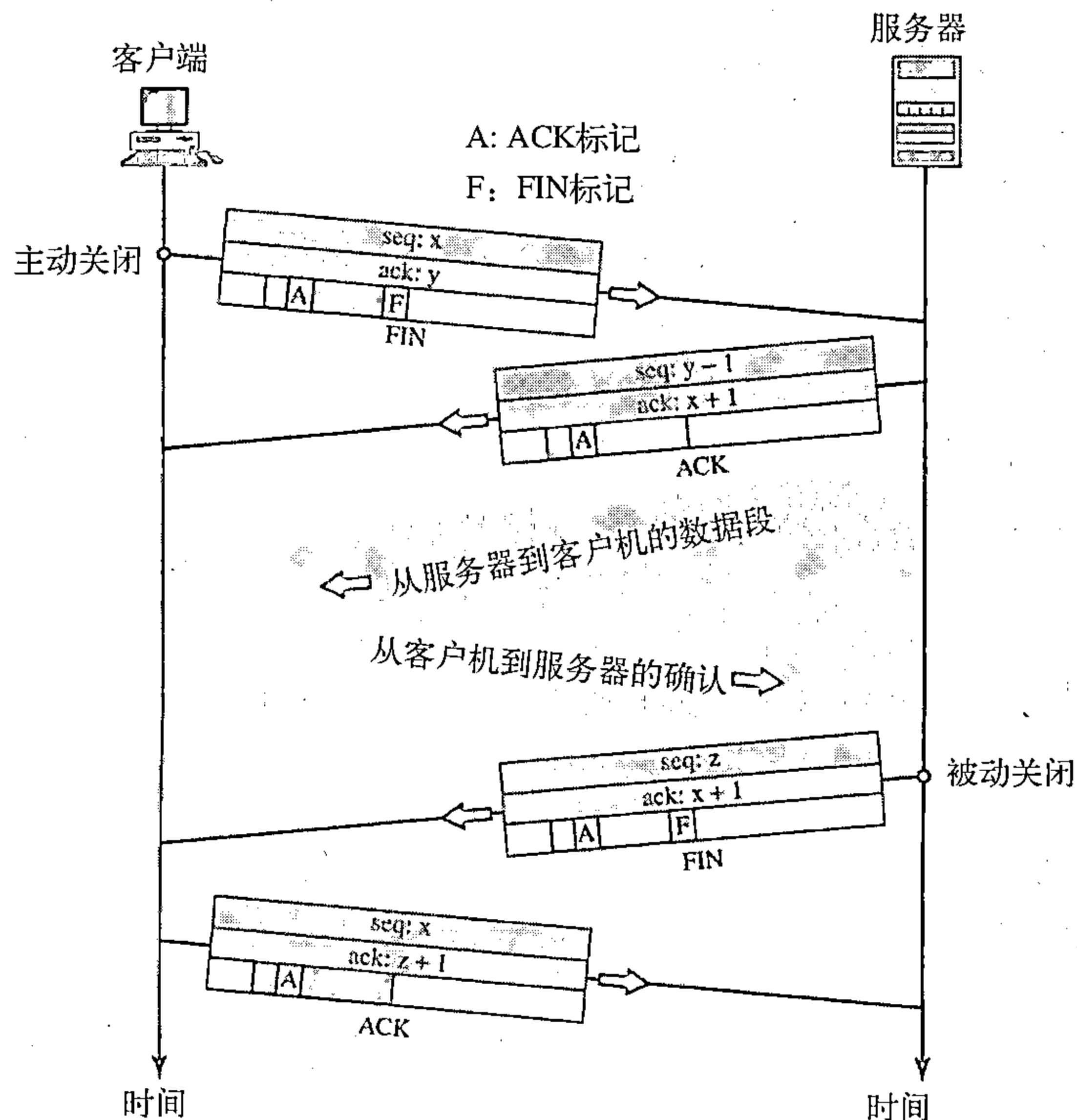


图23.21 半关闭

连接半关闭后，数据可从服务器传送给客户端，而确认可从客户端传送给服务器。注意所用的序列号，第二个段（ACK）没有占用序列号。虽然客户端已接收了序列号 $y-1$ ，正在期望的是 y ，但服务器的序列号还是 $y-1$ 。当连接最后关闭时，最后ACK段的序列号还是 x 。因为在那个方向传输数据时，不占用序列号。

23.3.5 流量控制

如第11章讨论的那样，TCP使用滑动窗口处理流量控制。TCP使用滑动窗口协议。但是，TCP使用的滑动窗口协议介于回退N帧与选择重发之间的滑动窗口。TCP提供的滑动窗口协议由于它不使用NAK，看起来像回退N帧协议；由于接收方保存失序的段直到丢失的段到达，它看起来又像选择重发。可是，这里的滑动窗口与数据链路层所用的滑动窗口有两大点不同。第一点，TCP的滑动窗口是面向字节的，而数据链路层讨论的滑动窗口是面向帧的；第二点，TCP的滑动窗口是可变大小，而数据链路层讨论的滑动窗口是固定大小。

图23.22表示了TCP中的滑动窗口，窗口覆盖了存储着从进程接收到的字节的一部分缓冲区。窗口内的字节都是在传输中的字节，可以发送它们而不必考虑来自另一端的确认。这个设想的窗口有两个边沿：左边沿和右边沿。

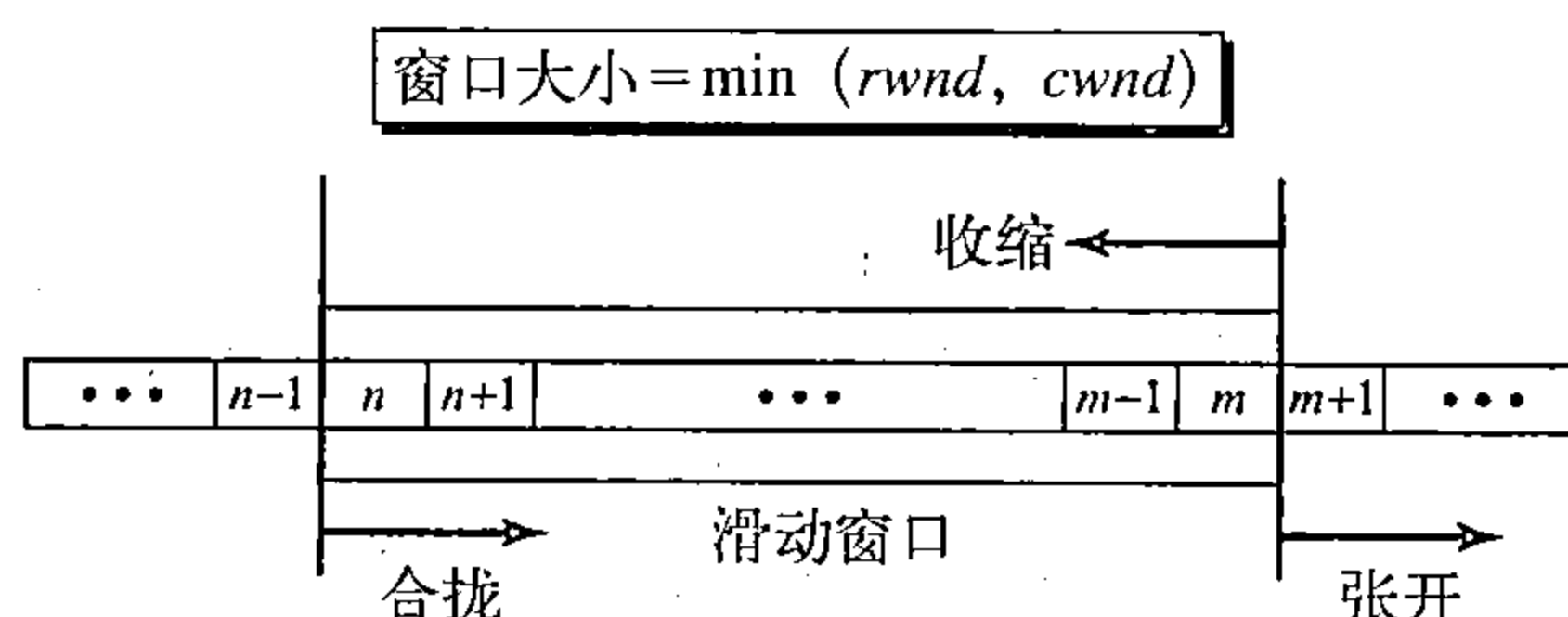


图23.22 滑动窗口

窗口是张开的、合拢的或收缩的。正如我们将看到的，这三种活动是由接收方控制（取决于网络中的拥塞）而不是发送方。在这个问题上，发送方必须服从接收方的命令。

窗口张开是指右边沿向右移动，它将允许缓存区中存储符合发送条件更新的字节。窗口合拢是指左边沿向右移动，这就是说有些字节已被确认，发送方不必再为这些字节而担忧。窗口收缩是指右边沿向左移动。因为这种活动废除某些符合条件的字节发送，因此强烈建议在实现中不使用它。如果发送方已发送过这些字节了，这就有问题了。注意：左边沿不能向左移动，因为这将宣告先前发送的某些确认无效。

使用滑动窗口可使传输更加有效，同时也可以控制数据流，使得目的端不致因数据来得过多而瘫痪。TCP的滑动窗口是面向字节的。

一端的TCP窗口大小取决于两个值中较小的一个：接收方窗口（rwnd）值和拥塞窗口（cwnd）。接收方窗口值是在另一端的确认段中宣布的值，它就是另一端在它的缓存溢出和数据被丢弃之前能够接收的字节个数。拥塞窗口值是网络防止拥塞所确定的值，我们将在本章后面讨论拥塞。

例23.4

如果接收方主机B有一个5 000个字节的缓冲区，已接收到但并未处理的1 000个字节数据。试问主机A的接收方窗口（rwnd）的值是多少？

解：

$rwnd$ 的值 = $5\,000 - 1\,000 = 4\,000$ ，主机B在它的缓冲溢出之前仅能接收4 000个字节。主机B在下一段向主机A宣布。

例23.5

如果 $rwnd$ 值是3 000，而 $cwnd$ 值是3 500，试问主机A的窗口大小是多少？

解：

窗口大小是 $rwnd$ 和 $cwnd$ 中较小的一个，它应是3 000个字节。

例23.6

图23.23表示了一个不切实际的滑动窗口，发送方已发送了202个字节。我们假定 $cwnd$ 是20（实际上，这个值是上千字节），接收方已发一个 $rwnd$ 为9（实际上，这个值也是上千字节）而确认号为200的ACK段。发送窗口的大小是 $rwnd$ 和 $cwnd$ 的最小值，即9个字节。字节号200到202都已发送但未被确认。字节号203到208可以发送而不必考虑从另一端来的确认。字节号209和它以上各字节不可能发送。

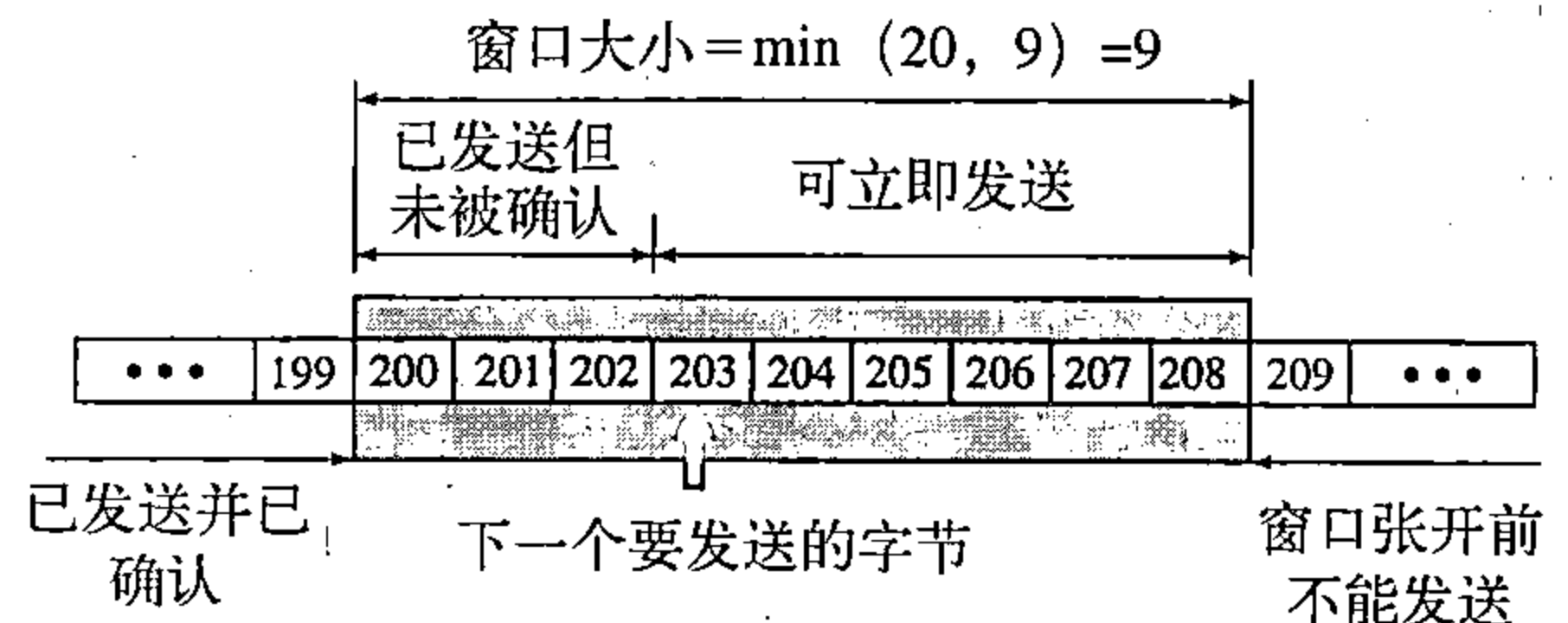


图23.23 例23.6

TCP滑动窗口的要点如下：

- 窗口大小是 $rwnd$ 和 $cwnd$ 中的最小值；
- 发送方不必发送一个全窗口大小的数据；
- 接收方可张开或合拢窗口，但不能收缩窗口；
- 只要不引起窗口收缩，目的方随时可发送一个确认；
- 接收方可暂时关闭窗口，但在窗口关闭后发送方总是发送一个1字节的段。

23.3.6 差错控制

TCP是一个可靠的传输层协议。这意味着将数据流传递给TCP的应用程序依靠TCP将整个数据流传递给另一端的应用程序，并且是按序的、无差错的，没有任何一部分丢失或重复的。

TCP使用差错控制提供可靠性。差错控制包括一些机制：检测受到损坏的段、丢失的段、失序的段和重复的段。差错控制还包括检测出差错后纠正差错的机制。TCP中的差错检测和纠正是通过三种简单工具来完成：校验和、确认和超时。

校验和

每个段都包括校验和字段，用来检查受到损坏的段。如果段被损坏，它将被目的端TCP丢弃，并认为是丢失了。TCP在每段中强制使用一个16位的校验和。在第24章中，我们将会看到这16位的校验和对于新的传输层协议SCTP是不适合的。因为SCTP将整个头部格式重新配置，而TCP不可能改变它。

确认

TCP使用确认方法来证实收到了数据段。不携带数据但占用序列号的一些控制段也要确认，但ACK段是不要确认的。

ACK段不占用序列号，它不需要确认。

重传

差错控制机制的核心是段的重传。当一个段损坏、丢失或延迟时，就重传这个段。在当前实现中，有两种情况要重传段：重传计时器（retransmission timer）到时或当发送方收到三个重复的ACK时。

在当前的实现中，如果重传计时器到时或当发送方收到三个重复的ACK时，则发生重传。

注意：不占用序列号的段不重传。尤其是，ACK段不重传。

对ACK段不设置重传计时器。

RTO后重传 TCP对所有重要的段（已发送，但还未被确认）的最新实现使用了一个重传

超时 (retransmission time-out, RTO) 计时器。当计时器到时,即使可能是由于段被延迟、ACK被延迟或丢失确认等未接收到ACK时,重发一个最早的重要的段。注意:对仅携带确认的段没有设置超时计时器,这说明这样的段不需要重发。在TCP中,RTO的值是动态的,根据段的往返时间(round-trip time, RTT)为进行更新。RTT是一个段到达目的端并接收到一个确认所需要的时间,它使用往返策略与第12章中讨论的相似。

三个重复ACK段之后的重发 如果RTT的值不是很大,段重发前的规则是可用的。但有时某一段丢失了,而接收端收到了许多失序的段,它们不可能都被存储(有限的存储器大小)。为了缓解这种情况,当今大多数实现遵循三次重复ACK规则,立即重发缺少的段。这一特点称为**快速重传 (fast retransmission)**,在稍后的例中我们将会看到。

失序的段

当一个段被延迟、丢失或废弃时,后面一些段的到达就失序了。TCP最初的设计是丢弃所有失序的段,并重传那个缺少的段以及其后的一些段。当今大多数的实现不丢弃失序的那些段,暂时将它们存储,并标记它们为失序的段直到缺少的那个段到达。但是注意:这些失序的段不传递给进程,TCP确保数据按序传递给进程。

数据可以失序到达,并被接收的TCP暂时存储。但是TCP确保传递给进程的段是无失序的。

某些情况

在本节中,我们给出TCP操作时某些情况的例子。在这些情况中,用长方形表示段。如果段携带数据,我们显示字节序列号的范围和确认字段的值。如果段仅是一个确认,我们仅用小方框中的确认号表示。

正常操作 第一种情况表示了两个系统之间的双向数据传输,如图23.24所示。客户TCP发送一个段,而服务器TCP发送三个段。图23.24表示了应用于每个确认的规则。有数据要发送,所以该段显示期望接收的下一个字节序号。当客户端接收到来自服务器的第一个确认时,它没有更多的数据要发送,它仅发送一个ACK段。但是,该确认段需要延迟500ms以观察是否还有更多的段到达。当计时器到时,客户端触发一个确认。这样做是因为客户端不知道是

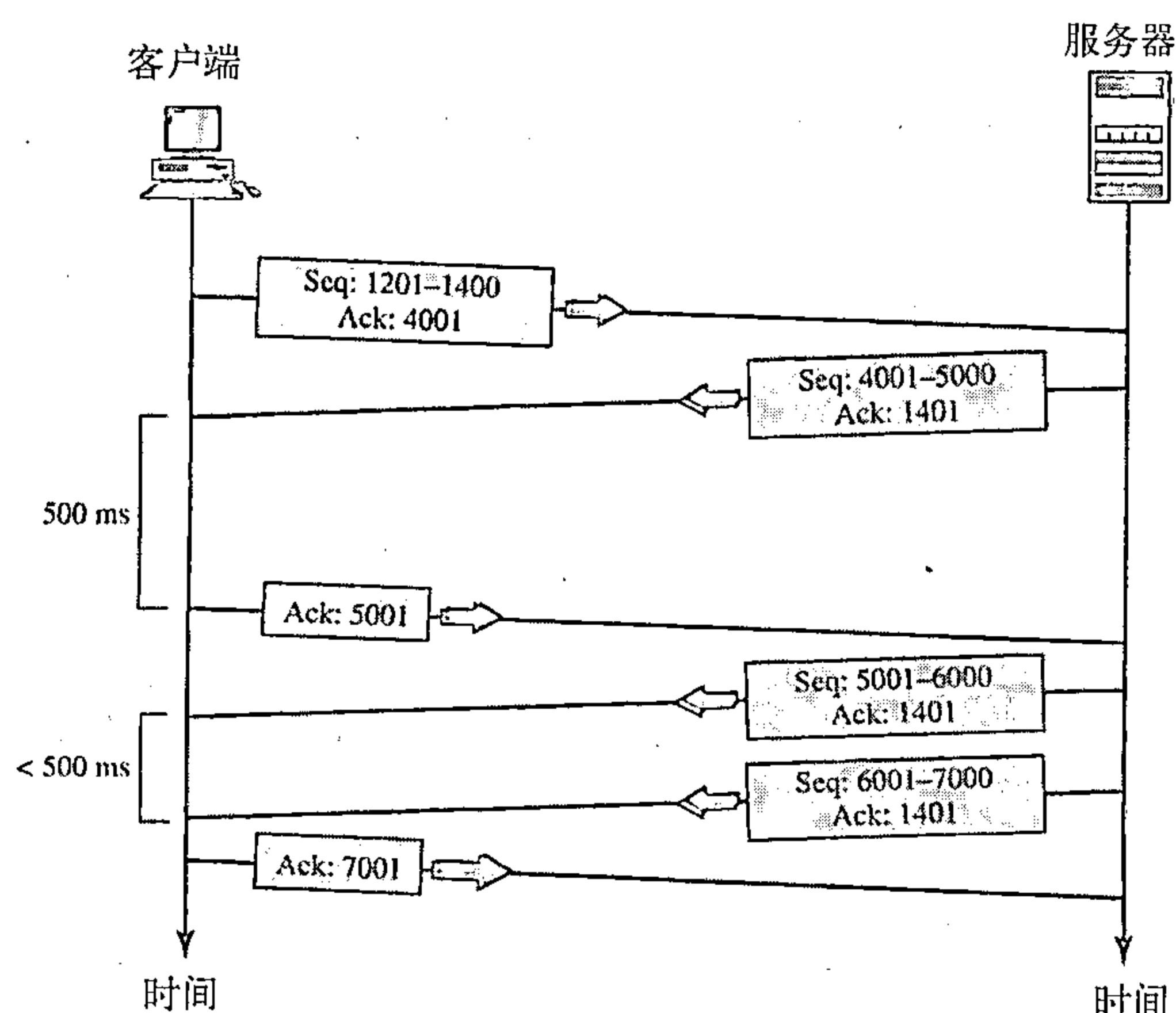


图23.24 正常操作

否还有其他的段到来，它不再延迟这个确认。当下一个段到达时，启动另一个确认计时器。但是，在它到时之前，第三个段到达，第三个段的到达触发另一个确认。

丢失的段 在这种情况下，我们说明了当一个段丢失或损坏时，将会发生什么。接收方对丢失的段和损坏的段用同样的方法处理。丢失的段是在网络中的某处被丢失的，损坏的段是被接收方本身丢弃的。图23.25表示了一种情况，一个段被丢失，被在网络中的某个路由器丢弃掉，或许由于拥塞引起。

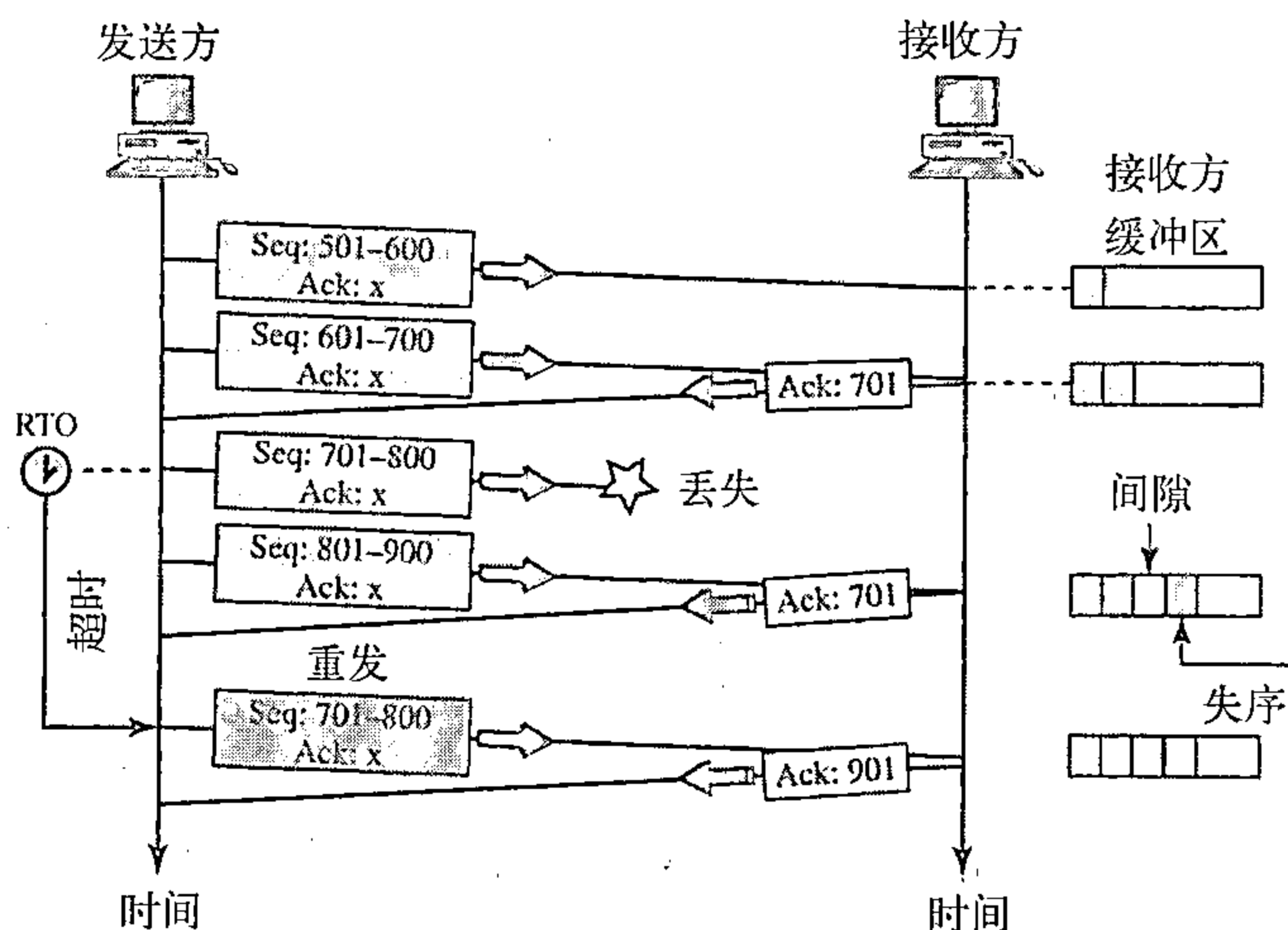


图23.25 丢失的段

我们假定数据传输是单向的：一方发送，另一方接收。在这个特定情况下，发送方发送段1和段2，这两个段立即被一个ACK确认。但是，段3被丢失了，接收方接收到段4，发生了失序。接收方将数据存储在其缓冲区的段中但留出一个间隙指明数据中存在不连续性。接收方立即给发送方发送一个确认，表示了它期望的下一个字节。注意：接收方存储从801到900的字节，但在这个间隙被填充之前不将这些字节传递给应用程序。

接收方TCP仅将有序的数据传递给进程。

我们已表示了最早的重要段的计时器，因为接收方永远不会给丢失或失序段发送确认，因此这个定时器肯定会超时。当该计时器到时，发送方TCP重发段3，这个到达时才真正地被确认。注意：按照相应的规则，第二个确认和第三个确认的值是不同的。

快速重传 在这个情况中，我们要表示快速重传的思想。除了RTO具有较大的值外，该情况与第二种情况相同（见图23.26）。

当接收方接收到第四个、第五个和第六个段时，它触发一个确认。发送方

接收到4个具有相同值的确认（3个是重复的）。虽然段3的计时器还没有到时，但快速传输要

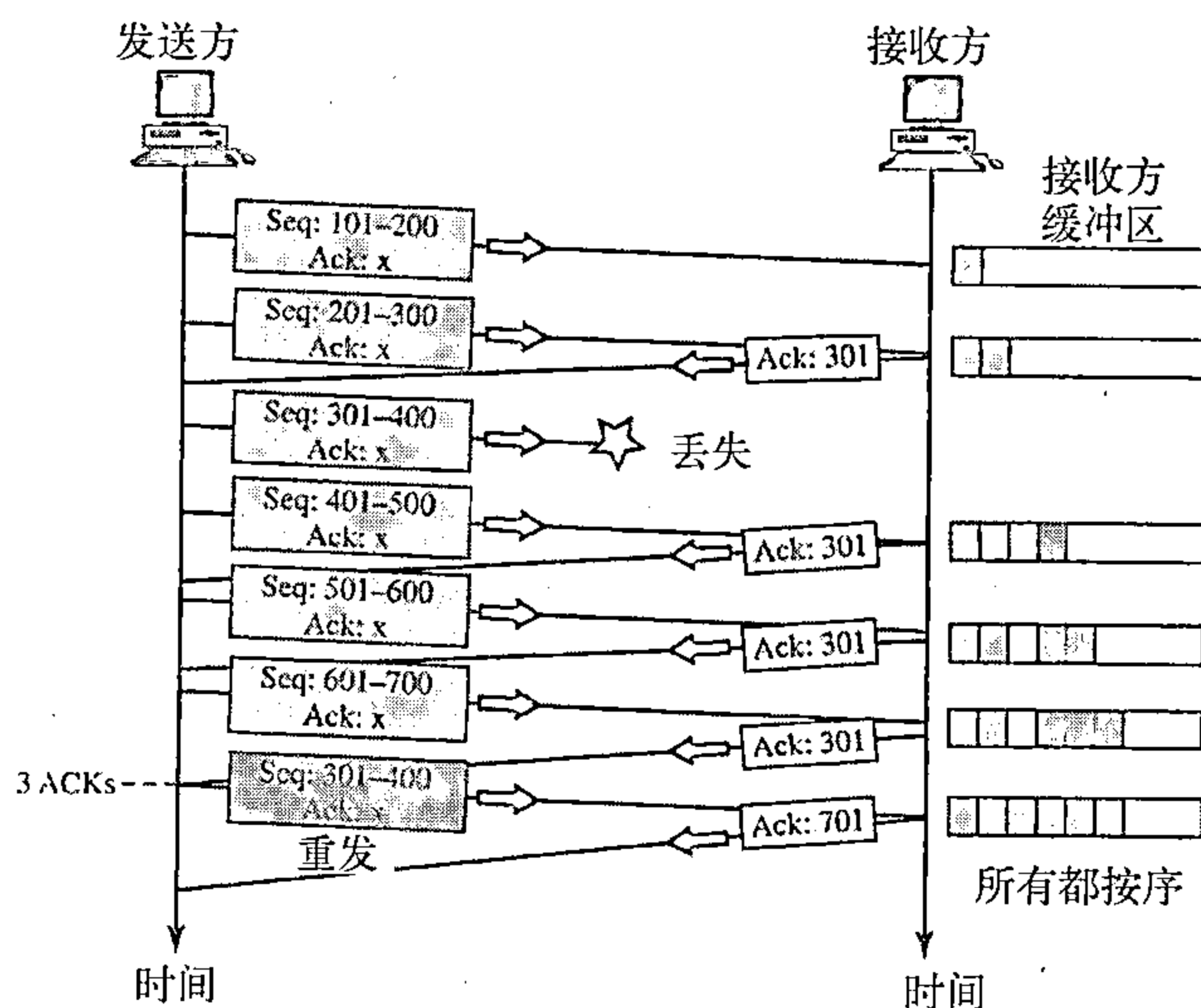


图23.26 快速重传

求立即重发段3，该段是所有这些确认所期望的。注意：虽然4个段都没有被确认，但仅仅重发一个段。当发送方接收到这个重发的ACK时，因为确认是累计的，因此它知道这4个段是安全的和可靠的。

23.3.7 拥塞控制

我们将在第24章讨论TCP的拥塞控制。

23.4 SCTP

流控制传输协议（SCTP）是一种新的可靠的、面向报文的传输层控制协议。然而，SCTP 主要是为最近引入的因特网应用而设计的。这些新的应用，如IUA（IP上的ISDN）、M2UA和M3UA（电话信令）、H.248（媒体网关控制）、H.323（IP电话）和SIP（IP电话）等都需要提供比TCP更复杂的服务。SCTP提供了更高的性能和可靠性。我们简要比较UDP、TCP和STCP的性能。

- UDP是一个面向报文（message-oriented）的协议，进程将报文传递给UDP，将它封装在一个用户数据报中，通过网络发送。UDP保留报文边界，每个报文与其他报文无关。正如我们在本书后面将看到的，当我们处理一些应用，如IP电话和实时数据传输时，这个特性是我们想要的。但是UDP是不可靠的，发送方不知道报文的命运，报文可能会丢失、重复或失序。UDP也还缺乏其他特性，如缺少友好的传输层协议所需要的拥塞控制和流量控制。
- TCP是一个面向字节（byte-oriented）的协议。它从进程中接收一个报文或多个报文，并以一个字节流的方式存储它们，并以段的方式发送它们。没有保存报文的边界。但是，TCP是一个可靠的协议，重复段会被删除，丢失的段会重发，并且字节按序传递到端进程。TCP还有拥塞控制和流量控制机制。
- SCTP兼有UDP和TCP的特性。SCTP是一个可靠的面向报文的协议，它保存报文的边界，同时它检测丢失的数据、重复的数据和失序的数据。它还有拥塞控制和流量控制机制。稍后我们将会看到SCTP还有UDP和TCP中没有的一些新特性。

SCTP是一个面向报文的可靠的协议，它兼有UDP和TCP好的特性。

23.4.1 SCTP服务

在我们讨论SCTP操作之前，让我们先说明SCTP向应用层进程提供的服务。

进程到进程通信

SCTP使用TCP空间中的熟知端口号，表23.4列出了SCTP使用的一些额外的端口。

多流

我们在前一节讨论过TCP是一种面向流的协议，TCP客户端与TCP服务器的每一次连接都包含了一个单一的流。这个方法存在的问题是在流中任何一点的丢失会阻塞其余数据的传递。当传输文本时，这还可以接

受；但当传输实时数据（音频或视频）时，这就不能接受了。SCTP在每一次连接中提供多流服务（multistream service），用SCTP专门名词称为关联。如果多流中某一个被阻塞，其余的

表23.4 某些SCTP应用

协 议	端 口 号	说 明
IUA	9990	IP上的ISDN
M2UA	2904	SS7电话信令
M3UA	2905	SS7电话信令
H.248	2945	媒体网关控制
H.323	1718, 1719, 1720, 11720	IP电话
SIP	5060	IP电话

流仍然可传递它们的数据。这种思想类似于高速公路上的多车道，每一个车道可用于不同类型的交通量。例如，一条车道用于正常的交通，另一条车道用于合伙车。如果正常的交通受到阻塞，合伙车辆依旧可达目的地。图23.27表示了多流传递的思想。

SCTP的一次关联能包含多个流。

多接口

一个TCP连接包括了一个源IP地址和一个目的IP地址。这就是说，即使发送方或接收方是一个多接口主机（即连接到带有多个IP地址的多个物理地址上），在连接期间每端的这些IP地址中仅有一个是可用的。另一方面，SCTP关联支持多接口服务（multihoming service）。发送主机和接收主机对每一次关联在每一端可定义多个IP地址。在这个容错方法中，当一条通路发生故障时，可以用另一个接口不中断地传递数据。当发送和接收实时有效载荷时，如因特网上的电话，这个特性是很有用的。图23.28表示了多接口的思想。

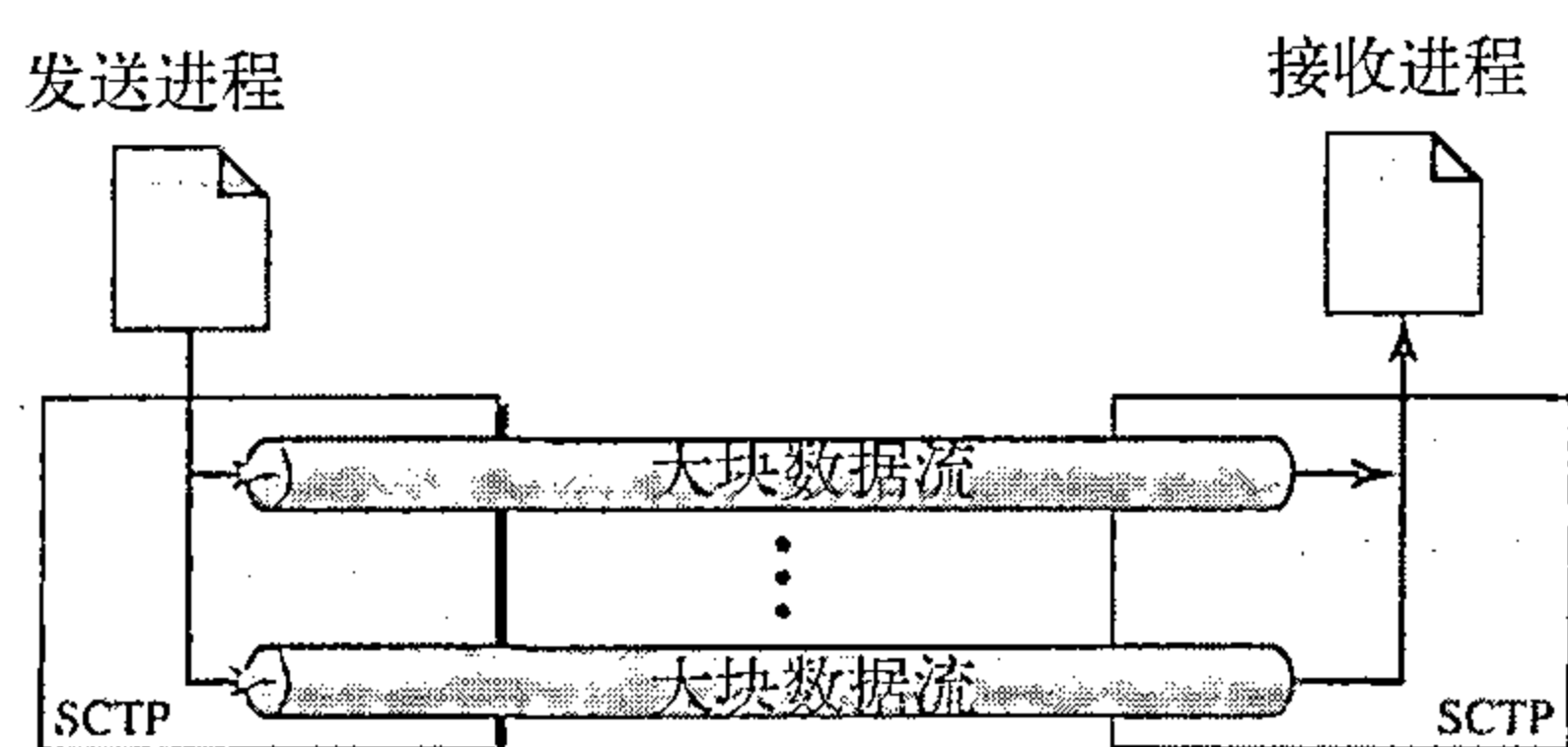


图23.27 多流的概念

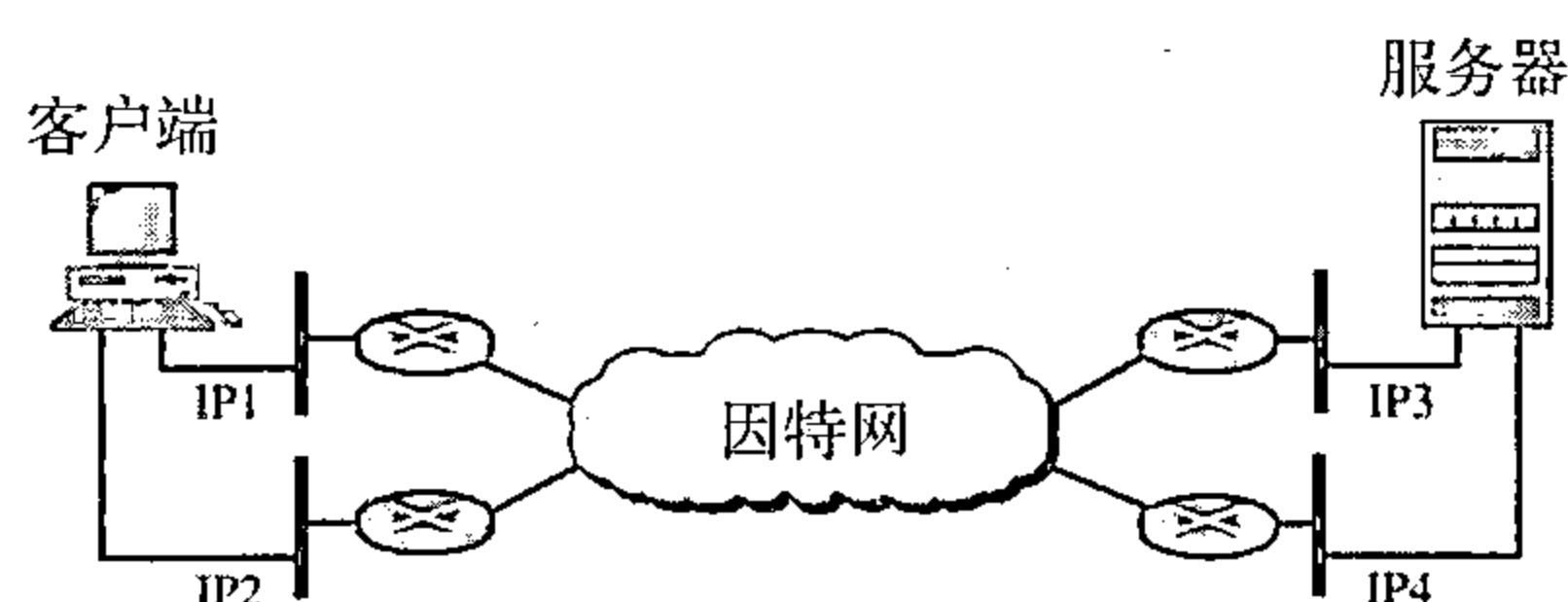


图23.28 多接口概念

在图23.28中，客户端用两个IP地址分别连接两个局域网络，服务器也用两个IP地址分别连接两个网络。客户端和服务端可用四个不同的IP地址对做出一个关联。但是应注意，在目前SCTP的实现中，仅有一对IP地址可用于正常的通信；当选用的主路径有故障时，可使用替换路径。换言之，目前SCTP不允许不同路径共用。

SCTP关联允许每个端有多个IP地址。

全双工通信

像TCP一样，SCTP提供全双工服务。在全双工服务中，数据可同时在两个方向流动。每一个SCTP都有一个发送和接收缓存区，在两个方向都可以发送分组。

面向连接服务

像TCP一样，SCTP是面向连接的协议。但是，在SCTP中，连接被称为关联。站点A的进程要向站点B的另一个进程发送或接收数据时，其步骤如下：

1. 两个SCTP彼此建立关联；
2. 双方向交换数据；
3. 终止关联。

可靠的服务

像TCP一样，SCTP是可靠的传输协议。它利用确认机制检测数据是否安全和可靠地到达。我们将在差错控制这一节进一步讨论这个特性。

23.4.2 SCTP特性

首先让我们讨论SCTP的一般特性，然后将这些特性与TCP特性相比较。

传输序列号

TCP的数据单元是字节，在TCP中用序列号对字节编号控制数据传输。另一方面，SCTP的数据单元是数据大块（DATA chunk）。由于分段原因，数据大块与来自进程的报文可能有也可能没有一一对应的关系（以后讨论）。在SCTP中用数据大块的编号控制数据传输，SCTP使用传输序列号（transmission sequence number, TSN）对数据大块进行编号。换言之，SCTP中的TSN所起的作用类似于TCP中的序列号。TSN是32位长，初始值可取0到 $2^{32}-1$ 之间的随机数。每个数据大块的头部必须有相应的TSN。

在SCTP中，数据大块按TSN编号。

流标识符

在TCP中，每一次连接仅有一个流。而在SCTP中，每次关联可以有多个流。因此，每个流需要用流标识符（SI）进行标识。每个数据大块的头部必须有SI，这样当它到达目的端时，它可正确地放入它的流中。SI是从0开始的16位数字。

为了区别不同的流，SCTP使用SI。

流序列号

当一个数据大块到达目的SCTP时，它被传递到适当的流中，并按原序排列。这就是说，除了SI之外，SCTP还用流序列号（stream sequence number, SSN）对每个流中的每个数据大块进行定义。

为了区别属于同一个流中的不同数据大块，SCTP使用SSN。

分组

在TCP中，段携带数据和控制信息。数据作为一组字节被携带，而控制信息是用它头部的6个控制标记来定义。而SCTP的设计完全不同：数据作为数据大块被携带，控制信息作为控制大块被携带。多个控制大块与数据大块可合并成一个分组。SCTP中一个分组起的作用与TCP中的段相同。图23.29比较了TCP的段与SCTP的分组。让我们扼要列出SCTP的分组和TCP的段不同：

TCP有段；SCTP有分组。

1. TCP中的控制信息在头部，而SCTP中的控制信息是在控制大块中。有几种类型的控制大块，每个类型用于不同的目的；

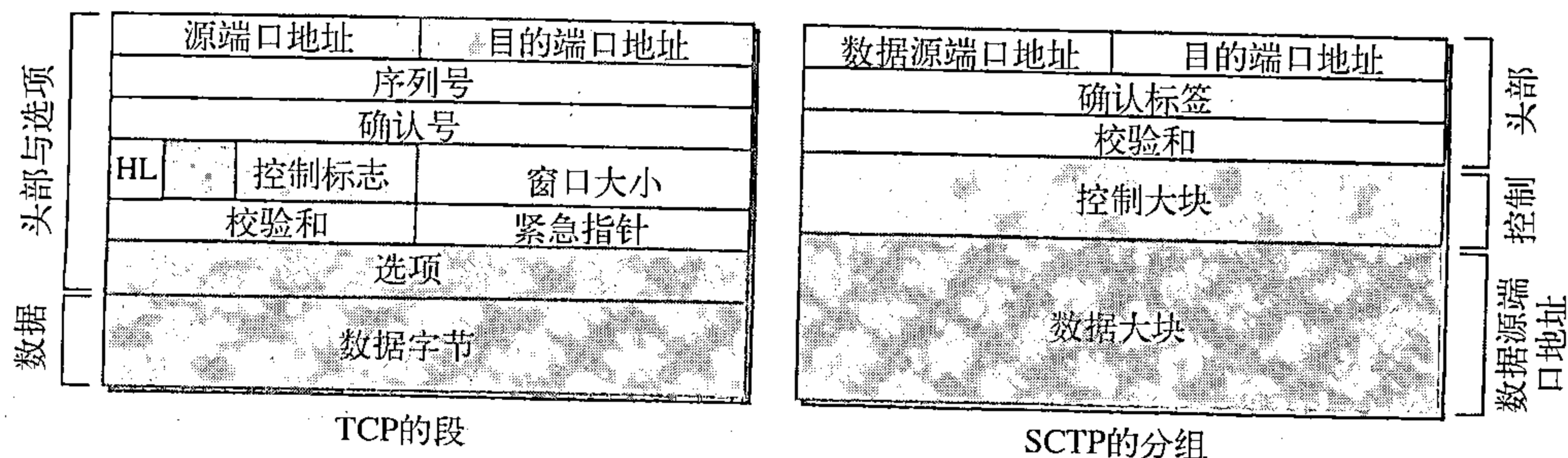


图23.29 TCP段与SCTP分组的比较

2. 一个TCP段中的数据作为一个实体处理；而一个SCTP分组可以携带多个数据大块，每

个可能属于不同的流。

3. 选项可以是TCP段的一部分；而SCTP分组不存在选项，SCTP中的选项是由定义新的大块字节来处理的。

4. TCP头部的强制性部分是20个字节；而SCTP头部仅是12个字节。SCTP头部较短是由于下列因素：

- a. SCTP序列号 (TSN) 属于每个数据大块，因此它位于数据大块的头部；
 - b. 确认号和窗口大小都是每个控制大块的一部分；
 - c. SCTP头部的长度是固定的 (12个字节)，因此不需要头长度字段 (TCP段中用HL表示)，不存在用选项生成头部的长度；
 - d. 在SCTP中没有紧急指针。
5. TCP中的校验和是16位；而SCTP中的校验和是32位。

6. SCTP中确认 (verification) 标签是关联标识符；而在TCP中是没有的。TCP中的IP地址和端口地址一起定义了一个连接；而在SCTP中，用不同的IP地址实现多接口。定义每个关联都需要唯一的确认标签。

7. TCP段的头部有一个序列号，它定义数据部分中的第一个字节编号；而SCTP分组可包含多个数据大块，用TSN、SI和SSN定义每个数据大块。

8. TCP中有些段携带控制信息 (如SYN和FIN)，它要占用一个序列号；在SCTP中，控制大块不使用TSN、SI和SSN。这三个标识符仅属于数据大块，而不是属于整个大块。

在SCTP中，控制信息和数据信息在分开的大块中携带。

在SCTP中，有数据大块、流和分组。一个关联可发送多个分组，一个分组可包含多个大块，而各个大块可属于不同的流。为了使这些术语定义得更清晰，让我们假定进程A用3个流发送11个报文给进程B。前面的4个报文用第一个流，其次的3个报文用第二个流，最后的4个报文用第三个流。

虽然一个长的报文可能被多个数据大块携带，但是我们还是假定每个报文装载在一个数据大块中。因此，我们在3个流中有11个数据大块。

应用进程传递11个报文给SCTP，这里对每个报文标上适合的流记号。虽然进程可传递第一个流的一个报文，然后再传递第二个流的另一个报文。但是，我们假定首先传递属于第一个流的所有报文，然后再传递属于第二个流的所有报文，最后传递属于第三个流的所有报文。

我们还假定网络仅允许每个组有三个数据大块。这就是说，我们需要4个分组如图23.30所示。第一个分组和一部分第二个分组流0中的数据大块，第二个分组和第三个分组携带流1中的数据大块，第三个分组和第四个分组携带流2中的数据大块。

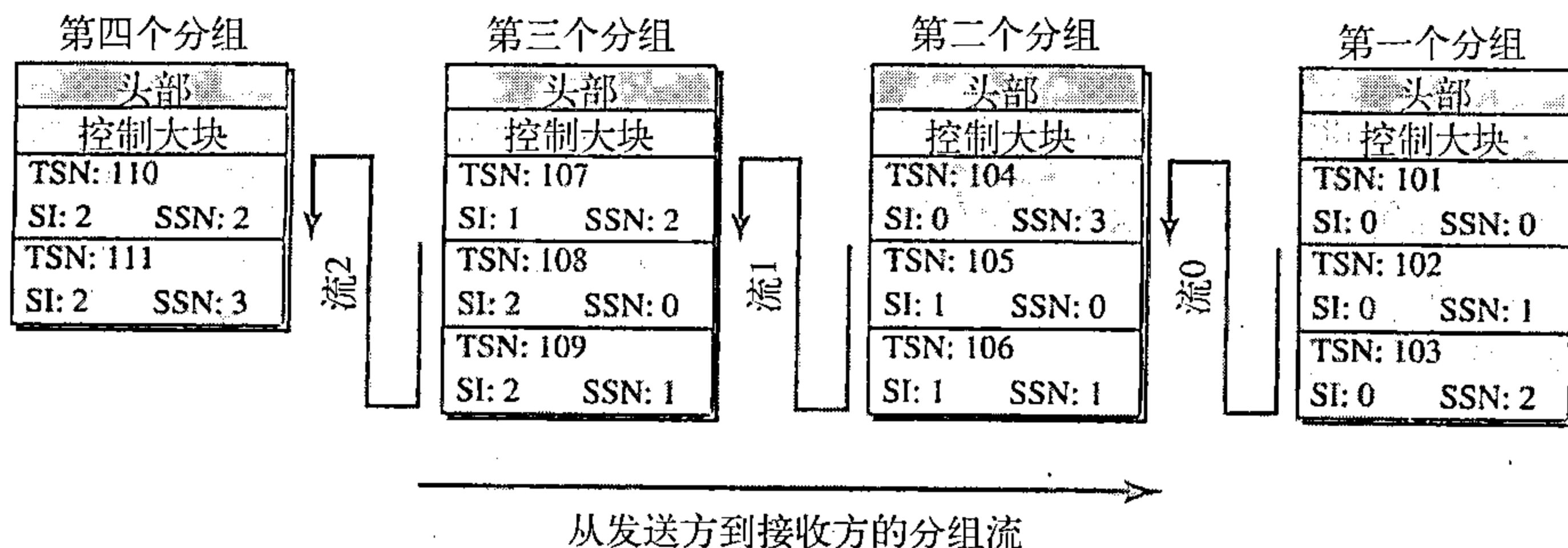


图23.30 分组、数据大块和流

注意每个数据大块需要三个标识符：TSN、SI和SSN。TSN是累计数，将会看到它也用于流量控制和差错控制。SI定义这些块所属的流。SSN定义了在一定流中数据大块的次序。在我们的例子中，对每个流，SSN都从0开始编号计数。

数据大块由三个标识符来标识：TSN、SI和SSN。TSN是用来标识关联的累计数；SI定义流；SSN定义流中的块。

确认号

TCP确认号是面向字节的，它指的是序列号，而SCTP确认号是面向大块的，它指的是TSN。TCP与SCTP确认号的第二个不同是控制信息，在TCP中控制信息是段头部的一部分。为了让确认段仅携带控制信息，TCP使用序列号和确认号（例如，SYN段需要用ACK段确认）。但在SCTP中，控制大块携带控制信息，它不需要TSN。而这些控制大块都由另一个适当类型的控制大块来确认（有些不需要确认）。例如，INIT控制大块是由INIT ACK大块来确认，它不需要序列号或确认号。

在SCTP中，确认号仅用于对数据大块的确认。如果需要的话，控制大块用另一个控制大块来确认。

流量控制

像TCP一样，SCTP实现流量控制以避免接收方崩溃，我们将在本章后面讨论SCTP的流量控制。

差错控制

像TCP一样，SCTP实现差错控制以提供可靠性。TSN号和确认号用于差错控制，我们将在本章后面讨论差错控制。

拥塞控制

像TCP一样，SCTP实现拥塞控制以决定多少个数据大块可置于网络中。我们将在第24章讨论拥塞控制。

23.4.3 分组格式

在这一节，我们说明了分组格式和大块的各种类型。在本节中所列出的大多数信息将在后面变得更加清楚。第一次阅读可以跳过本节或仅作为参考。SCTP分组有一个强制性的通用头部和一个称为大块的块的集合。大块有两种类型：数据大块和控制大块。控制大块控制和维持关联，而数据大块携带用户数据。在一个分组中，控制大块在数据大块之前。图23.31表示了SCTP的分组一般格式。

在SCTP分组中，控制大块在数据大块之前。

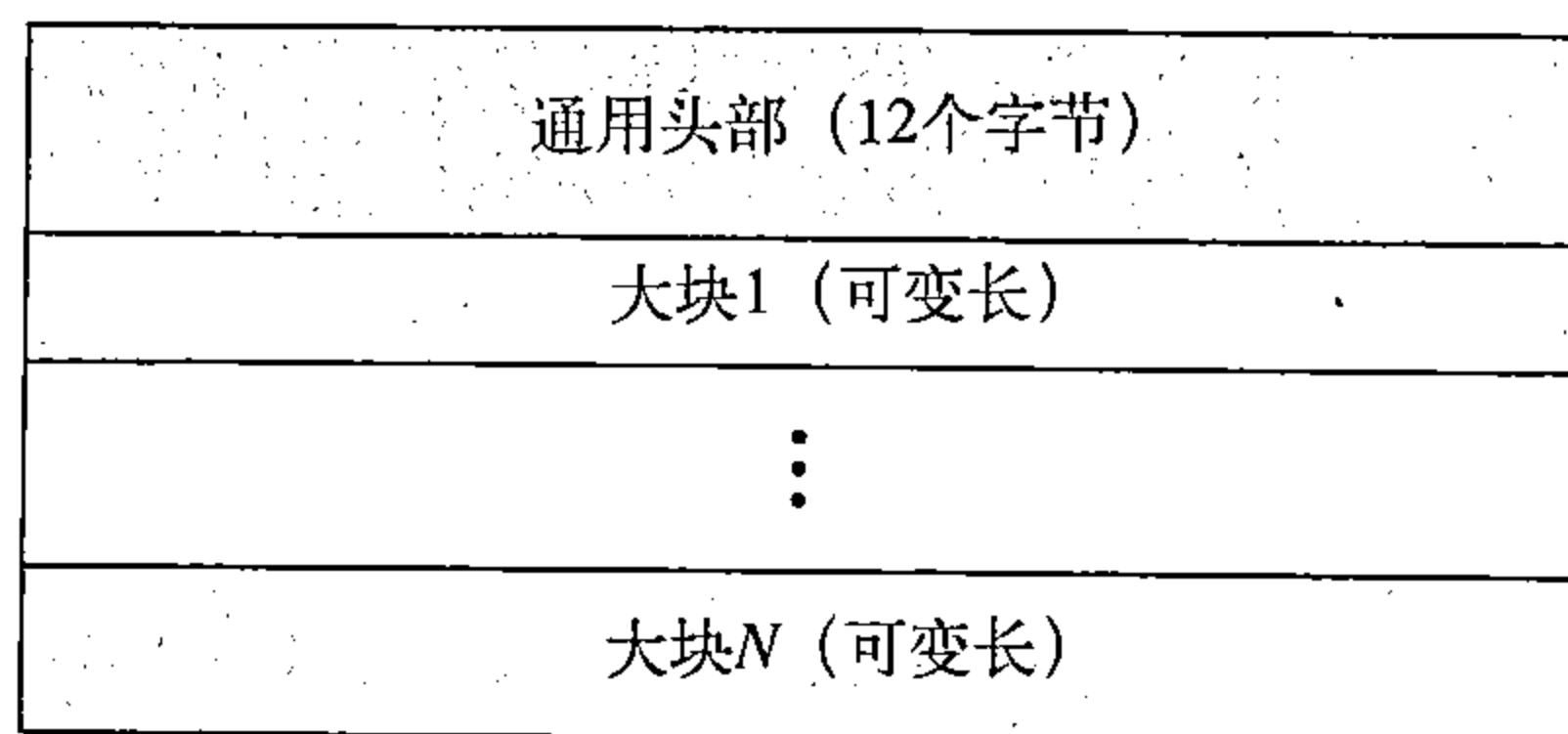


图23.31 SCTP的分组格式

通用头部

通用头部 (general header) 定义分组属于每个关联的两端，保证了分组属于一个特定的关联，并保护分组内容包括本身头部的完整性。通用头部格式如图23.32所示。

通用头部有4个字段：

- 源端口地址。这是一个16位字段，它定义进程发送分组的端口号；
- 目的端口地址。这是一个16位字段，它定义进程接收分组的端口号；
- 确认标签。这是一个数，它将一个分组与一个关联相匹配。它防止将以前关联的一个分组误认为是这个关联中的一个分组。它作为关联的一个标识符。在关联期间，它在每个分组中都是相同的。在关联中，每个方向有各自的确认。
- 校验和。这是一个32位字段，它包含CRC-32校验和。注意校验和的长度从16位（UDP、TCP和IP）增加到32位，并允许用CRC-32校验和。

源端口地址 16位	目的端口地址 16位
确认标签 32位	
校验和 32位	

图23.32 通用头部

大块

大块携带控制信息或用户数据，每块的详细格式超出了本书的范围，详细内容可阅读[For06]。所有大块的前三个字段是相同的，信息字段取决于大块类型。最重要的是，SCTP要求信息部分是4字节的倍数；如果不是，在信息部分的末尾增加填充字节（8个0）。一些大块和它们的说明见表23.5。

表23.5 大块

类 型	大 块	说 明
0	DATA	用户数据
1	INIT	建立关联
2	INIT ACK	INIT大块的确认
3	SACK	选择性的确认
4	HEARTBEAT	探测对等方的可访问性
5	HEARTBEAT ACK	确认HEARTBEAT大块
6	ABORT	异常中止关联
7	SHUTDOWN	关闭关联
8	SHUTDOWN ACK	确认SHUTDOWN大块
9	ERROR	报告没有关闭错误
10	COOKIE ECHO	关联建立中的第三个分组
11	COOKIE ACK	确认COOKIE ECHO大块
14	SHUTDOWN CPMLETE	关闭关联中的第三个分组
192	FORWARD TSN	调整累计TSN

23.4.4 SCTP关联

像TCP一样，SCTP是一个面向连接的协议。但是，为了强调其多接口性，SCTP中的连接被称为关联。

SCTP中的连接被称为关联。

关联建立

SCTP中的关联建立（association establishment）要求四次握手（four-way handshake）。在这个过程中，一个进程（通常是客户）想要与另一个进程（通常是服务器）建立关联，使用SCTP作为传输层协议。与TCP相似，SCTP服务器需要准备接收一个关联（被动打开），

而由客户发起建立关联（主动打开）。SCTP关联建立，如图23.33所示。在正常情形下，步骤如下：

1. 客户端发送第一个分组，它是一个INIT大块（INIT chunk）；
2. 服务器发送第二个分组，它是一个INIT ACK大块（INIT ACK chunk）；
3. 客户端发送第三个分组，它包含一个COOKIE ECHO大块（COOKIE ECHO chunk），这是一个很简单的大块，它毫无改变地回应由服务器发送的cookie。SCTP允许在这个分组中包含数据大块；
4. 服务器发送第四个分组，它包含对接收到COOKIE ECHO大块进行确认的COOKIE ACK大块（COOKIE ACK chunk）。SCTP允许在这个分组中包含数据大块。

携带INIT或INITACK大块的分组中不允许有其他大块；而COOKIE ECHO或COOKIE ACK大块可携带数据大块。

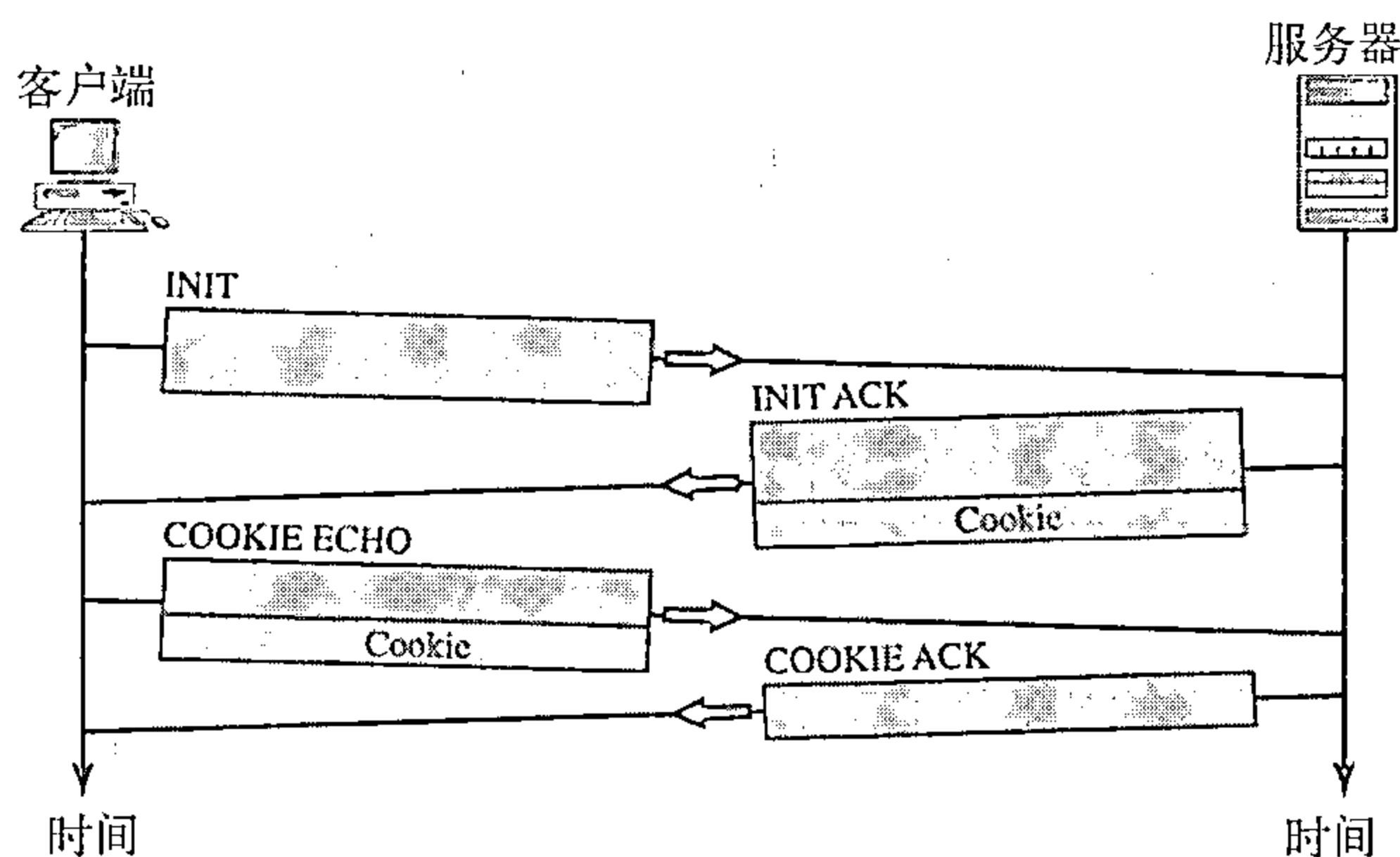


图23.33 四次握手

Cookie 在前几节中，我们讨论过SYN洪泛攻击。利用TCP，一个恶意的攻击者可以向一个TCP服务器发送大量的具有不同假IP地址的伪造的SYN段。每次服务器接收到一个SYN段时，它建立一个状态表并分配更多的资源等待下一个段的到达。然而，不久服务器由于资源耗尽而崩溃。

SCTP设计者使用一种策略防止这种类型的攻击。当发送方IP地址被确认时，这个策略是推迟资源分配直到接收到第三个分组。第一个分组中接收到的信息必须以某种方式保存到第三个分组到达。但是，如果服务器保存该信息，那就需要资源（存储器）的分配，这是一个困境。解决办法是包装该信息，并把它送回到客户。这称为生成一个cookie，这个cookie与第二个分组一起发送给接收第一个分组的地址。有两种可能的情形：

1. 如果第一个分组的发送方是一个攻击者，那么服务器永远不会接收到第三个分组，cookie丢失了，但没有分配资源。服务器的唯一的工作是“生成”（“baking”）cookie。
2. 如果第一个分组的发送方是需要建立连接的一位诚实的客户，那么它接收到带有cookie的第二个分组。它毫无改变地用该cookie发送一个分组（在序列中是第三个分组）。服务器接收到第三个分组而且知道它来自一位诚实的客户，因为发送方发送的cookie已经在那里。现在服务器可以分配资源了。

如果没有实体可以“吃掉”由服务器“生成”的cookie，那么上述策略就可以起作用。为了保证这一点，服务器用它自己的密钥产生该信息的一个摘要（见第30章）。信息和摘要一起生成一个cookie，它在第二个分组中被发送给客户。当cookie在第三个分组中返回时，服务器

计算信息的摘要。如果该摘要与发送的摘要相匹配，那么该cookie就没有被其他的实体修改。

数据传输

关联的目的是为了在两端之间传输数据。关联建立后，就可以进行双方向的数据传输，客户端与服务器都可发送数据。像TCP一样，SCTP支持捎带。

然而TCP与SCTP在数据传输方面存在着很大的不同，TCP从进程接收报文，这些报文作为字节流在其间没有可识别的边界。为了它的对等方的使用，进程可插入一些边界，但TCP将那些标记作为文本的一部分对待。换言之，TCP把每个报文附加到它的缓存中。一个段可携带两个不同的报文。TCP使用的唯一的排序系统是字节编号。

另一方面，SCTP识别并维护边界，来自进程的每个报文作为一个单元处理，并插入到一个DATA大块中除非这个报文被分段（后面讨论）。在这种情况下，SCTP与UDP一样具有一个很大优点是数据大块彼此有关。

从进程接收到的一个报文成为一个DATA大块；如果要分段，则成为几个DATA大块。把DATA大块的头部加到报文上。一个报文或一个报文的分段形成的每个DATA大块都有一个TSN。我们要记住，只有DATA大块使用TNS，也只有DATA大块使用SACK大块进行确认。

在SCTP中，只有DATA大块占用TNS，也只有这种大块要进行确认。

图23.34表示了一个简单数据传输的情形，客户端发送4个DATA大块，从服务器接收两个DATA大块。稍后，我们将讨论SCTP中的流量和差错控制，此时假定在这种情形下一切都顺利。

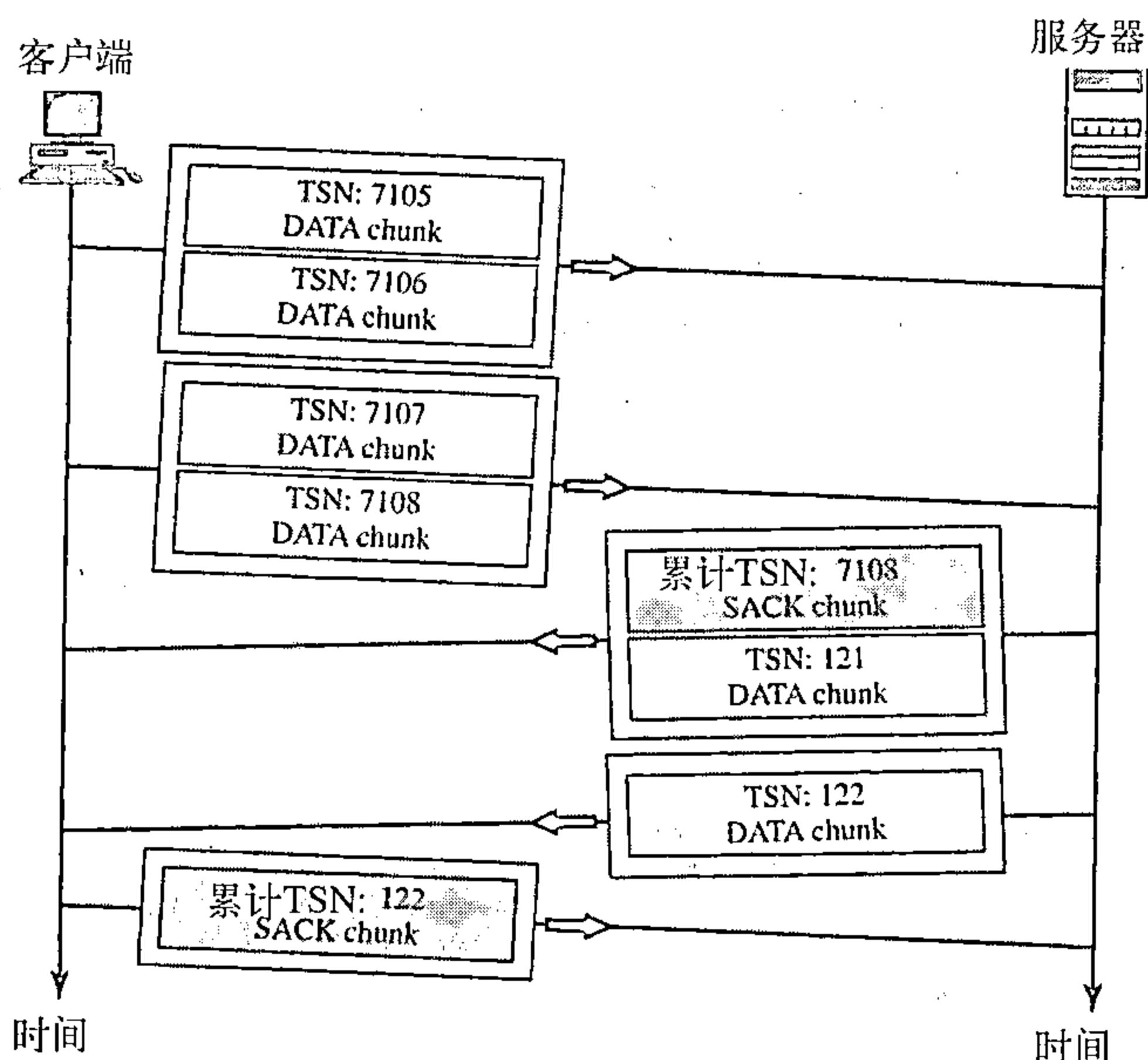


图23.34 简单数据传输

1. 客户端发送第一个分组，它携带TSN分别为7 105和7 106的两个DATA大块。
2. 客户端发送第二个分组，它携带TSN分别为7 107和7 108的两个DATA大块。
3. 第三个分组来自服务器，它包括了对收到来自客户端的DATA大块进行确认所需要的SACK大块。与TCP相反，SCTP只有按顺序收到的最后一个TSN进行确认，而不是下一个所期望的。第三个分组还包括了TSN为121的来自服务器的第一个DATA大块。
4. 不久，服务器发送另一个携带TSN为122的最后一个DATA大块的分组，但由于接收来

自客户端的最后的DATA大块已被确认，所以在这个分组中不包括SACK大块。

5. 最后，客户端发送一个分组，该分组包含了一个SACK大块，该SACK大块是对接收到来自服务器最后两个DATA大块的确认。

在SCTP中的确认定义了累计TSN，最后数据大块的TSN是按序接收的。

多接口数据传输 我们讨论SCTP多接口的性能，它与UDP和TCP不同的特性。多接口允许双方为了通信定义多个IP地址。但是，这些地址中只有一个地址能被定义为主地址(primary address)，其余的地址都是可选地址。在关联建立期间，定义主地址。主要的点是一个端的主地址是由另一端来确定的。换言之，源端定义目的端的主地址。

多流传递 SCTP的一个重要特性是数据传输和数据传递是不同的。SCTP使用TSN号处理数据传输，在源端与目的端之间移动数据大块。数据大块的传递是用SI和SSN来控制。SCTP支持多流，这就是说发送进程可定义不同的流，一个报文可属于这些流中某一个。每个流被赋予一个流标识符(SI)，它唯一地定义这个流。

分段 在数据传输中的另一个问题是分段(fragmentation)。虽然SCTP也用IP中分段这个术语，但IP中的分段与SCTP中的分段属于不同的级别：前者是在网络层，而后者是在传输层。

如果一个报文(封装在IP数据报中)的长度不超过路径的MTU，当这个报文创建一个DATA大块的时，SCTP保存进程到进程的报文的边界。携带一个报文的IP数据报的长度等于报文的长度(以字节为单位)加上4个开销：数据大块的头部、必需的SACK大块、SCTP通用头部和IP头部。如果总长度超过MTU，则报文需要分段。

关联终止

像TCP一样，在SCTP中交换数据双方(客户端与服务器)的任一方都可关闭连接。但与TCP不同的是，SCTP不允许有半关闭的情形。如果某一端关闭了关联，另一端必须停止发送新的数据。如果在终止请求的队列中还有数据，那么将这些数据发送，并关闭关联。关联终止(association termination)使用三个分组，如图23.35所示。注意：虽然在图中终止是由客户端发起的，但也可由服务器发起。关联终止可以有多种情况，在本章末提到的参考文献中有讨论。

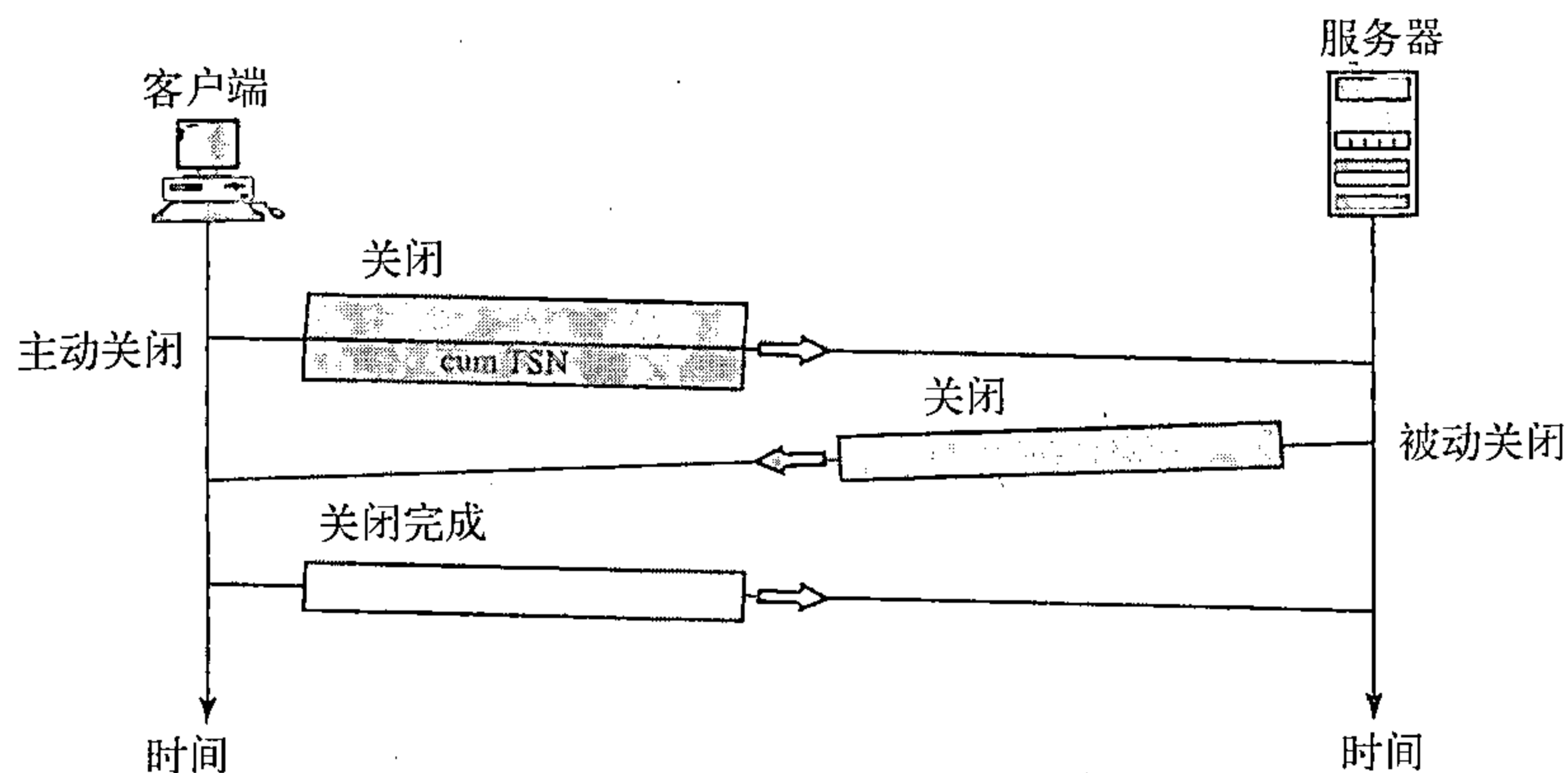


图23.35 关联终止

23.4.5 流量控制

SCTP的流量控制与TCP的流量控制相类似。在TCP中，我们需要处理仅一个数据单元，字节。在SCTP，我们需要处理两个数据单元的数据：字节和大块。*rwnd*和*cwnd*的值用字节表

示, TSN的值和确认用大块表示。为了说明这个概念, 我们做某些不实际的假定, 假设网络永远不会拥塞和永远没有差错。换言之, 我们假定 $cwnd$ 是无穷大, 分组不丢失、不延迟、不 disorder 到达。还假定数据传输是单向的。在后面几节我们将改进这些不实际的假定。关于流量控制, 现在SCTP的实现依旧是使用一个面向字节的窗口。然而, 我们为了使概念变成更加容易了解, 用大块表示缓冲区。

接收方站点

接收方有一个缓冲区(队列)和三个变量。该队列保存接收到但还未被进程读出的大块。第一个变量保存最后收到的TSN值, 即 $cumTSN$ 。第二个变量保存缓冲区大小, 即 $winSize$ 。第三个变量保存最后的累计确认值, 即 $lastACK$ 。图23.36表示接收方站点的队列和变量。

1. 当站点接收到一个数据大块时, 将它存储在缓冲区(队列)的末端并从 $winSize$ 减去该数据大块的大小。大块的TSN号存储在 $cumTSN$ 变量中;

2. 当进程读出一个大块时, 该块从队列中移去, 并在 $winSize$ 中加上移去大块的大小(循环使用);

3. 当接收方决定发送一个SACK时, 它检验 $lastACK$ 的值; 如果它小于 $cumTSN$, 接收方就发送一个具有累计TSN号等于 $cumTSN$ 的SACK, 同时该SACK也包含 $wizeSize$ 的值作为该窗口的大小。

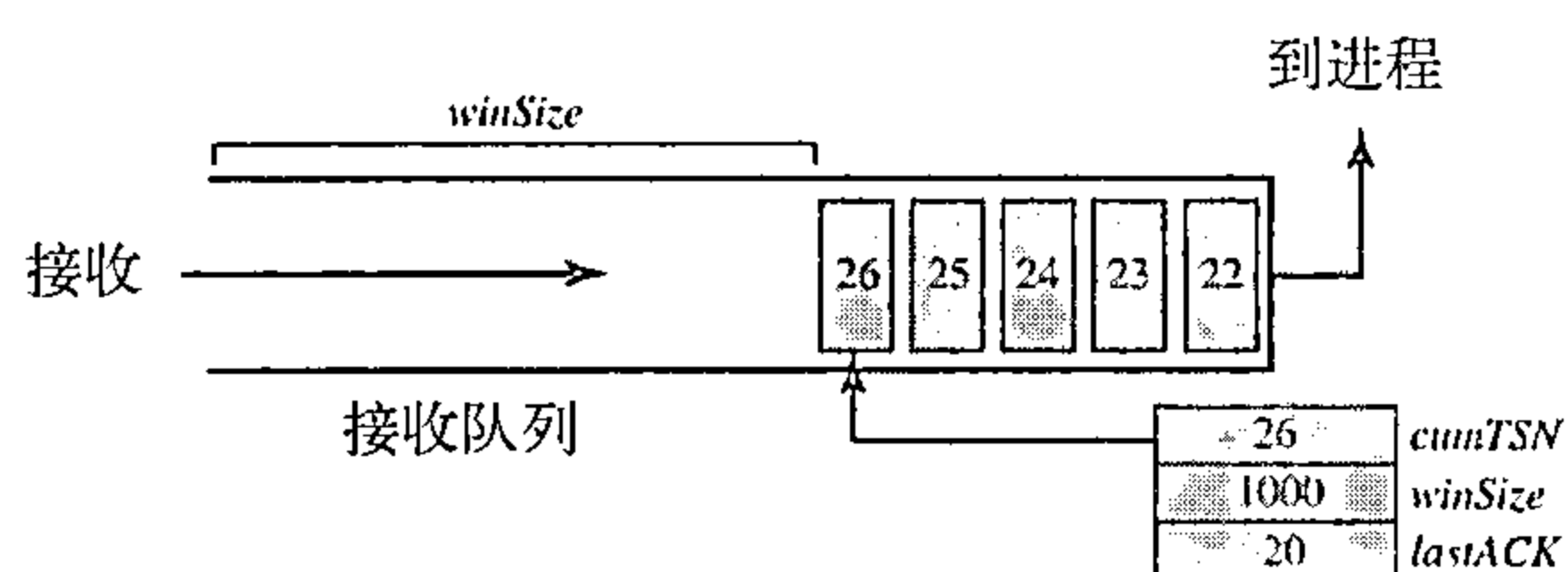


图23.36 接收方站点的流量控制

发送方站点

发送方有一个缓冲区(队列)和三个变量: $curTSN$ 、 $rwnd$ 和 $inTransit$ 。如图23.37所示。我们假定每个大块的长度为100个字节。

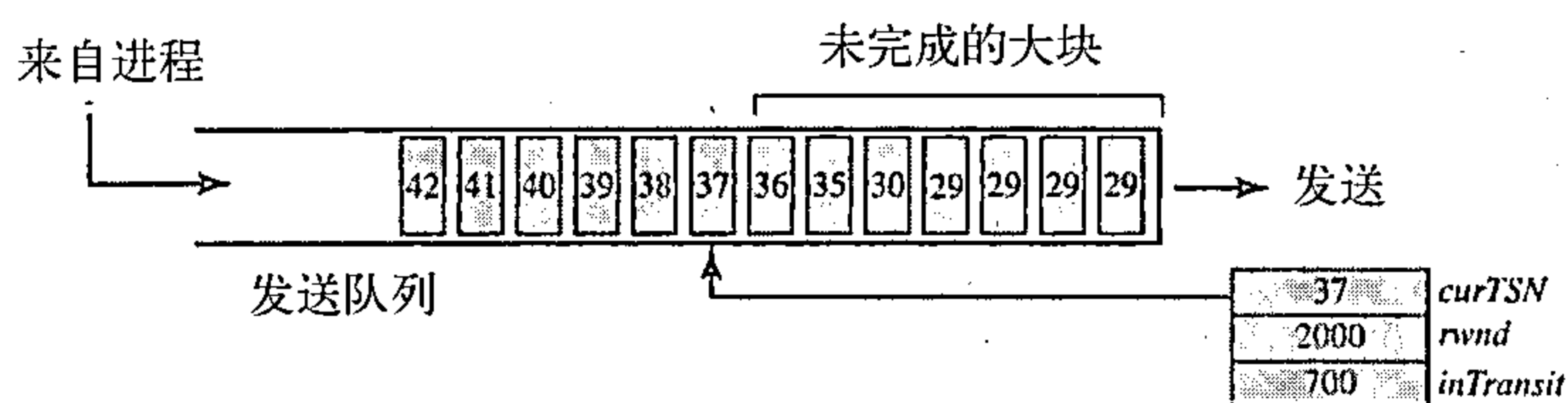


图23.37 发送方站点的流量控制

缓冲区保存由进程产生的大块, 这些大块或者已被发送或者准备发送。第一个变量 $curTSN$ 指的是下一个要发送的大块, 队列中所有小于这个值的大块都已被发送, 但还没有被确认, 它们是未完成的。第二个变量 $rwnd$ 保存接收方通告窗口的最新值(以字节为单位)。第三个变量 $inTransit$ 保存正在传输中的字节个数, 即字节已发送但还未被确认。下面是发送方所用的过程。

1. 如果 $curTSN$ 指向的大块的数据大小等于或小于 $rwnd - inTransit$ 时, 该大块可以发送。发送后, $curTSN$ 值增1并指向下一个要发送的大块。 $inTransit$ 的值加上已被发送大块的数据大小。

2. 当接收到一个SACK, 队列中具有TSN值小于或等于SACK中的TSN值的那些大块从队列中移去并丢弃。发送方不再关心它们。 $inTransit$ 值减去被丢弃块总的大小。用SACK中通告窗口的值更新 $rwnd$ 的值。

一个实例

让我们给出如图23.38所示的一个简单实例。开始时，发送方站点的 $rwnd$ 和接收方站点的 $winSize$ 都是2 000（在关联建立期间通告）。发送方队列初始有4个报文，它发送一个数据大块并在 $inTransit$ 变量中加上字节数（1 000）。之后，发送方检查 $rwnd$ 与 $inTransit$ 的差，它是1 000，所以发送方可发送另一个数据大块。现在两个变量的差为0，不能再发送数据大块。过一会儿，确认数据大块1和2的SACK到达。这两个大块从队列中移去，此时 $inTransit$ 的值是0。但是，SACK通告了接收窗口的值是0，这使得发送方将 $rwnd$ 值更新为0。现在发送方被阻塞，它不能发送任何数据大块（有一个例外稍后说明）。

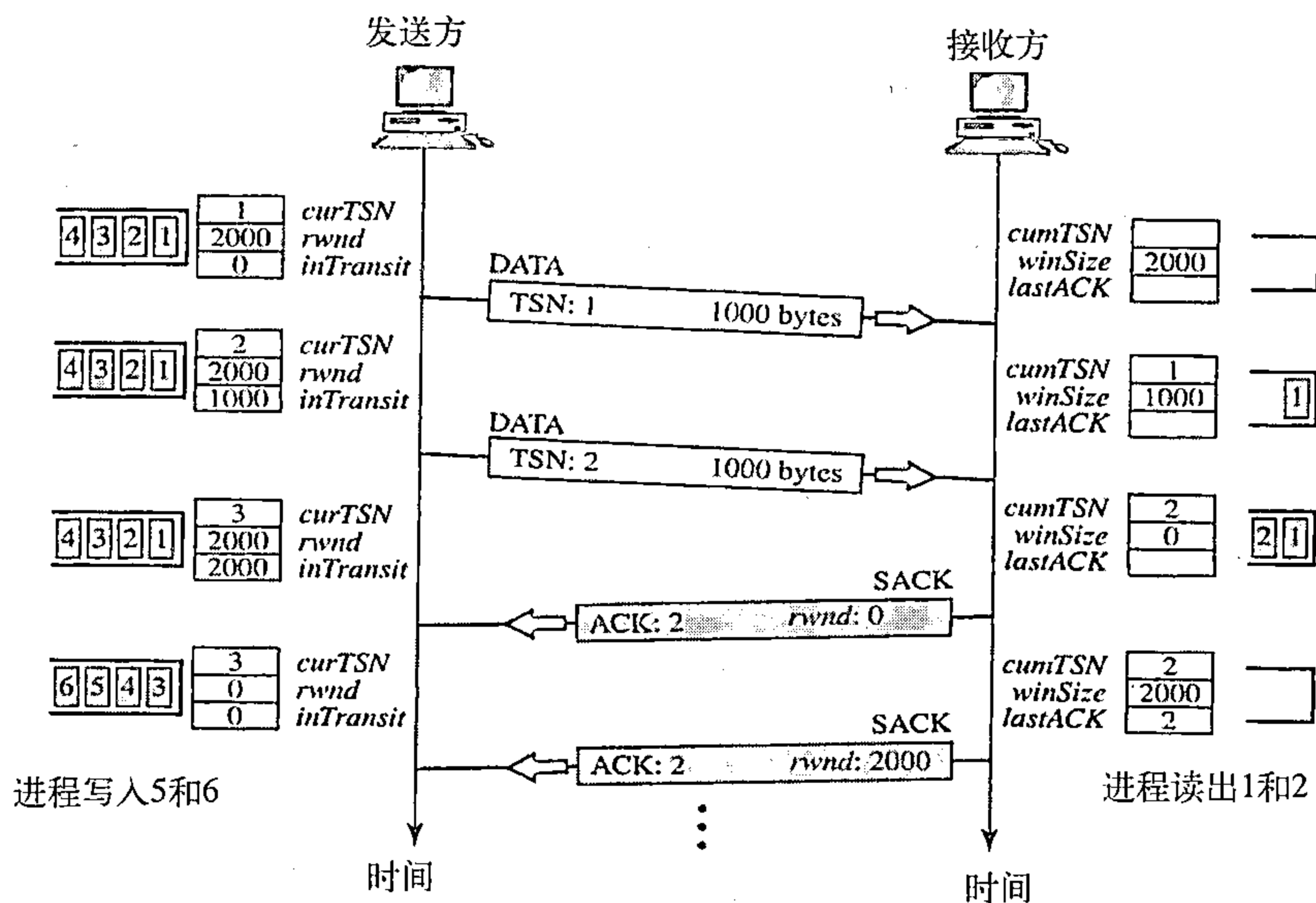


图23.38 流控制情况实例

在接收方，开始时队列是空的。当第一个数据大块接收到后，队列中有一个报文并且 $cumTSN$ 值为1。 $winSize$ 的值减小到1 000，这是因为第一个报文占用1 000个字节。接收到第二个报文后，窗口大小的值为0而 $cumTSN$ 为2。此时，我们将看到要求接收方发送累计TSN为2的SACK。第一个SACK发送后，进程读出这两个报文，这就是说现在队列有空间；接收方用SACK通告这种情况允许发送方发送更多的数据大块。余下的事件在图中没有表示。

23.4.6 差错控制

像TCP一样，SCTP是一个可靠的传输层协议。它使用一个SACK大块向发送方报告接收方缓冲区的状态。在接收方站点与发送方站点每个实现都使用不同的实体和计时器集合。我们用一个最简单设计告诉读者这个概念。

接收方站点

在我们的设计中，接收方将所有到达的大块存储在队列中，包括失序大块。但是，它给有错的大块保留空间。它丢弃那些重复的报文，但追踪它们并向发送方报告。图23.39表示了一个接收方站点的典型设计和在一个特定时间接收队列的状态。

发送的最后确认是对数据大块20，有效窗口大小是1 000个字节。大块21到23已按序收到。第一个失序的块包含大块26到28，第二个失序块包含大块31到34。一个变量保存 $cumTSN$ 的值。一组变量保存失序的那些块的首块TSN和末块TSN。一组变量保存接收到那些重复的大

块。注意：在队列中，不需要对重复的大块进行排序，它们被丢弃。图23.39中也显示了将要发送给发送方的接收方状态的SACK大块。那些失序大块的TSN号是相对于（偏移量）累计TSN的。

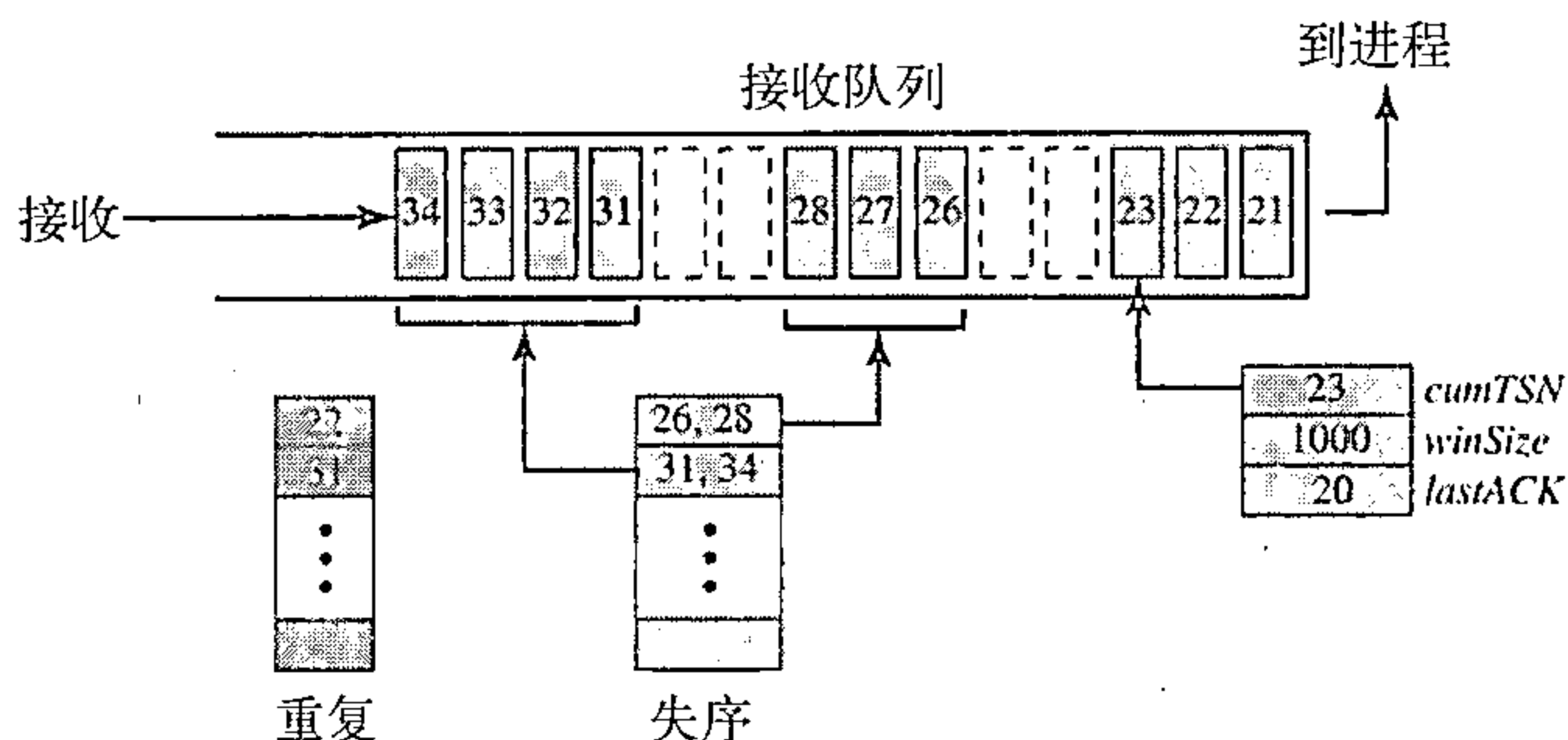


图23.39 接收方站点的差错控制

发送方站点

在发送方站点，我们的设计要求两个缓冲区（队列）：一个发送队列和一个重传队列。也使用前一节描述的三个变量 $rwnd$ 、 $inTransit$ 和 $curTSN$ 。图23.40表示了一个典型的设计。

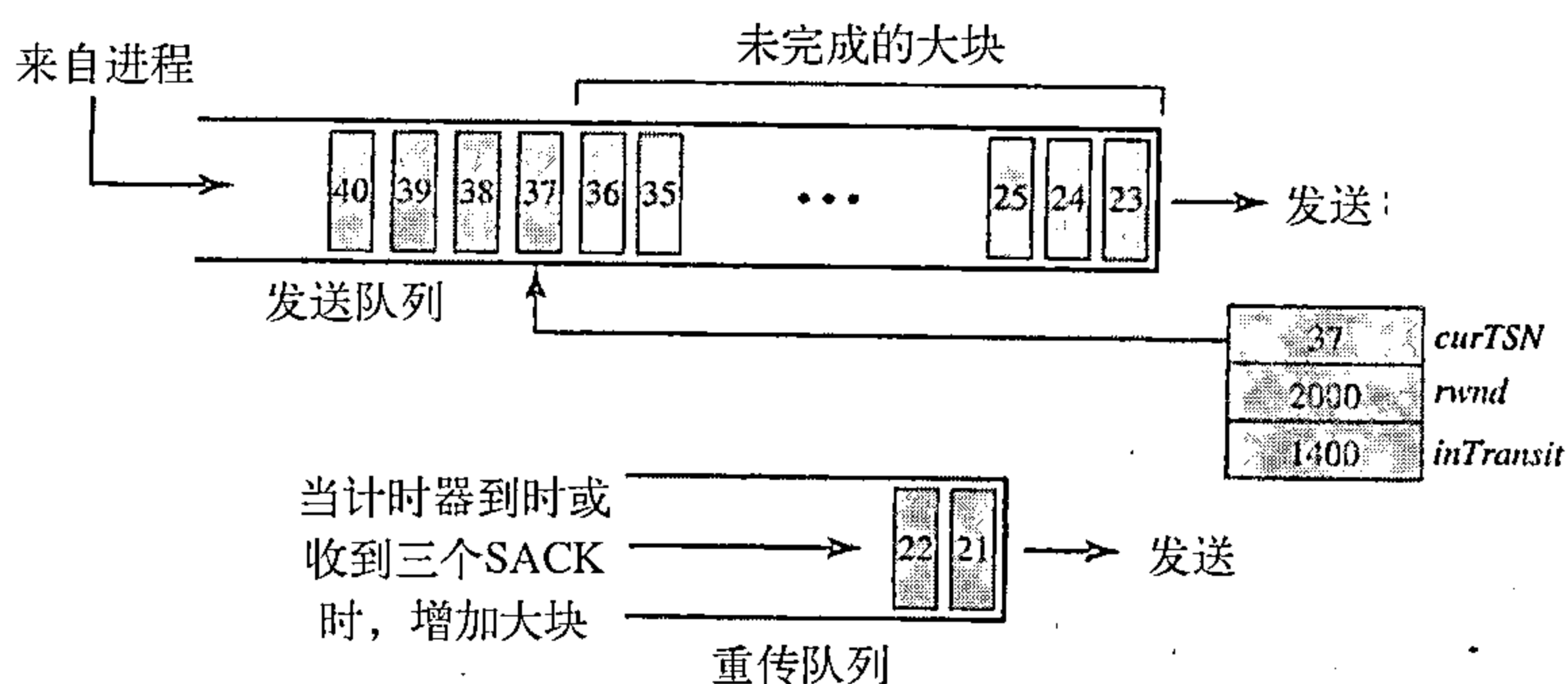


图23.40 发送方站点差错控制

发送队列保存大块23到40。其中大块23到36已发送，但还未被确认，它们是未完成的大块。 $curTSN$ 指向下一个要发送的大块（37）。假定每个大块是100个字节，这就是说数据中的1400个字节（大块23到36）是正在传输中的。这时发送方有一个重传队列，当发送一个分组时，那个分组（那个分组中的所有大块）的重传计时器开始启动。有些实现对整个关联使用一个计时器，但我们为了简单起见继续按惯例对每个分组使用一个计时器。当一个分组的重传计时器到时，或声称一个分组丢失的4个重复的SACK（在第12章中讨论过快速重传）到达时，那个分组中的所有大块都移到重传队列中被重新发送。在重传队列中的大块具有优先权。换言之，下一次发送方发送的大块将来自重传队列的大块21。

发送数据大块

每当在发送队列中有那些具有TSN的值大于或等于 $curTSN$ 的数据大块时，或者在重传队列中有数据大块时，一个端可发送一个数据大块。重传队列中的大块具有优先发送权。但是，一个分组所包含的数据大块的大小或多个数据大块总的大小必须小于或等于 $rwnd - inTransit$ ，在前几节我们已讨论过，帧的总的大小必须小于或等于MTU。

重发 像TCP一样，为了控制丢失的或丢弃的大块，SCTP采用两种策略：使用重发计时器和接收到同一丢失大块的4个SACK。

生成SACK大块

差错控制中的另一个问题是生成SACK大块。生成SCTP SACK大块的规则与TCP中使用ACK标记进行确认所用规则相似。

23.4.7 拥塞控制

像TCP一样，SCTP是一个传输层协议，其分组会遭受到网络拥塞。SCTP设计者也使用相同的策略，我们将在第24章中讨论TCP拥塞控制。SCTP具有慢速启动（指数增加），拥塞避免（渐加）和拥塞检测（倍增降低）等阶段。像TCP一样，SCTP也使用快速重传和快速恢复。

推荐读物

为了深入讨论与本章有关的主题，推荐下列读物和网站，在本书末列出方括号[……]中的参考资料。

读物

UDP在[For06]的第11章、[Ste94]的第11章和[Com00]的第12章中有讨论。TCP可参看[For06]的第12章、[Ste94]的第17到第24章和[Com00]的第13章。SCTP在[For06]的第13章和[SX02]中有讨论。UDP和TCP在[Tan03]的第6章中都有讨论。

网站

- www.ietf.org/rfc.html 有关请求评论信息。

请求评论

UDP的讨论可在RFC769中找到。

TCP的讨论可在下列请求评论中找到：

675, 700, 721, 761, 793, 879, 896, 1078, 1106, 1110, 1144, 1145, 1146, 1263, 1323, 1337, 1379, 1644, 1693, 1901, 1905, 2001, 2018, 2488, 2580

SCTP的讨论可在下列请求评论中找到：

2960, 3257, 3284, 3285, 3286, 3309, 3436, 3554, 3708, 3758

本章小结

- 在客户服务器模型中，在本地主机上的称为客户机的应用程序，需要来自远程主机上的称为服务器的应用程序上的服务。
- 每个应用程序有一个端口号，可以将同一个机器同时运行的程序区分开来。
- 给客户程序指定一个随机端口号，称为临时端口号，给服务器程序指定一个通用端口号，称为熟知端口号。
- ICANN已经指明了不同类型端口号的范围。
- IP地址与端口号的组合称为套接字地址，它定义了一个进程和一个主机。
- UDP是一个无连接不可靠的传输层协议，除了采用校验和进行差错检测外，它没有流量和差错控制机制。
- UDP分组称为用户数据报。一个用户数据报封装在IP数据报的数据字段中。
- 传输控制协议（TCP）是TCP/IP协议族中的一个传输层协议。
- TCP提供进程到进程、全双工和面向连接的服务。

- 采用TCP软件的两个设备之间传输的数据单元称为段，它有20到60个字节的头部，其后是来自应用程序的数据。
- 一个TCP连接通常包括三个阶段：连接建立、数据传输和连接终止。
- 连接建立要求三次握手，连接终止要求三次或四次握手。
- TCP使用滑动窗口机制进行流量控制避免接收方由于数据过多而崩溃。
- TCP的窗口大小由接收方通告窗口大小 (*rwnd*) 或拥塞窗口大小 (*cwnd*) 来确定，看哪一个较小。窗口可由接收方张开或合拢，但不能收缩。
- TCP在每一次连接中对传输的数据字节进行编号，初始号是随机产生的。
- TCP使用差错控制提供可靠服务。差错控制是由校验和、确认和超时来处理。受损坏的和丢失的段要重传，而重复的段应丢弃。数据可能失序到达，接收方的TCP将它们临时存储起来，但TCP保证传递给进程的是没有失序的段。
- 在现代实现中，如果重传计时器到时或三个重复ACK段已到达，则开始重发。
- SCTP是一个面向报文的、可靠的协议，它兼有UDP和TCP的特性。
- SCTP提供了UDP和TCP没有提供的服务，如多流和多接口服务。
- SCTP是一个面向连接的协议。一个SCTP连接被称为一个关联。
- SCTP使用的传输单元称为分组。
- 在SCTP中，控制信息与数据信息在不同的大块中携带。
- SCTP分组可以包含控制大块和数据大块，而控制大块是在数据大块之前。
- 在SCTP中，每个数据大块用传输序列号 (TSN) 来编号。
- 为了区别不同的流，SCTP使用序列标识符 (SI)。
- 为了区别属于同一流的不同数据大块，SCTP使用流序列号 (SSN)。
- 数据大块是用三个标识符来标识：TSN、SI和SSN。TSN是由整个关联识别的累计号，而SSN在每个流中以0开始。
- SCTP的确认号仅用来确认数据大块。如需要确认控制大块，则使用另一个控制大块。
- 通常用4个分组（四次握手）建立一个，SCTP关联使用3个分组（三次握手）终止一个关联。
- 一个SCTP关联使用cookie防止无法控制的洪泛攻击，使用确认标签防止插入攻击。
- SCTP提供流量控制、差错控制和拥塞控制。
- SCTP的确认SACK报告累计TSN、按序接收到的最后数据大块的TSN以及已接收到那些TSN。

习题

复习题

1. 在可靠性不是最重要的情况下，UDP可能是一个好的传输协议。试给出这种情况的一些例子。
2. UDP和IP的不可靠程度是否相同？为什么是或为什么不是？
3. 端口地址是否是唯一的？是或不是，为什么？为什么端口地址比IP地址短？
4. 在字典上*ephemeral*的定义是什么？它如何应用在临时端口号这个概念上？
5. UDP数据报的最小长度是多少？
6. UDP数据报的最大长度是多少？
7. 封装在UDP数据报中的进程数据的最小长度是多少？

8. 封装在UDP数据报中的进程数据的最大长度是多少?
9. 比较UDP与TCP的头部, 试列出TCP头部中那些UDP头部中没有的字段, 并说出它们没有的原因。
10. UDP是面向报文的协议, 而TCP是面向字节的协议。如果应用需要保护它的报文边界, 应该选用哪个协议, UDP还是TCP?
11. 当控制字段的值为下列的一个时, 这个TCP段的含义是什么?
 - a. 000000
 - b. 000001
 - c. 010001
12. TCP头部的最大长度是多少? TCP头部的最小长度是多少?

练习题

13. 一个UDP用户数据报携带了从TFTP客户到TFTP服务器的报文, 试给出其头部的各项。试选择适当的临时端口号和正确的熟知端口号。数据的长度是40个字节。并用图23.9的格式表示UDP分组。
14. 驻留在IP地址为122.45.12.7主机中一个SNMP客户进程发送一个报文给驻留在IP地址为200.112.45.90主机中一个SNMP服务器进程。在这次通信中所使用的套接字地址是什么?
15. 驻留在IP地址为130.45.12.7主机中一个TFTP服务器进程发送一个报文给驻留在IP地址为14.90.90.33主机中一个TFTP客户进程。在这次通信中所使用的套接字地址是什么?
16. 一个客户有一个68 000字节的分组, 试说明怎样使用UDP用户数据报将这个分组进行传输?
17. 一个客户使用UDP将数据发送给一个服务器, 数据共有16个字节。试计算在UDP级的传输效率(有用字节与总字节之比)。
18. 重做练习17, 但在IP级计算传输效率。假定IP头部无选项。
19. 重做练习18, 但在数据链路级计算传输效率。假定IP头部无选项, 数据链路层使用以太网。
20. 下面是打印出来的十六进制格式的UDP头部。
06 32 00 0D 00 1C E2 17
 - a. 源端口号是什么?
 - b. 目的端口号是什么?
 - c. 用户数据报的总长度是多少?
 - d. 数据的长度是多少?
 - e. 该分组是直接由客户发送到服务器还是从服务器发送到客户?
 - f. 客户机进程是什么?
21. 一个IP数据报携带目的地址为130.14.16.17/16的TCP段。目的端口地址受到损坏, 它到达的目的地址为130.14.16.19/16。接收方的TCP对这个差错如何反应?
22. 在TCP中, 如果HLEN值是0111, 试问这个段包含的选项有多少个字节?
23. 一个TCP报文携带了从FTP客户到FTP服务器的报文, 试给出其头部的各项。将校验和字段填入0。试选择适当的临时端口号和正确的熟知端口号。数据长度为40个字节。
24. 下面是打印出来的十六进制格式的TCP头部。
05320017 00000001 00000000 500207FF 00000000
 - a. 源端口号是什么?
 - b. 目的端口号是什么?
 - c. 序列号是什么?
 - d. 确认号是什么?
 - e. 头部长度是多少?
 - f. 段的类型是什么?
 - g. 窗口大小是多少?
25. 为了要使初始序号是成为一个随机数, 大多数的系统计数器从1开始每隔 0.5 秒自动引导增加64 000。认为计数器要环绕多久?